TRATADO DE ALGORTIMOS Y DATOS I

Desde Cero con el Lenguaje C

Nahuel Villafañe

Versión 1.0

2026

Índice general

Introducción **11**

Unidad I: Los Cimientos de la Programación en C **13**
Concepto de Algoritmo . . . 13
Concepto de Lenguaje Estructurado . . . 13
Comparación de C con otros lenguajes . . . 14
Concepto de Compiladores e Intérpretes . . . 14
El Lenguaje de Programación C . . . 15
Un vistazo a un programa básico . . . 15
Variables, Constantes y Operadores . . . 15
Concepto de Variable . . . 15
Declaración, Inicialización y Asignación . . . 15
Reglas para Identificadores . . . 16
Palabras Reservadas . . . 16
Tipos de Variables según su Alcance (Ámbito) . . . 16
Variables Locales . . . 16
Parámetros Formales . . . 16
Variables Globales . . . 16
Constantes . . . 17
Constantes Literales . . . 17
Constantes Simbólicas . . . 17
Operadores en C . . . 18
Aritméticos . . . 18
Asignación . . . 18
Incremento y Decremento . . . 18
Relacionales . . . 18
Lógicos . . . 18
Operadores de Bits . . . 18
Conversiones y Modificaciones de Tipo . . . 19
Conversión Implícita . . . 19
Conversión Explícita (Casting) . . . 19
Modificadores de Tipo . . . 19
Especificadores de Tipo de Almacenamiento . . . 19
Arreglos de Caracteres (Strings) . . . 20
Tipos Enumerativos (`enum`) . . . 20
La declaración `typedef` . . . 20

El operador `sizeof` . . . 21
Resumen de la Unidad I . . . 21

Unidad II: El Control del Flujo - Tomando Decisiones y Repitiendo Tareas 23
Las Construcciones Fundamentales del Lenguaje . . . 23
Sentencias de Bloque y Expresión . . . 23
Sentencias de Expresión . . . 23
Sentencias de Bloque . . . 24
Selección: Tomar Decisiones . . . 25
La sentencia `if` . . . 25
La sentencia `if-else` . . . 26
La sentencia `else if` . . . 27
La sentencia `switch` . . . 27
Iteración: Repetir Tareas (Bucles) . . . 29
El bucle `while` . . . 29
El bucle `do-while` . . . 30
El bucle `for` . . . 31
Sentencias de Salto . . . 32
`break` . . . 32
`continue` . . . 33
`goto` (El Gran Evitado) . . . 34
`return` . . . 34
Sentencias de Etiquetado . . . 35
Resumen de la Unidad II . . . 35

Unidad III: Arrays y Punteros - El Corazón de C 37
Arrays (Arreglos) . . . 37
Declaración e Inicialización de Arrays . . . 38
Acceso a los elementos . . . 38
Arrays Multidimensionales . . . 39
Declaración e Inicialización . . . 39
Introducción a los Punteros . . . 40
Declaración de Punteros . . . 41
Operadores de Punteros . . . 41
Asignación de Punteros . . . 42
Aritmética de Punteros . . . 42
Arrays y Punteros: Una Relación Íntima . . . 43
Punteros a Punteros . . . 44
Arrays de Punteros . . . 44
Punteros a Funciones . . . 45
Argumentos en la Línea de Comandos . . . 46
Funciones para Manejo de Memoria Dinámica . . . 46
`malloc()` . . . 47
`realloc()` . . . 47
Resumen de la Unidad III . . . 49

Unidad IV: Principios de Diseño de Algoritmos y Programas - Escribiendo Código de Calidad **51**
Principios Fundamentales de Diseño . . . 52
Denominaciones Significativas (Naming) . . . 52
Documentación y Formato (Estilo de Código) . . . 53
Refinamiento (Top-Down Design) . . . 54
Codificación y Pruebas . . . 55
Codificación . . . 55
Pruebas (Testing) . . . 55
Modularización: Divide y Vencerás . . . 56
Criterios y Principios de Modularización . . . 57
Reusabilidad . . . 58
Abstracción de Datos . . . 58
Aspectos Acerca de la Calidad del Software . . . 60
Resumen de la Unidad IV . . . 61

Unidad V: Funciones - Los Bloques de Construcción de Programas **63**
Concepto de Función . . . 63
Sintaxis Básica de una Función . . . 63
La Instrucción `return` . . . 65
Valores de Retorno . . . 66
Reglas de Alcance de Funciones . . . 67
Argumentos de Funciones . . . 67
Llamada por Valor (Pass by Value) . . . 67
Llamada por Referencia (Simulada con Punteros) . . . 68
Comparación: Por Valor vs Por Referencia . . . 69
Prototipos de Funciones . . . 69
Retorno de Punteros . . . 70
Modelos de Memoria en C . . . 72
Segmentos de Memoria . . . 73
Ejemplo de ubicación de variables . . . 73
Recursividad . . . 74
Componentes de una Función Recursiva . . . 74
Ejemplo Clásico: Factorial . . . 74
Ejemplo: Fibonacci . . . 75
Recursividad vs Iteración . . . 76
Resumen de la Unidad V . . . 76

Unidad VI: Compilación, Enlace y el Preprocesador - Construyendo Proyectos Profesionales **77**
El Proceso Completo: De Código Fuente a Ejecutable . . . 77
Fase 1: Preprocesamiento . . . 77
Fase 2: Compilación . . . 78
Fase 3: Ensamblado . . . 79
Fase 4: Enlace (Link) . . . 79
El Preprocesador de C en Profundidad . . . 80

Las Directivas del Preprocesador 80
Compilación Condicional 82
Directivas Principales 82
Usos Comunes de la Compilación Condicional 83
Archivos Cabecera (`.h`) - La Interfaz Pública 84
Qué debe ir en un archivo `.h` 84
Qué NO debe ir en un archivo `.h` 84
Ejemplo Completo de un Módulo Bien Diseñado 85
Concepto de Proyecto 86
Concepto de Makefile 87
Estructura básica de una regla en Makefile 87
Ejemplo de Makefile para nuestro proyecto 87
Concepto de Biblioteca (Librería) 88
Bibliotecas Estáticas (`.a` en Linux/Mac, `.lib` en Windows) 88
Bibliotecas Dinámicas (Compartidas) (`.so` en Linux, `.dylib` en Mac, `.dll` en Windows) 89
Bibliotecas Estándar de C 89
Biblioteca Estándar de Entrada-Salida (`<stdio.h>`) 90
Biblioteca para Manejo de Strings (`<string.h>`) 91
Otras Bibliotecas Estándar Importantes 93
Presentación Comparativa de Varios Compiladores 95
Comparativa de Mensajes de Error (Un ejemplo vale más que mil palabras) 95
Compilando con Diferentes Compiladores 96
Resumen de la Unidad VI 96

Unidad VIII: Entrada y Salida - Comunicación con el Mundo Exterior 99
El Concepto de Stream (Flujo) 99
Entrada y Salida Estándar 99
Concepto de Archivo 100
Tipos de Archivos (desde la perspectiva de C) 100
Operaciones Básicas con Archivos 101
Apertura y Cierre: `fopen()` y `fclose()` 101
Escritura en Archivos 102
Lectura de Archivos 102
La Importancia de `fclose()` 104
Funciones de Streams Avanzadas 104
Verificación de Fin de Archivo (EOF) 104
Posicionamiento en el Archivo: `ftell()`, `fseek()`, `rewind()` 105
Gestión de Errores 106
Funciones Tipo Unix de Manejo de I/O (Bajo Nivel) 106
¿Por qué usar funciones de bajo nivel? 107
Las funciones clave (en sistemas Unix/POSIX) 107
Tabla Comparativa: Funciones Estándar vs. Unix 109
Ejemplo: Copiar un archivo con funciones de bajo nivel 109
Resumen de la Unidad VIII 110

,

Abstract

Hola! Que tal? Nos encontramos de nuevo. Soy Nahuel Villafañe, un estudiante UBA Exactas, como veras y te topaste con mi nuevo apunte que hice. Espero te sirva, lo hice con mucho amor, paciencia, notas, investigacion, consultas y con muchas ganas de ayudar. Como digo siempre, mi idea es ayudar haciendo estos apuntes para las distintas carreras, porque se lo dificil que es la facultad. Ademas, lo escribi con mis propias palabras y con ayuda de la IA para verificar que este bien. Te deseo exitos en la cursada, si te sirve, no te rindas, estudiar es aprender. Uno aprende hasta perfeccionarse porque de eso se trata , se trata de aprender y ser persistente. Lo digo por experiencia propia. En fin, exitos!

Figura 1: *El secreto de la existencia humana no sólo está en vivir, sino también en saber para qué se vive* Manuel Sadosky

Introducción: El Arte de Instruir a una Máquina

Imagina que quieres enseñarle a un amigo a hacer un café. No le dices simplemente "haz un café", porque esa orden es demasiado abstracta. En cambio, le das una secuencia de pasos muy detallada:

1. Ve a la cocina.
2. Toma la cafetera.
3. Vierte agua hasta la marca de 2 tazas.
4. Coloca un filtro.
5. Añade dos cucharadas de café.
6. Presiona el botón de encendido.
7. Espera a que termine.
8. Sirve el café.

Eso es, en esencia, un **algoritmo**: una serie de pasos ordenados y finitos para resolver un problema.

Nuestro .ªmigo.en este caso es la **computadora**. Es una máquina increíblemente rápida y obediente, pero solo entiende un lenguaje muy básico de unos y ceros (lenguaje máquina). La **programación** es el puente entre nuestra capacidad para resolver problemas y la capacidad de la máquina para ejecutar órdenes.

Unidad I: Los Cimientos de la Programación en C

Concepto de Algoritmo

Un algoritmo no es solo una receta. Debe cumplir con características muy estrictas:

- **Preciso:** Cada paso debe estar definido sin ambigüedad.
- **Determinista:** Dada una misma entrada, debe producir siempre la misma salida.
- **Finito:** Debe terminar después de un número finito de pasos.

Ejemplo: Algoritmo para calcular el área de un rectángulo

1. **INICIO**
2. Leer el valor de la base (B).
3. Leer el valor de la altura (H).
4. Calcular el área: $A = B \times H$.
5. Mostrar el resultado (A).
6. **FIN**

Concepto de Lenguaje Estructurado

Un lenguaje estructurado nos permite escribir programas siguiendo una lógica clara, basada en tres estructuras fundamentales:

Estructura	**Descripción**
Secuencia	Instrucciones se ejecutan una detrás de otra, en orden.
Selección	Permite elegir entre caminos alternativos basándose en una condición (ej. `if-else`).
Iteración	Permite repetir un bloque de instrucciones mientras se cumpla una condición (ej. `while`, `for`).

Representación Gráfica

```
[Secuencia]    [Seleccion]        [Iteracion]
   Inicio          ?                  ?
     |           (Cond)            (Cond)
     V           /    \             /    \
  Paso 1        Si     No         Si      No
     |          /       \         /        \
     V        Bloque   Bloque  Bloque      |
  Paso 2        Si       No      |         |
     |          \       /         \        |
     V           \     /           \       |
  Paso 3          V   V             V      V
     |          Continuar        Continuar (vuelve)
     V
   Fin
```

Comparación de C con otros lenguajes

C es el "latín"de la programación. Muchos lenguajes modernos derivan de él.

- **C vs. Python:** Python es de alto nivel e interpretado (automático). C es de nivel medio y compilado (control total). C es más rápido y te da control del hardware, pero es más complejo.
- **C vs. Java:** Java está orientado a objetos y corre en una máquina virtual. C es más directo y procedural.
- **C vs. C++:** C++ es C con orientación a objetos y otras características añadidas. C es un subconjunto de C++.

Ventajas de C: Eficiencia, velocidad, control de memoria, portabilidad. **Desventajas de C:** Gestión manual de la memoria, curva de aprendizaje pronunciada.

Concepto de Compiladores e Intérpretes

La computadora solo entiende código máquina. Necesitamos un traductor.

- **Compilador:** Traduce todo el programa de una vez a un ejecutable. (Ej: C, C++, Go).
- **Intérprete:** Traduce y ejecuta el programa línea por línea cada vez. (Ej: Python, JavaScript).

```
[Compilador]
programa.c  ---> [COMPILADOR] ---> programa.exe  ---> [EJECUCION] ---> Resultado

[Interprete]
programa.py  ---> [INTERPRETE (ej. python)] ---> Resultado
```

El Lenguaje de Programación C

Un vistazo a un programa básico

```c
/* Este es mi primer programa. */
#include <stdio.h>  // Incluye funciones de entrada/salida

int main() {        // Funcion principal
    printf("Hola, mundo!\n"); // Imprime en pantalla
    return 0;       // Indica que termino correctamente
}
```

Listing 1: Nuestro primer programa en C

Variables, Constantes y Operadores

Concepto de Variable

Una **variable** es un espacio en memoria donde guardamos un valor que puede cambiar.

```
Nombre: edad
   +--------+
   |  25    |  <-- Valor
   +--------+
Direccion: 0x7FFF
```

- **Nombre:** La etiqueta para referirnos a ella.
- **Tipo:** Define qué datos puede guardar y cuánto espacio ocupa.
- **Valor:** El dato actual.

Declaración, Inicialización y Asignación

```c
#include <stdio.h>
int main() {
    int edad;           // Declaracion (valor "basura" ↩
        inicial)
    edad = 25;          // Asignacion

    float altura = 1.75; // Declaracion e inicializacion
    char inicial = 'N';

    edad = 26;          // Re-asignacion
    return 0;
}
```

Listing 2: Trabajando con variables

Reglas para Identificadores

- Pueden contener letras, dígitos y guión bajo (_).
- **Deben comenzar con letra o guión bajo.**
- C es **sensible a mayúsculas/minúsculas** (‘Edad‘ $\neq$ ‘edad‘).
- No se pueden usar **palabras reservadas** (‘int‘, ‘float‘, ‘return‘, etc.).

Palabras Reservadas

Palabras con significado especial para el compilador. `auto`, `break`, `case`, `char`, `const`, `continue`, `default`, `do`, `double`, `else`, `enum`, `extern`, `float`, `for`, `goto`, `if`, `int`, `long`, `register`, `return`, `short`, `signed`, `sizeof`, `static`, `struct`, `switch`, `typedef`, `union`, `unsigned`, `void`, `volatile`, `while`.

Tipos de Variables según su Alcance (Ámbito)

Variables Locales

Declaradas dentro de una función o bloque ‘‘. Solo existen allí.

```
int main() {
    int x = 10; // Local a main
    if (x > 5) {
        int y = 20; // Local al if
    }
    // y ya no existe aqui
}
```

Parámetros Formales

Variables en la definición de una función. Actúan como locales.

```
int sumar(int a, int b) { // a y b son parametros formales
    return a + b;
}
```

Variables Globales

Declaradas fuera de todas las funciones. Accesibles desde cualquier función. Su uso excesivo es mala práctica.

```
int contador = 0; // Global

void inc() {
```

```
    contador++;
}
```

```
[Ambito de Variables]
+-------------------------------------+
|  PROGRAMA                           |
|  int global = 100;                  |
|                                     |
|  +----------------------------+     |
|  | main()                      |    |
|  | int local_main = 10;        |    |
|  |  +----------------------+   |    |
|  |  | if {                 |   |    |
|  |  | int local_if = 20;   |   |    |
|  |  +----------------------+   |    |
|  +----------------------------+     |
|                                     |
|  +----------------------------+     |
|  | otra_funcion( int param )   |    |
|  | int local_otra = 30;        |    |
|  +----------------------------+     |
+-------------------------------------+
```

Constantes

Valores que no cambian.

Constantes Literales

- **Enteras:** `10`, `-5`
- **Reales:** `3.14`, `-0.5`
- **Carácter:** `'A'`, `'7'` (comillas simples)
- **Cadena:** `"Hola"` (comillas dobles)

Constantes Simbólicas

```
#define PI 3.1416  // Directiva del preprocesador

const int DIAS = 7; // Palabra clave 'const'
```

Operadores en C

Aritméticos

+	Suma
-	Resta
*	Multiplicación
/	División
%	Módulo (resto)

Nota:

Dividir enteros da entero: `5/2 = 2`. Para decimal, usa `5/2.0`.

Asignación

= asigna. Compuestos: `+=, -=, *=, /=, %=`.

```
a += 5; // Equivale a a = a + 5;
```

Incremento y Decremento

```
int x=5, y=5;
int a = x++; // a=5, x=6 (post-incremento)
int b = ++y; // y=6, b=6 (pre-incremento)
```

Relacionales

Comparan, resultado 1 (verdadero) o 0 (falso). `==, !=, >, <, >=, <=`

Lógicos

Combinan expresiones booleanas. `&&` (AND), `||` (OR), `!` (NOT).

Operadores de Bits

Actúan sobre bits. Avanzado. `&, |, ; ~, «, »`

```
Ej: 6 & 3 = 2
  6 = 00000110
  3 = 00000011
  & = 00000010 = 2
```

Conversiones y Modificaciones de Tipo

Conversión Implícita

El compilador convierte automáticamente tipos "menores.ª "mayores".

```
float r = 5 + 2.5; // 5 se convierte a 5.0 implicitamente
```

Conversión Explícita (Casting)

El programador fuerza la conversión.

```
float pi = 3.14;
int p = (int) pi;   // p = 3 (se trunca)
float div = (float) 10 / 3; // 3.333...
```

Modificadores de Tipo

- `short`, `long`: Modifican el rango de `int`.
- `signed`: Con signo (por defecto).
- `unsigned`: Solo positivos (duplica el rango positivo).

```
unsigned int cont = 0;
long long int numero_grande;
```

Especificadores de Tipo de Almacenamiento

- `auto`: Por defecto para locales (raro usarlo).
- `register`: Sugiere almacenar en registro de CPU.
- `static`: Variable local que conserva su valor entre llamadas; o variable global con visibilidad limitada al archivo.

```
void contar() {
    static int c = 0; // Se inicializa una sola vez
    c++;
}
```

Arreglos de Caracteres (Strings)

En C, un string es un arreglo de `char` que termina en `\0` (carácter nulo).

```c
#include <stdio.h>
#include <string.h>

int main() {
    char nombre1[] = "Nahuel"; // El compilador agrega '\0'
    char nombre2[20] = "Nahuel";

    printf("%s\n", nombre1);
    printf("Longitud: %zu\n", strlen(nombre1)); // strlen del <string.h>
    return 0;
}
```

```
nombre1 en memoria:
[0] [1] [2] [3] [4] [5] [6]
 N   a   h   u   e   l  \0
```

Tipos Enumerativos (`enum`)

Crea un conjunto de constantes con nombre.

```c
enum dias { LUN, MAR, MIE, JUE, VIE, SAB, DOM };

int main() {
    enum dias hoy = MIE; // hoy = 2 (por defecto empieza en 0)
    return 0;
}
```

La declaración `typedef`

Crea un alias para un tipo existente.

```c
typedef int Entero;
typedef unsigned char Byte;

int main() {
    Entero edad = 30; // Lo mismo que 'int edad'
    return 0;
}
```

El operador `sizeof`

Devuelve el tamaño en bytes de un tipo o variable.

```
#include <stdio.h>
int main() {
    int a;
    double b;
    int arr[10];

    printf("Tamanio de int: %zu bytes\n", sizeof a);
    printf("Tamanio de arr: %zu bytes\n", sizeof arr);
    printf("Elementos en arr: %zu\n", sizeof arr / sizeof ↩
        arr[0]);
    return 0;
}
```

Resumen de la Unidad I

- Un **algoritmo** es una secuencia de pasos para resolver un problema.
- C es un lenguaje **estructurado** y **compilado**.
- Las **variables** son espacios en memoria con un nombre y un tipo.
- Las **constantes** son valores fijos.
- Los **operadores** permiten manipular datos.
- El **ámbito** de una variable define dónde existe.
- Los **strings** son arreglos de char terminados en `\0`.
- `enum`, `typedef` y `sizeof` son herramientas fundamentales.

Unidad II: El Control del Flujo - Tomando Decisiones y Repitiendo Tareas

En la Unidad I, aprendimos a declarar variables y a realizar operaciones con ellas. Nuestros programas eran una simple **secuencia** de instrucciones que se ejecutaban de arriba a abajo, una detrás de otra. Para crear programas que resuelvan problemas del mundo real, necesitamos poder controlar el **flujo de ejecución**: la capacidad de tomar decisiones (elegir qué camino seguir) y de repetir bloques de código (ejecutar algo varias veces). Esta unidad está dedicada por completo a dominar estas estructuras de control.

Las Construcciones Fundamentales del Lenguaje

Como vimos en la unidad anterior, la programación estructurada se basa en tres pilares. Ahora los exploraremos a fondo en C.

1. **Secuencia**: La ejecución lineal y ordenada de instrucciones. Es el modo por defecto.
2. **Selección (o Decisión)**: Permite ejecutar un bloque de código u otro en función de una condición. Las herramientas en C son: `if`, `else if`, `else` y `switch`.
3. **Iteración (o Ciclo o Bucle)**: Permite ejecutar un bloque de código repetidamente mientras se cumpla una condición. Las herramientas en C son: `while`, `do-while` y `for`.

Además de estas, C nos proporciona sentencias para alterar el flujo de manera más fina: las **sentencias de salto** (`break`, `continue`, `goto`, `return`).

Sentencias de Bloque y Expresión

Antes de sumergirnos en las estructuras de control, debemos entender dos conceptos básicos pero cruciales.

Sentencias de Expresión

En C, la mayoría de las líneas de código son **sentencias de expresión**. Una sentencia de expresión es cualquier expresión válida en C seguida de un punto y coma (`;`). La

expresión se evalúa, pero el resultado de esa evaluación (si lo hay) se descarta. Su propósito es el **efecto secundario**, como cambiar el valor de una variable o llamar a una función.

```
#include <stdio.h>
int main() {
    int x = 5;            // Sentencia de expresion: asignacion
    x = x + 2;            // Sentencia de expresion: asignacion
    x++;                  // Sentencia de expresion: incremento
    printf("Hola");       // Sentencia de expresion: llamada a funcion
    3 + 4;                // Sentencia de expresion VALIDA pero INUTIL (no tiene efecto secundario)
    ;                     // Sentencia vacia (tambien valida)
    return 0;             // Sentencia de salto (return)
}
```

Listing 3: Sentencias de expresión

Como ves, casi todo lo que termina en `;` es una sentencia de expresión. La línea `3 + 4;` es sintácticamente correcta, pero el resultado de la suma se calcula y se pierde, por lo que no hace nada útil.

Sentencias de Bloque

Un **bloque** (o **sentencia compuesta**) es un grupo de declaraciones y sentencias encerradas entre llaves `{ }`. El bloque se considera una sola unidad sintáctica. Es decir, donde el lenguaje espera una única sentencia (por ejemplo, después de un `if`), podemos poner un bloque con muchas sentencias. Además, las variables declaradas dentro de un bloque tienen ámbito local a ese bloque.

```
#include <stdio.h>
int main() {
    int a = 10;
    printf("Valor de a fuera del bloque: %d\n", a);

    { // Inicio del bloque
        int b = 20; // b solo existe dentro de este bloque
        a = a + b;  // Podemos modificar a, que es externa
        printf("Dentro del bloque: a=%d, b=%d\n", a, b);
    } // Fin del bloque, b se destruye

    // printf("b = %d", b); // ERROR! b no existe aqui
    printf("Valor de a despues del bloque: %d\n", a);
    return 0;
}
```

Listing 4: Sentencia de bloque

Selección: Tomar Decisiones

La capacidad de tomar decisiones es lo que dota de ïnteligencia.ª nuestros programas. En C, la decisión se basa en el valor de una **expresión condicional**: si la expresión es verdadera (cualquier valor distinto de cero), se ejecuta un bloque; si es falsa (cero), se ejecuta otro (opcionalmente).

La sentencia `if`

Es la estructura de decisión más simple.

```
if (condicion) {
    // Bloque de codigo que se ejecuta SI la condicion es verdadera (distinta de cero)
}
```

Listing 5: Sintaxis de `if`

```
[Diagrama de if]
      |
      V
 +---------+
 | condicion?--No--+
 +---------+       |
      |            |
     Si            |
      |            |
      V            |
 [ Bloque if ]     |
      |            |
      +------------+
      |
      V
   (Continua)
```

```
#include <stdio.h>
int main() {
    int temperatura = 30;

    if (temperatura > 25) {
        printf("Hace calor. Prende el aire acondicionado.\n");
    }

    printf("Esta linea siempre se ejecuta.\n");
    return 0;
}
```

Listing 6: Ejemplo de `if`

La sentencia `if-else`

Amplía el `if` para ejecutar un bloque alternativo cuando la condición es falsa.

```
if (condicion) {
    // Bloque que se ejecuta si la condicion es verdadera
} else {
    // Bloque que se ejecuta si la condicion es falsa
}
```

Listing 7: Sintaxis de `if-else`

```
[Diagrama de if-else]
      |
      V
 +---------+
 | condicion?--No--+--------+
 +---------+       |        |
      |            |        |
     Si            |        |
      |            |        |
      V            |        V
 [ Bloque if ]     |   [ Bloque else ]
      |            |        |
      +------------+--------+
      |
      V
   (Continua)
```

```
#include <stdio.h>
int main() {
    int edad = 16;

    if (edad >= 18) {
        printf("Eres mayor de edad. Puedes votar.\n");
    } else {
        printf("Eres menor de edad. Aun no puedes votar.\n");
    }
    return 0;
}
```

Listing 8: Ejemplo de `if-else`

La sentencia `else if`

Cuando tenemos más de dos caminos posibles, podemos encadenar varias condiciones con `else if`. Es, en esencia, un `if` dentro del `else` de otro `if`, pero con una sintaxis más limpia.

```
if (condicion1) {
    // Bloque si condicion1 es verdadera
} else if (condicion2) {
    // Bloque si condicion1 es falsa Y condicion2 es ↩
        verdadera
} else if (condicion3) {
    // Bloque si condicion1 y condicion2 son falsas Y ↩
        condicion3 es verdadera
} else {
    // Bloque si todas las condiciones anteriores son falsas
}
```

Listing 9: Sintaxis de `else if`

```
#include <stdio.h>
int main() {
    int nota = 85;

    if (nota >= 90) {
        printf("Calificacion: A (Excelente)\n");
    } else if (nota >= 80) {
        printf("Calificacion: B (Bien)\n");
    } else if (nota >= 70) {
        printf("Calificacion: C (Suficiente)\n");
    } else {
        printf("Calificacion: D (Reprobado)\n");
    }
    return 0;
}
```

Listing 10: Ejemplo de `else if`

La sentencia `switch`

La sentencia `switch` es una forma especializada de selección múltiple. Se usa cuando queremos comparar el valor de una **única expresión** contra una lista de **constantes enteras** (o de tipo carácter, que en el fondo son enteros). Es más eficiente y legible que una larga cadena de `else if` en estos casos.

```
switch (expresion) {
    case constante1:
        // Bloque de codigo si expresion == constante1
        break; // Opcional, pero crucial
```

```
    case constante2:
        // Bloque de codigo si expresion == constante2
        break;
    // ... mas casos ...
    default:
        // Bloque de codigo si expresion no coincide con ↩
           ninguna constante
        // (opcional)
}
```

Listing 11: Sintaxis de `switch`

Puntos clave del `switch`:

- La `expresion` debe ser de un tipo entero (int, char, enum) o poder convertirse a entero.
- Cada `case` va seguido de una constante (un valor literal, no una variable).
- Al final de cada bloque de `case` se suele poner `break;`. El `break` hace que el programa salga del `switch`. Si no se pone, ocurre algo llamado **fall through** (caída): la ejecución continúa con el siguiente `case`, ignorando su condición, hasta encontrar un `break` o el final del `switch`. A veces, el *fall through* es intencional y útil.

```
#include <stdio.h>
int main() {
    char operador;
    float num1, num2, resultado;

    printf("Ingresa un operador (+, -, *, /): ");
    scanf("%c", &operador);
    printf("Ingresa dos numeros: ");
    scanf("%f %f", &num1, &num2);

    switch (operador) {
        case '+':
            resultado = num1 + num2;
            printf("Resultado: %.2f\n", resultado);
            break;
        case '-':
            resultado = num1 - num2;
            printf("Resultado: %.2f\n", resultado);
            break;
        case '*':
            resultado = num1 * num2;
            printf("Resultado: %.2f\n", resultado);
            break;
        case '/':
            if (num2 != 0) {
                resultado = num1 / num2;
```

```
                printf("Resultado: %.2f\n", resultado);
            } else {
                printf("Error: No se puede dividir por ↩
                    cero.\n");
            }
            break;
        default:
            printf("Error: Operador no valido.\n");
    }
    return 0;
}
```

Listing 12: Ejemplo de `switch`

Iteración: Repetir Tareas (Bucles)

Los bucles nos permiten ejecutar un bloque de código múltiples veces. Son la base para procesar listas de datos, realizar cálculos repetitivos y mucho más.

El bucle `while`

El bucle `while` ejecuta un bloque de código **mientras** una condición sea verdadera. La condición se evalúa **antes** de cada iteración (incluso antes de la primera). Si la condición es falsa desde el principio, el bloque no se ejecuta nunca.

```
while (condicion) {
    // Bloque de codigo a repetir
}
```

Listing 13: Sintaxis de `while`

```
[Diagrama de while]
      |
      V
 +---------+
 | condicion?--No--+
 +---------+       |
      |            |
     Si            |
      |            |
      V            |
 [ Bloque while ]  |
      |            |
      +------------+
      |
      V
   (Continua)
```

```c
#include <stdio.h>
int main() {
    int contador = 1;
    int limite = 5;

    while (contador <= limite) {
        printf("Iteracion numero: %d\n", contador);
        contador++; // ES CRUCIAL modificar la variable de la condicion para no crear un bucle infinito
    }
    printf("Bucle terminado.\n");
    return 0;
}
```

Listing 14: Ejemplo de `while`

El bucle `do-while`

El bucle `do-while` es muy similar al `while`, con una diferencia fundamental: la condición se evalúa **después** de ejecutar el bloque de código. Esto garantiza que el bloque se ejecute **al menos una vez**.

```c
do {
    // Bloque de codigo a repetir
} while (condicion); // <--- Nota el punto y coma al final
```

Listing 15: Sintaxis de `do-while`

```
[Diagrama de do-while]
      |
      V
 [ Bloque do ]  (Se ejecuta siempre al menos una vez)
      |
      V
 +---------+
 | condicion?--Si--+
 +---------+       |
      |            |
     No            |
      |            |
      V            |
   (Continua)      |
                   |
      +------------+
```

```
#include <stdio.h>
int main() {
    int numero;

    do {
        printf("Adivina el numero (del 1 al 10): ");
        scanf("%d", &numero);
    } while (numero != 7); // Se repite MIENTRAS el numero NO sea 7

    printf("Correcto! El numero era 7.\n");
    return 0;
}
```

Listing 16: Ejemplo de `do-while`

En este ejemplo, el programa siempre pide un número al menos una vez, y sigue pidiendo hasta que el usuario acierta.

El bucle `for`

El bucle `for` es el más compacto y se usa típicamente cuando sabemos de antemano cuántas iteraciones queremos realizar (por ejemplo, para recorrer un arreglo). Agrupa en una sola línea tres partes: la **inicialización**, la **condición** y la **actualización**.

```
for (inicializacion; condicion; actualizacion) {
    // Bloque de codigo a repetir
}
```

Listing 17: Sintaxis de `for`

El flujo de ejecución de un `for` es:

1. Se ejecuta la `inicializacion` (solo una vez, al principio).
2. Se evalúa la `condicion`. Si es falsa, se sale del bucle.
3. Si es verdadera, se ejecuta el `bloque de codigo`.
4. Se ejecuta la `actualizacion`.
5. Se vuelve al paso 2.

```
[Diagrama de for]
      |
      V
 [ Inicializacion ]
      |
      V
 +---------+
```

```
| condicion?--No--+
+---------+       |
     |            |
    Si            |
     |            |
     V            |
[ Bloque for ]    |
     |            |
     V            |
[ Actualizacion ] |
     |            |
     +------------+
     |
     V
  (Continua)
```

```
#include <stdio.h>
int main() {
    int i;

    for (i = 1; i <= 5; i++) {
        printf("Iteracion numero: %d\n", i);
    }

    printf("Bucle terminado. i vale: %d\n", i); // i vale 6
    return 0;
}
```

Listing 18: Ejemplo de `for`

Este ejemplo produce exactamente la misma salida que el ejemplo de `while` con contador, pero es mucho más conciso.

Sentencias de Salto

Estas sentencias nos permiten transferir el control del programa a otro punto, "saltando"por encima de la ejecución lineal.

`break`

Ya lo vimos en `switch`. Dentro de un bucle (`while`, `do-while`, `for`), `break` provoca la **salida inmediata** del bucle más interno que lo contiene. La ejecución continúa en la primera línea después del bucle.

```
#include <stdio.h>
int main() {
    int i;
```

```
    for (i = 1; i <= 10; i++) {
        if (i == 5) {
            printf("Encontre el 5! Saliendo del bucle.\n");
            break; // El bucle termina aqui
        }
        printf("Valor de i: %d\n", i);
    }
    printf("Bucle terminado en i=%d\n", i); // i=5
    return 0;
}
```

Listing 19: Ejemplo de `break`

Este programa imprimirá los valores del 1 al 4, luego el mensaje de ."Encontre el 5", y luego "Bucle terminado en i=5". Las iteraciones del 6 al 10 nunca ocurren.

`continue`

La sentencia `continue` solo puede usarse dentro de bucles. Hace que se **omit** el resto del cuerpo del bucle para la iteración actual y que el programa pase directamente a la siguiente iteración (evaluando la condición y, en el caso del `for`, ejecutando la actualización).

```
#include <stdio.h>
int main() {
    int i;
    for (i = 1; i <= 5; i++) {
        if (i == 3) {
            printf("Saltando la iteracion %d...\n", i);
            continue; // Salta al incremento i++ y luego ↩
                a la condicion
        }
        printf("Valor de i: %d\n", i);
    }
    return 0;
}
```

Listing 20: Ejemplo de `continue`

Este programa imprimirá:

```
Valor de i: 1
Valor de i: 2
Saltando la iteracion 3...
Valor de i: 4
Valor de i: 5
```

Observa que cuando `i` vale 3, la línea de `printf` inferior se omite y el bucle continúa con la siguiente iteración.

`goto` (El Gran Evitado)

La sentencia `goto` permite saltar incondicionalmente a una **etiqueta** en cualquier lugar de la misma función. Es una reliquia de la programación desestructurada y su uso está **muy desaconsejado** porque crea código .espagueti", difícil de leer y mantener. Casi siempre hay una forma mejor y más clara de hacer lo mismo con las estructuras de control vistas. Lo mencionamos solo por completitud.

```c
#include <stdio.h>
int main() {
    int x = 0;

    inicio: // Esta es una etiqueta
    printf("x = %d\n", x);
    x++;
    if (x < 3) {
        goto inicio; // Salta de nuevo a la etiqueta 'inicio'
    }
    printf("Fin.\n");
    return 0;
}
```

Listing 21: Ejemplo de `goto` (No lo hagas en casa)

`return`

La sentencia `return` termina la ejecución de la función actual y devuelve el control a la función que la llamó (en el caso de `main`, se la devuelve al sistema operativo). Puede devolver un valor (que debe coincidir con el tipo de retorno de la función) o puede usarse sin valor en funciones declaradas como `void`.

```c
#include <stdio.h>

int sumar(int a, int b) {
    int resultado = a + b;
    return resultado; // Devuelve el valor y termina la funcion 'sumar'
}

void saludar() {
    printf("Hola, mundo!\n");
    return; // Opcional en funciones void. Termina la funcion.
    printf("Esta linea nunca se ejecutara.\n");
}

int main() {
    int s = sumar(5, 3);
```

```
    printf("La suma es: %d\n", s);
    saludar();
    return 0; // Termina main y el programa
}
```

Listing 22: Ejemplo de `return`

Sentencias de Etiquetado

En C, podemos poner una **etiqueta** a una sentencia. Una etiqueta es un identificador seguido de dos puntos (`:`). Su único uso práctico en C moderno es como destino de un `goto` (que ya vimos que es mala práctica) o dentro de la sentencia `switch` (con las etiquetas `case` y `default`), que ya hemos visto. Por lo tanto, no profundizaremos más en ellas, ya que su aplicación principal ya la hemos cubierto con `switch`.

Resumen de la Unidad II

Hemos cubierto las herramientas que nos permiten escribir programas con una lógica compleja y no meramente lineal.

- **Secuencia:** La base de todo, instrucción tras instrucción.
- **Selección:** Usamos `if`, `else if` y `else` para decisiones basadas en cualquier expresión booleana. Usamos `switch` para comparar una expresión con múltiples constantes enteras, prestando atención al `break` para evitar *fall through* no deseado.
- **Iteración:**
  - `while`: Repite mientras una condición sea verdadera (evalúa al inicio).
  - `do-while`: Repite mientras una condición sea verdadera, pero garantiza al menos una ejecución (evalúa al final).
  - `for`: Ideal para repeticiones con un contador, agrupa inicialización, condición y actualización.
- **Sentencias de Salto:**
  - `break`: Sale inmediatamente de un bucle o `switch`.
  - `continue`: Salta al final del cuerpo del bucle para la iteración actual y pasa a la siguiente.
  - `return`: Termina la función actual y devuelve un valor (si corresponde).
  - `goto`: Salta a una etiqueta. Su uso está desaconsejado.
- **Sentencias de Bloque y Expresión**: Un bloque (`{}`) agrupa sentencias. Una sentencia de expresión es cualquier expresión seguida de `;`.

Con estas herramientas, ya podemos escribir programas que realmente resuelvan problemas: podemos procesar listas de números, validar entradas de usuario, y crear una lógica de decisión compleja. En la siguiente unidad, abordaremos uno de los temas más poderosos y distintivos de C: los **arrays y punteros**, que nos darán un control aún más fino sobre la memoria y los datos.

Unidad III: Arrays y Punteros - El Corazón de C

Bienvenido a la unidad más crucial y, a menudo, la más desafiante del lenguaje C. Los **arrays** (arreglos) y los **punteros** son las herramientas que le dan a C su increíble poder y flexibilidad para manejar la memoria de manera directa. Dominar estos conceptos es lo que separa a un programador de C casual de un verdadero experto.

Imaginemos la memoria RAM como una larga calle con muchas casas. Cada casa tiene una dirección única (su **dirección de memoria**) y dentro de cada casa podemos guardar un dato (el **valor** de una variable). Hasta ahora, trabajábamos con variables individuales, que son como casas aisladas. Los arrays nos permiten tener una **hilera de casas contiguas**, todas del mismo tamaño, y los punteros nos permiten crear **carteles** que contienen la dirección de una casa (o de la primera de una hilera). Con un puntero, podemos "señalar.ª una variable y luego movernos por la memoria.

Arrays (Arreglos)

Un **array** es una colección de elementos del mismo tipo, almacenados en posiciones de memoria **contiguas**. Piensa en él como una fila de casilleros numerados.

Representación de un Array en Memoria

```
int edades[5];  // Un array de 5 enteros

Direccion de memoria (ejemplo):
+---------+---------+---------+---------+---------+
| edades[0] | edades[1] | edades[2] | edades[3] | edades[4] |
+---------+---------+---------+---------+---------+
0x100     0x104     0x108     0x10C     0x110
(Asumiendo que int ocupa 4 bytes en este sistema)
```

Características clave:

- Todos los elementos son del mismo tipo (definido en la declaración).
- Ocupan bloques de memoria contiguos.
- Cada elemento se accede mediante un **índice**. El primer elemento tiene índice **0**.
- El nombre del array (`edades`, en el ejemplo) es un puntero **constante** a la dirección del primer elemento (&edades[0]).

Declaración e Inicialización de Arrays

Declaración

La sintaxis es: `tipo nombre[tamaño];`

```
int numeros[10];          // Array de 10 enteros (indices 0 ↩
    a 9)
float promedios[5];       // Array de 5 floats
char nombre[50];          // Array de 50 caracteres (para ↩
    un string)
```

Listing 23: Declaración de arrays

Inicialización

Podemos inicializar un array al momento de declararlo de varias formas.

```
#include <stdio.h>

int main() {
    // Inicializacion completa
    int a[5] = {10, 20, 30, 40, 50};
    // a[0]=10, a[1]=20, a[2]=30, a[3]=40, a[4]=50

    // Inicializacion parcial: los elementos no ↩
        inicializados se ponen a 0
    int b[5] = {1, 2}; // b = {1, 2, 0, 0, 0}

    // Inicializacion sin especificar tama o (el ↩
        compilador lo cuenta)
    int c[] = {5, 10, 15}; // c tendra tama o 3

    // Inicializacion con designadores (C99 en adelante)
    int d[10] = { [2]=100, [5]=200 };
    // d = {0, 0, 100, 0, 0, 200, 0, 0, 0, 0}

    // Inicializacion a cero
    int e[10] = {0}; // Todos los elementos a 0

    return 0;
}
```

Listing 24: Inicialización de arrays

Acceso a los elementos

Se usa el nombre del array y el índice entre corchetes `[]`.

```c
#include <stdio.h>
int main() {
    int notas[4] = {85, 90, 78, 95};

    printf("La primera nota es: %d\n", notas[0]); // notas[0] = 85
    printf("La tercera nota es: %d\n", notas[2]); // notas[2] = 78

    // Modificar un elemento
    notas[1] = 92;
    printf("La segunda nota ahora es: %d\n", notas[1]);

    // Recorrer un array con un bucle
    int suma = 0;
    for (int i = 0; i < 4; i++) {
        suma += notas[i];
    }
    printf("La suma de las notas es: %d\n", suma);
    return 0;
}
```

Listing 25: Acceso a elementos de un array

Arrays Multidimensionales

Podemos tener arrays de más de una dimensión. Un array bidimensional es como una tabla con filas y columnas, o una matriz. En memoria, se almacenan por filas (fila 0 completa, luego fila 1, etc.).

Representación de un Array Bidimensional

```
int matriz[3][4];  // 3 filas, 4 columnas

Visualmente:
        Col 0  Col 1  Col 2  Col 3
Fila 0: [0,0]  [0,1]  [0,2]  [0,3]
Fila 1: [1,0]  [1,1]  [1,2]  [1,3]
Fila 2: [2,0]  [2,1]  [2,2]  [2,3]

En memoria (contiguo):
[0,0] [0,1] [0,2] [0,3] [1,0] [1,1] [1,2] [1,3] [2,0] [2,1] [2,2] [2,3]
```

Declaración e Inicialización

```c
#include <stdio.h>
int main() {
    // Declaracion e inicializacion de una matriz 2x3
    int matriz[2][3] = {
        {1, 2, 3},   // Fila 0
        {4, 5, 6}    // Fila 1
    };

    // Otra forma de inicializar (menos legible)
    int matriz2[2][3] = {1, 2, 3, 4, 5, 6};

    // Inicializacion parcial (los no inicializados a 0)
    int matriz3[2][3] = { {1}, {4, 5} };
    // Resultado: {{1,0,0}, {4,5,0}}

    // Acceso
    printf("Elemento en fila 1, columna 2: %d\n", ↩
       matriz[1][2]); // 6

    // Recorrido con bucles anidados
    for (int i = 0; i < 2; i++) {        // i = fila
        for (int j = 0; j < 3; j++) {    // j = columna
            printf("%d ", matriz[i][j]);
        }
        printf("\n"); // Nueva linea al final de cada fila
    }
    return 0;
}
```

Listing 26: Arrays multidimensionales

Introducción a los Punteros

Un **puntero** es una variable que **almacena una dirección de memoria**. Es decir, en lugar de guardar un valor como 25, guarda la dirección donde está guardado el 25.

Comparación: Variable Normal vs. Puntero

```
Variable normal 'edad':
+--------+
|   25   |
+--------+
0x7FFF   (direccion)
```

```
Puntero 'ptr':
 +----------+
 | 0x7FFF   |  (almacena una direccion)
 +----------+
  0x7FF0   (su propia direccion)
      |
      | apunta a
      V
  +--------+
```

```
|   25   |
+--------+
0x7FFF   (direccion de edad)
```

Declaración de Punteros

Se declara usando el símbolo * antes del nombre de la variable. El tipo del puntero indica a qué tipo de dato apunta.

```c
int *ptr_int;       // Puntero a un entero
float *ptr_float;   // Puntero a un float
char *ptr_char;     // Puntero a un caracter (muy usado para strings)
void *ptr_void;     // Puntero generico (puede apuntar a cualquier tipo)
```

Listing 27: Declaración de punteros

Operadores de Punteros

Hay dos operadores fundamentales para trabajar con punteros:

- **& (operador de dirección)**: Devuelve la dirección de memoria de una variable.
- *** (operador de indirección o desreferencia)**: Dado un puntero, accede al valor almacenado en la dirección a la que apunta.

```c
#include <stdio.h>
int main() {
    int edad = 25;          // Variable normal
    int *ptr;               // Declaramos un puntero a entero

    ptr = &edad;            // ptr ahora contiene la direccion de edad

    printf("Valor de edad: %d\n", edad);            // 25
    printf("Direccion de edad: %p\n", &edad);       // 0x7fff...
    printf("Valor de ptr (direccion que guarda): %p\n", ptr); // misma direccion
    printf("Valor al que apunta ptr: %d\n", *ptr); // 25 (desreferencia)

    // Modificar edad a traves del puntero
    *ptr = 30;              // Cambia el valor en la direccion apuntada por ptr

    printf("Nuevo valor de edad: %d\n", edad);      // 30
```

```
    return 0;
}
```

Listing 28: Uso de & y *

Asignación de Punteros

Podemos asignar a un puntero la dirección de otra variable, o el valor de otro puntero del mismo tipo.

```
int x = 10, y = 20;
int *p1, *p2;

p1 = &x;     // p1 apunta a x
p2 = p1;     // p2 ahora apunta a donde apunta p1, es decir, a x
p1 = &y;     // p1 ahora apunta a y, p2 sigue apuntando a x

// CUIDADO: No confundir * en declaracion con * en asignacion
*p2 = 30;    // Cambia x a 30 (desreferencia p2)
```

Listing 29: Asignación de punteros

Aritmética de Punteros

Esta es una de las características más potentes de C. Podemos realizar operaciones aritméticas con punteros: suma, resta, incremento, decremento. **No es aritmética normal de enteros**. Cuando sumamos 1 a un puntero, este avanza al siguiente elemento del tipo al que apunta, no al siguiente byte.

```
#include <stdio.h>
int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int *p = arr;  // p apunta al primer elemento (arr[0])

    printf("p apunta a: %d\n", *p);        // 10

    p++;  // Ahora p apunta al siguiente entero (arr[1])
    printf("p++ apunta a: %d\n", *p);      // 20

    p = p + 2;  // Ahora p apunta 2 posiciones mas alla (arr[3])
    printf("p+2 apunta a: %d\n", *p);      // 40

    // Podemos recorrer el array con un puntero
    printf("\nRecorriendo el array con punteros:\n");
```

```
    for (p = arr; p < arr + 5; p++) {
        printf("%d ", *p);
    }
    printf("\n");
    return 0;
}
```

Listing 30: Aritmética de punteros

Visualización de la aritmética de punteros

```
int arr[5] = {10, 20, 30, 40, 50};
Memoria (cada bloque son 4 bytes asumiendo int de 32 bits):

arr:  [10] [20] [30] [40] [50]
dir:  100  104  108  112  116

int *p = arr;    // p = 100
p++;             // p = 104 (NO 101)
p = p + 2;       // p = 112 (104 + 2*4)
```

La regla general es: `ptr + n` avanza `n * sizeof(tipo_al_que_apunta)` bytes.

Arrays y Punteros: Una Relación Íntima

En C, el nombre de un array es un **puntero constante al primer elemento**. Es decir, no podemos cambiar a dónde apunta (no podemos hacer `arr++`), pero podemos usar aritmética de punteros para acceder a sus elementos.

```
#include <stdio.h>
int main() {
    int arr[5] = {10, 20, 30, 40, 50};

    // Son equivalentes:
    printf("%d\n", arr[2]);      // Notacion de indice
    printf("%d\n", *(arr + 2)); // Notacion de punteros

    // Para recorrer el array:
    int *p;
    for (p = arr; p < arr + 5; p++) {
        printf("%d ", *p);
    }
    printf("\n");

    // arr++ es ILEGAL (arr es constante)
    // arr = p;  // ILEGAL
    return 0;
}
```

Listing 31: Relación array-puntero

Punteros a Punteros

Un puntero puede apuntar a otro puntero. Es decir, una variable que almacena la dirección de otra variable que también es un puntero. Se usa mucho en matrices de strings y para modificar punteros desde funciones.

Visualización de puntero a puntero

```
int x = 10;
int *p = &x;       // p apunta a x
int **pp = &p;     // pp apunta a p

pp  --->  p  --->  x
(0x200)  (0x100)  (0x50)
```

```c
#include <stdio.h>
int main() {
    int x = 10;
    int *p = &x;
    int **pp = &p;

    printf("x = %d\n", x);
    printf("*p = %d\n", *p);
    printf("**pp = %d\n", **pp); // Accedemos al valor de x

    **pp = 20; // Modificamos x a traves de pp
    printf("x ahora es: %d\n", x);
    return 0;
}
```

Listing 32: Puntero a puntero

Arrays de Punteros

Podemos tener arrays cuyos elementos son punteros. Es muy común para manejar listas de strings.

```c
#include <stdio.h>
int main() {
    // Array de 3 punteros a char (strings)
    char *nombres[] = {"Ana", "Carlos", "Maria"};

```

```
    // nombres[0] es un puntero al string "Ana"
    // nombres[1] es un puntero al string "Carlos", etc.

    for (int i = 0; i < 3; i++) {
        printf("Nombre %d: %s\n", i, nombres[i]);
    }

    // Tambien podemos tener arrays de punteros a enteros
    int a = 1, b = 2, c = 3;
    int *arr_ptr[] = {&a, &b, &c};

    printf("Valor apuntado por arr_ptr[1]: %d\n", ↩
       *arr_ptr[1]); // 2
    return 0;
}
```

Listing 33: Array de punteros

Punteros a Funciones

Sí, ¡las funciones también tienen dirección! Un puntero a función almacena la dirección de una función, permitiendo pasarla como argumento a otra función o almacenarla en estructuras. Es avanzado pero muy útil.

```
#include <stdio.h>

// Dos funciones con la misma firma (tipo de retorno y ↩
   parametros)
int sumar(int a, int b) { return a + b; }
int restar(int a, int b) { return a - b; }

int main() {
    // Declaracion de un puntero a funcion que recibe dos ↩
       ints y devuelve un int
    int (*operacion)(int, int);

    operacion = &sumar; // o simplemente = sumar (el & es ↩
       opcional)
    printf("Suma: %d\n", operacion(10, 5));

    operacion = restar;
    printf("Resta: %d\n", operacion(10, 5));

    // Ejemplo mas util: pasar una funcion como parametro
    // (Lo veremos en detalle mas adelante)
    return 0;
}
```

Listing 34: Punteros a funciones

Argumentos en la Línea de Comandos

¿Has notado que programas como `ls -l` reciben argumentos? En C, la función `main` puede recibir parámetros para procesar estos argumentos.

```c
#include <stdio.h>

// argc: numero de argumentos (argument count)
// argv: array de strings con los argumentos (argument vector)
int main(int argc, char *argv[]) {
    printf("Numero de argumentos: %d\n", argc);
    printf("Nombre del programa: %s\n", argv[0]); // argv[0] es el nombre del ejecutable

    for (int i = 1; i < argc; i++) {
        printf("Argumento %d: %s\n", i, argv[i]);
    }
    return 0;
}
```

Listing 35: Argumentos de línea de comandos

Si compilamos este programa como `programa` y lo ejecutamos como:

```
./programa hola mundo 123
```

La salida será:

```
Numero de argumentos: 4
Nombre del programa: ./programa
Argumento 1: hola
Argumento 2: mundo
Argumento 3: 123
```

Funciones para Manejo de Memoria Dinámica

Hasta ahora, toda la memoria que usábamos era **estática** o **automática** (en la pila). Sabíamos de antemano el tamaño de los arrays. Pero, ¿y si no sabemos cuántos datos va a ingresar el usuario hasta que el programa se está ejecutando? Necesitamos **memoria dinámica**: memoria que podemos solicitar y liberar en tiempo de ejecución desde el **montón** (heap).

Las funciones para esto están en `<stdlib.h>`.

malloc()

`malloc` (memory allocation) reserva un bloque de memoria del tamaño especificado (en bytes) y devuelve un puntero `void*` al inicio de ese bloque. Si no hay suficiente memoria, devuelve `NULL`.

```
#include <stdio.h>
#include <stdlib.h> // Para malloc, free, etc.

int main() {
    int n;
    printf("Cuantos numeros quieres guardar? ");
    scanf("%d", &n);

    // Reservamos memoria para n enteros
    int *arr = (int*) malloc(n * sizeof(int));

    // SIEMPRE verificar que malloc no haya fallado
    if (arr == NULL) {
        printf("Error: No se pudo reservar memoria.\n");
        return 1; // Salida con error
    }

    // Usamos el array como si fuera normal
    for (int i = 0; i < n; i++) {
        arr[i] = i * 10;
    }

    for (int i = 0; i < n; i++) {
        printf("arr[%d] = %d\n", i, arr[i]);
    }

    // IMPORTANTE: Liberar la memoria cuando ya no la ↩
       necesitemos
    free(arr);

    return 0;
}
```

Listing 36: Uso de `malloc`

realloc()

`realloc` (reallocation) cambia el tamaño de un bloque de memoria previamente asignado con `malloc`, `calloc` o `realloc`. Puede agrandarlo o reducirlo. Si es posible, intenta mantener los datos existentes.

```
#include <stdio.h>
#include <stdlib.h>
```

```c

int main() {
    // Empezamos con espacio para 5 enteros
    int *arr = (int*) malloc(5 * sizeof(int));
    if (arr == NULL) { /* manejar error */ }

    for (int i = 0; i < 5; i++) {
        arr[i] = i;
    }

    printf("Array original:\n");
    for (int i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    // Ahora necesitamos espacio para 10 enteros
    int *temp = (int*) realloc(arr, 10 * sizeof(int));

    if (temp == NULL) {
        // realloc fallo! arr sigue siendo valido, pero ↩
           no tenemos mas espacio
        printf("Error al redimensionar.\n");
        free(arr); // Liberamos la memoria original
        return 1;
    } else {
        arr = temp; // Asignamos el nuevo bloque (puede ↩
           ser la misma direccion u otra)
    }

    // Inicializamos los nuevos elementos
    for (int i = 5; i < 10; i++) {
        arr[i] = i * 10;
    }

    printf("Array redimensionado:\n");
    for (int i = 0; i < 10; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    free(arr); // Liberamos la memoria
    return 0;
}
```

Listing 37: Uso de `realloc`

Otras funciones relacionadas:

- `calloc()`: Similar a `malloc`, pero inicializa todos los bytes a 0. Recibe el número

de elementos y el tamaño de cada uno.

- `free()`: Libera la memoria asignada dinámicamente. Es crucial para evitar **fugas de memoria** (memory leaks).

Resumen de la Unidad III

Hemos cubierto el tema más fundamental de C:

- **Arrays**: Colecciones de elementos del mismo tipo en memoria contigua. Se acceden por índice.
- **Arrays multidimensionales**: Arrays de arrays, como matrices.
- **Punteros**: Variables que guardan direcciones de memoria. Operadores `&` (dirección) y `*` (desreferencia).
- **Aritmética de punteros**: Sumar/restar a un puntero avanza según el tamaño del tipo al que apunta.
- **Relación arrays-punteros**: El nombre de un array es un puntero constante al primer elemento. `arr[i]` es equivalente a `*(arr + i)`.
- **Punteros a punteros**: Útiles para matrices de strings y modificar punteros en funciones.
- **Arrays de punteros**: Muy usado para listas de strings.
- **Punteros a funciones**: Permiten pasar funciones como argumentos.
- **Argumentos de línea de comandos**: `argc` y `argv` en `main`.
- **Memoria dinámica**: `malloc`, `realloc`, `free` para gestionar memoria en tiempo de ejecución.

Esta unidad es la base para todo lo que sigue. Tómate tu tiempo para asimilarla, experimenta con los ejemplos y, sobre todo, **¡practica mucho!** La comprensión de punteros es la clave del dominio de C.

Unidad IV: Principios de Diseño de Algoritmos y Programas - Escribiendo Código de Calidad

Hasta ahora, nos hemos enfocado en aprender la sintaxis de C y en resolver problemas específicos. Sin embargo, en el mundo real, los programas no son solo ejercicios aislados; son sistemas complejos que evolucionan con el tiempo, son mantenidos por diferentes personas y deben ser confiables. Esta unidad trata sobre las **buenas prácticas** y los **principios fundamentales** que transforman un código que "funciona.^en un código **de calidad**, que es fácil de entender, modificar y mantener.

El Ciclo de Vida del Software (Visión Simplificada)

```
+-------------+
|  ANALISIS   |  (¿Qué problema resolver?)
+-------------+
       |
       V
+-------------+
|   DISEÑO    |  (¿Cómo lo vamos a resolver? - ¡ESTA UNIDAD!)
+-------------+
       |
       V
+-------------+
| CODIFICACION|  (Escribir el código)
+-------------+
       |
       V
+-------------+
|   PRUEBAS   |  (Verificar que funciona)
+-------------+
       |
       V
+-------------+
| MANTENIMIENTO| (Arreglar bugs, añadir features)
+-------------+
```

Principios Fundamentales de Diseño

Diseñar un programa no es sentarse y empezar a teclear código. Es un proceso de reflexión y planificación. Los siguientes principios nos guían en este proceso.

Denominaciones Significativas (Naming)

El nombre que le damos a las variables, funciones, estructuras, etc., es la primera forma de documentación. Un buen nombre explica por sí mismo el propósito del elemento.

```c
#include <stdio.h>

// MAL: Nombres cr pticos y sin significado
int f(int a, int b) {
    int c = a * b;
    int d = c / 2;
    return d;
}

// BIEN: Nombres autoexplicativos
int calcular_media(int suma_total, int ↩
   numero_de_elementos) {
    int media = suma_total / numero_de_elementos;
    return media;
}

int main() {
    int n1 = 10; // MAL:   Qu   es n1?
    int edad_del_usuario = 25; // BIEN: Queda claro que es

    // MAL: nombres de una letra en contexto amplio
    int a = 5;
    int b = 3;
    int r = f(a, b);

    // BIEN: Contexto claro
    int notas[] = {85, 90, 78};
    int num_notas = 3;
    int suma_notas = 85 + 90 + 78;
    int promedio = calcular_media(suma_notas, num_notas);

    return 0;
}
```

Listing 38: Malas y buenas denominaciones

Consejos:

- Usa nombres en inglés o español, pero sé consistente.

- Para variables, usa sustantivos: `contador`, `nombre_completo`.
- Para funciones, usa verbos: `calcular_promedio()`, `imprimir_factura()`.
- Evita abreviaturas crípticas (`tmp` está bien para un temporal de corta vida, pero `sz` no está claro).

Documentación y Formato (Estilo de Código)

El código se lee muchas más veces de las que se escribe. Un código bien formateado y comentado es más fácil de leer y mantener.

Comentarios

Los comentarios deben explicar el **porqué**, no el **cómo**. El çómo"debería ser evidente por el código mismo si está bien escrito.

```c
#include <stdio.h>

// MAL: Comentario redundante
int i = 0; // Inicializo i a 0

// BIEN: Explica una decision o logica compleja
int calcular_interes(double saldo, int dias) {
    // Usamos la regla de 365/360 segun la normativa bancaria XYZ
    const int DIAS_BASE = 360;
    double interes = saldo * 0.05 * dias / DIAS_BASE;
    return interes;
}

/*
 *  Funcion: procesar_archivo
 *  Proposito: Lee un archivo de configuraci n y carga las opciones.
 *  Parametros:
 *      nombre_archivo: Ruta al archivo a procesar.
 *  Retorna: 0 si OK, -1 si error.
 */
int procesar_archivo(char *nombre_archivo) {
    // ... codigo ...
}
```

Listing 39: Buenos y malos comentarios

Formato (Indentación y Espacios)

La indentación consistente (usando espacios o tabs, pero no mezclar) es crucial para mostrar la estructura del programa.

```c
// DIFICIL DE LEER
#include <stdio.h>
int main(){
int x=5;
if(x>0){
printf("Positivo");
for(int i=0;i<3;i++){
printf("%d",i);
}
}
return 0;}

// FACIL DE LEER
#include <stdio.h>
int main() {
    int x = 5;
    if (x > 0) {
        printf("Positivo");
        for (int i = 0; i < 3; i++) {
            printf("%d", i);
        }
    }
    return 0;
}
```

Listing 40: Mal y buen formato

Refinamiento (Top-Down Design)

El refinamiento sucesivo, también conocido como diseño de arriba hacia abajo (*top-down design*), es una técnica donde empezamos con una descripción de alto nivel del problema y la vamos desglosando en pasos cada vez más detallados hasta llegar a instrucciones que podamos codificar directamente.

Ejemplo: Calcular el promedio de notas de un estudiante

```
Nivel 0 (Problema):
    Calcular el promedio de notas.

Nivel 1 (Subproblemas):
    1. Obtener las notas del usuario.
    2. Calcular la suma de las notas.
    3. Calcular el promedio (suma / cantidad).
    4. Mostrar el resultado.

Nivel 2 (Detalle de "Obtener notas"):
    1.1. Preguntar cuantas notas va a ingresar.
```

```
    1.2. Para cada nota:
        1.2.1. Pedir la nota.
        1.2.2. Almacenarla.

Nivel 3 (Detalle de "Calcular suma"):
    2.1. Inicializar suma = 0.
    2.2. Para cada nota almacenada:
        2.2.1. Sumarla al acumulador.

... y así sucesivamente.
```

Este proceso nos lleva directamente a una estructura de programa modular y clara.

Codificación y Pruebas

Una vez diseñado, pasamos a codificar. Pero la codificación y las pruebas van de la mano.

Codificación

Es la traducción de nuestro diseño a un lenguaje de programación (C). Seguir las convenciones de formato y denominación es parte de esta fase.

Pruebas (Testing)

El objetivo es encontrar errores (*bugs*). Nunca asumas que el programa funciona porque çompila.º porque "probé un caso".

- **Pruebas Unitarias:** Verificar que cada función o módulo por separado funciona correctamente.
- **Pruebas de Integración:** Verificar que los módulos funcionan bien juntos.
- **Pruebas de Regresión:** Al hacer un cambio, verificar que no se rompió nada que ya funcionaba.

Estrategias de prueba

- **Camino Feliz (Happy Path):** Probar con entradas típicas y esperadas.
- **Casos Límite (Edge Cases):** Probar con valores extremos (0, números muy grandes, listas vacías, etc.).
- **Casos de Error:** Probar con entradas inválidas (división por cero, punteros NULL, etc.).

```c
#include <stdio.h>
#include <assert.h> // Para pruebas simples con assert

// Funcion que queremos probar
int dividir(int a, int b, float *resultado) {
    if (b == 0) {
        return -1; // Indicar error
    }
    *resultado = (float)a / b;
    return 0; // Exito
}

// Pruebas
void test_dividir() {
    float res;

    // Happy path
    assert(dividir(10, 2, &res) == 0 && res == 5.0f);

    // Edge case: numeros negativos
    assert(dividir(-10, 2, &res) == 0 && res == -5.0f);

    // Edge case: division que da decimal
    assert(dividir(5, 2, &res) == 0 && res == 2.5f);

    // Caso de error: division por cero
    assert(dividir(10, 0, &res) == -1);

    printf("Todas las pruebas pasaron!\n");
}

int main() {
    test_dividir();
    return 0;
}
```

Listing 41: Ejemplo de función a probar y casos de prueba

Modularización: Divide y Vencerás

La **modularización** es el proceso de dividir un programa grande en partes más pequeñas y manejables llamadas **módulos** (en C, generalmente funciones y archivos separados). Cada módulo tiene una responsabilidad bien definida.

Criterios y Principios de Modularización

- **Alta Cohesión:** Un módulo debe realizar una única tarea bien definida. Todas sus partes deben estar estrechamente relacionadas. Una función que calcula promedios **y** imprime facturas tiene baja cohesión.
- **Bajo Acoplamiento:** Los módulos deben depender lo menos posible unos de otros. Deben comunicarse a través de interfaces pequeñas y bien definidas (parámetros y valores de retorno). Cambiar un módulo no debería forzar cambios masivos en otros.

Visualización de Cohesión y Acoplamiento

```
BAJA COHESION (Mala):
+---------------------+
|  Modulo "Utilidades" |
|  - leer_archivo()    |
|  - calcular_media()  |
|  - imprimir()        |
|  - enviar_email()    |
+---------------------+

ALTA COHESION (Buena):
+---------------------+
|  Modulo "Calculos"   |
|  - calcular_media()  |
|  - calcular_suma()   |
+---------------------+

ALTO ACOPLAMIENTO (Mala):
+--------+      +--------+
| Mod A  |----->| Mod B  |
| (usa   |      | (usa   |
|  datos |<-----|  datos |
|  internos     |  internos
|  de B) |      |  de A) |
+--------+      +--------+
   ^               ^
   |               |
   +---------------+
  (Dependencia circular)

BAJO ACOPLAMIENTO (Buena):
+--------+      +--------+
| Mod A  |----->| Mod B  |
| (llama |      | (recibe|
|  f()   |----->|  args, |
|  con   |      |  devuelve
|  args) |      |  valor)|
+--------+      +--------+
     |               |
     |               |
      +---------------+
    (Comunicacion limpia)
```

```c
// archivo: operaciones.h (Interfaz)
#ifndef OPERACIONES_H
#define OPERACIONES_H

float calcular_media(int suma, int cantidad);
int calcular_suma(int *datos, int cantidad);

#endif

// archivo: operaciones.c (Implementacion)
#include "operaciones.h"

float calcular_media(int suma, int cantidad) {
    if (cantidad == 0) return 0.0f;
```

```c
    return (float)suma / cantidad;
}

int calcular_suma(int *datos, int cantidad) {
    int suma = 0;
    for (int i = 0; i < cantidad; i++) {
        suma += datos[i];
    }
    return suma;
}

// archivo: main.c (Uso de los modulos)
#include <stdio.h>
#include "operaciones.h"

int main() {
    int notas[] = {85, 90, 78, 95};
    int num_notas = 4;
    int suma = calcular_suma(notas, num_notas);
    float media = calcular_media(suma, num_notas);
    printf("La media es: %.2f\n", media);
    return 0;
}
```

Listing 42: Ejemplo de modularización

Reusabilidad

La **reusabilidad** es la capacidad de usar un módulo o componente en diferentes programas o en diferentes partes del mismo programa sin modificarlo. Los beneficios son enormes: ahorro de tiempo, menos errores (porque el código ya ha sido probado) y mayor consistencia.

¿Cómo logramos reusabilidad?

- Escribiendo funciones genéricas que no dependan de datos globales.
- Creando bibliotecas (librerías) de funciones útiles (como hicimos con `operaciones.h`).
- Documentando claramente cómo usar el módulo (su interfaz).

Abstracción de Datos

La **abstracción de datos** es el concepto de ocultar los detalles de implementación de una estructura de datos y exponer solo las operaciones que se pueden realizar sobre ella. El usuario del tipo de dato no necesita saber cómo está implementado internamente, solo qué operaciones puede hacer.

En C, esto se logra típicamente con:

- **Tipos de datos definidos por el usuario** (`struct`).
- **Archivos de cabecera** (`.h`) que declaran las operaciones (la interfaz pública).
- **Archivos de implementación** (`.c`) que contienen el código y los detalles ocultos.
- Uso de `static` para hacer privadas las funciones internas.

```c
// punto.h - INTERFAZ PUBLICA
#ifndef PUNTO_H
#define PUNTO_H

// Declaracion adelantada (opaca) del tipo Punto
// El usuario solo sabe que existe, no su estructura interna
typedef struct Punto Punto;

// Operaciones publicas
Punto* punto_crear(float x, float y);
void punto_destruir(Punto *p);
void punto_mover(Punto *p, float dx, float dy);
float punto_distancia_origen(Punto *p);
void punto_imprimir(Punto *p);

#endif

// punto.c - IMPLEMENTACION PRIVADA
#include "punto.h"
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

// Aqui SI definimos la estructura (oculta al usuario)
struct Punto {
    float x;
    float y;
};

Punto* punto_crear(float x, float y) {
    Punto *nuevo = (Punto*) malloc(sizeof(Punto));
    if (nuevo != NULL) {
        nuevo->x = x;
        nuevo->y = y;
    }
    return nuevo;
}

void punto_destruir(Punto *p) {
    free(p);
}
```

```c

void punto_mover(Punto *p, float dx, float dy) {
    if (p != NULL) {
        p->x += dx;
        p->y += dy;
    }
}

float punto_distancia_origen(Punto *p) {
    return sqrt(p->x * p->x + p->y * p->y);
}

void punto_imprimir(Punto *p) {
    printf("Punto(%.2f, %.2f)", p->x, p->y);
}

// main.c - USO
#include <stdio.h>
#include "punto.h"

int main() {
    Punto *mi_punto = punto_crear(3.0, 4.0);
    punto_imprimir(mi_punto);
    printf(" -> Distancia al origen: %.2f\n", ↩
       punto_distancia_origen(mi_punto));

    punto_mover(mi_punto, -1.0, 2.0);
    printf("Despues de mover: ");
    punto_imprimir(mi_punto);
    printf("\n");

    punto_destruir(mi_punto);
    return 0;
}
```

Listing 43: Ejemplo de abstracción de datos: Un Punto en 2D

En este ejemplo, el usuario de `Punto` no sabe (y no le importa) si las coordenadas son `float` o `double`, o si la distancia se calcula con `sqrt` o de otra forma. Solo conoce las funciones que puede usar. Esto es abstracción de datos.

Aspectos Acerca de la Calidad del Software

¿Qué hace que un software sea de çalidad¿ No es solo que funcione. Hay varios atributos clave:

- **Corrección:** El software hace lo que debe hacer (satisface los requisitos).
- **Robustez:** El software se comporta de manera predecible ante situaciones inesperadas (errores de entrada, fallos de hardware, etc.). No .explota".

- **Eficiencia:** El software usa los recursos (tiempo de CPU, memoria) de manera óptima.
- **Mantenibilidad:** Es fácil de modificar para corregir errores o añadir nuevas funcionalidades. Esto es una consecuencia directa de la modularización, la documentación y las buenas prácticas de código.
- **Portabilidad:** El software puede ejecutarse en diferentes plataformas (sistemas operativos, hardware) con pocos o ningún cambio.
- **Usabilidad:** Es fácil de usar para el usuario final.
- **Reusabilidad:** Ya lo vimos, sus componentes pueden usarse en otros proyectos.

La Pirámide de la Calidad del Software

```
        /\          Usabilidad
       /  \         Portabilidad
      /    \        Reusabilidad
     /      \       Mantenibilidad
    /--------\      Eficiencia
   /----------\     Robustez
  /------------\    Correccion
 /--------------\
/----------------\
/     REQUISITOS   \
/____________________\
```

La base es la corrección (hacer lo que se pide). Sobre ella se construye la robustez (hacerlo de forma fiable), y así sucesivamente.

Resumen de la Unidad IV

Esta unidad nos alejó de la sintaxis pura de C y nos introdujo en el arte y la ingeniería de la programación.

- **Denominaciones:** Usa nombres claros y descriptivos. El código se lee más de lo que se escribe.
- **Documentación y formato:** Comenta el *porqué*, no el *cómo*. Usa un estilo de código consistente.
- **Refinamiento:** Diseña de arriba hacia abajo, descomponiendo problemas grandes en subproblemas más pequeños.
- **Codificación y pruebas:** Prueba tu código sistemáticamente (casos felices, límites y de error).
- **Modularización:** Divide el programa en módulos con alta cohesión (una sola responsabilidad) y bajo acoplamiento (interfaces simples).

- **Reusabilidad:** Escribe código pensando en que podría ser usado de nuevo.
- **Abstracción de datos:** Oculta los detalles de implementación y expone solo una interfaz limpia. Es la base de la programación orientada a objetos (que veremos más adelante).
- **Calidad del software:** No basta con que funcione. Buscamos corrección, robustez, eficiencia, mantenibilidad, etc.

Estos principios son universales y te servirán sin importar el lenguaje o el tipo de proyecto en el que trabajes. Aplicarlos desde el principio es la marca de un buen programador.

Unidad V: Funciones - Los Bloques de Construcción de Programas

Las funciones son el corazón de la programación estructurada en C. Desde el primer día hemos usado `main()` y `printf()`, pero ahora es momento de entender a fondo cómo crearlas, usarlas y dominarlas. Una función es, esencialmente, un bloque de código reutilizable que realiza una tarea específica.

Analogía: Una función es como un electrodoméstico

```
    [LICUADORA]
         |
    +----+----+
    |         |
Entrada   Proceso   Salida
(Frutas)  (Licuar)  (Jugo)
    |         |
    +----+----+
         |
    [FUNCION]
```

Así como una licuadora recibe frutas (entrada), las procesa y produce jugo (salida), una función recibe argumentos, realiza operaciones y devuelve un resultado.

Concepto de Función

Una **función** es un bloque de código con un nombre, que puede recibir parámetros y puede devolver un valor. Nos permite:

- **Modularizar:** Dividir problemas complejos en partes más pequeñas.
- **Reutilizar:** Escribir código una vez y usarlo muchas veces.
- **Abstraer:** Ocultar detalles complejos y usar la función como una çaja negra".

Sintaxis Básica de una Función

```c
tipo_de_retorno nombre_de_la_funcion(tipo_param1 nombre_param1, tipo_param2 nombre_param2) {
    // Cuerpo de la funcion
    // ... instrucciones ...
    return valor_del_tipo_de_retorno;
}
```

Listing 44: Estructura de una función en C

- **tipo_de_retorno**: El tipo de dato que la función devuelve (int, float, char, etc.). Si no devuelve nada, usamos `void`.
- **nombre_de_la_funcion**: Identificador con las mismas reglas que las variables.
- **parámetros**: Lista de variables que recibe la función (puede estar vacía: `void`).
- **cuerpo**: Bloque de código entre llaves `{}` que implementa la lógica.
- **return**: Instrucción que devuelve el resultado y termina la función.

```c
#include <stdio.h>

// Funcion que suma dos enteros y devuelve el resultado
int sumar(int a, int b) {
    int resultado = a + b;
    return resultado;
}

// Funcion que no recibe parametros y no devuelve nada
void saludar(void) {
    printf("Hola, mundo!\n");
    // No necesita return, pero se puede usar "return;" para salir antes
}

int main() {
    int x = 5, y = 3;
    int suma = sumar(x, y); // Llamada a la funcion
    printf("La suma es: %d\n", suma);
    saludar();
    return 0;
}
```

Listing 45: Ejemplo de función simple

La Instrucción `return`

La instrucción `return` tiene dos propósitos fundamentales:

1. **Terminar la ejecución de la función**: Cuando se encuentra un `return`, la función finaliza inmediatamente y el control vuelve al punto donde fue llamada.

2. **Devolver un valor**: Opcionalmente, puede devolver un valor al llamante.

```c
#include <stdio.h>
#include <stdbool.h> // Para usar bool, true, false

// 1. Funcion que devuelve un valor
int cuadrado(int x) {
    return x * x;
}

// 2. Funcion que puede terminar antes
int buscar_primer_par(int numeros[], int size) {
    for (int i = 0; i < size; i++) {
        if (numeros[i] % 2 == 0) {
            return i; // Termina aqui, devuelve el indice
        }
    }
    return -1; // Si no encuentra, devuelve -1
}

// 3. Funcion void que usa return para salir antes
void procesar(int valor) {
    if (valor < 0) {
        printf("Valor negativo, no se procesa.\n");
        return; // Salimos de la funcion sin hacer nada mas
    }
    printf("Procesando valor: %d\n", valor);
    // Aqui no hay return explicito, pero al final de la funcion se retorna implicitamente
}

int main() {
    printf("Cuadrado de 5: %d\n", cuadrado(5));

    int arr[] = {1, 3, 5, 6, 7};
    int pos = buscar_primer_par(arr, 5);
    if (pos != -1) {
        printf("Primer par encontrado en indice: %d\n", pos);
    } else {
        printf("No se encontro ningun par\n");
    }
```

```

    procesar(10);
    procesar(-5);

    return 0;
}
```

Listing 46: Diferentes usos de `return`

Valores de Retorno

El valor de retorno puede ser de cualquier tipo (int, float, char, puntero, struct, etc.). Es importante que el tipo de la expresión después de `return` sea compatible con el tipo de retorno declarado.

```
#include <stdio.h>

// Retorna un entero
int obtener_edad() {
    return 25;
}

// Retorna un float
float obtener_promedio(int a, int b) {
    return (a + b) / 2.0f;
}

// Retorna un caracter
char obtener_inicial(char *nombre) {
    return nombre[0];
}

// Retorna un puntero a char (string)
char *obtener_saludo() {
    return "Hola, mundo!";
}

int main() {
    printf("Edad: %d\n", obtener_edad());
    printf("Promedio: %.2f\n", obtener_promedio(10, 15));
    printf("Inicial: %c\n", obtener_inicial("Carlos"));
    printf("Saludo: %s\n", obtener_saludo());
    return 0;
}
```

Listing 47: Funciones con diferentes tipos de retorno

Reglas de Alcance de Funciones

El **ámbito (scope)** de una función se refiere a dónde puede ser llamada.

- Por defecto, todas las funciones tienen ámbito **global**: pueden ser llamadas desde cualquier parte del programa (siempre que hayan sido declaradas o definidas antes del punto de llamada).
- Podemos restringir el ámbito de una función al archivo actual usando la palabra clave `static`.

```c
#include <stdio.h>

// Funcion global: puede ser llamada desde cualquier archivo
void funcion_global() {
    printf("Puedo ser llamada desde cualquier lado.\n");
}

// Funcion estatica: solo visible en este archivo
static void funcion_privada() {
    printf("Solo visible en este archivo.\n");
}

void llamar_privada() {
    funcion_privada(); // OK: dentro del mismo archivo
}

int main() {
    funcion_global();   // OK
    llamar_privada();   // OK, llama a funcion_privada indirectamente
    // funcion_privada(); // Tambien OK porque estamos en el mismo archivo
    return 0;
}
```

Listing 48: Ámbito de funciones

Argumentos de Funciones

Los argumentos (o parámetros) son los valores que le pasamos a una función para que trabaje con ellos.

Llamada por Valor (Pass by Value)

En C, **todos los argumentos se pasan por valor por defecto**. Esto significa que la función recibe una **copia** del valor del argumento. Modificar el parámetro dentro de la función no afecta a la variable original.

```c
#include <stdio.h>

void duplicar(int x) {
    x = x * 2; // Modifica la copia
    printf("Dentro de duplicar: x = %d\n", x);
}

int main() {
    int a = 5;
    printf("Antes de llamar: a = %d\n", a);
    duplicar(a);
    printf("Despues de llamar: a = %d\n", a); // a sigue siendo 5!
    return 0;
}
```

Listing 49: Paso por valor

Visualización del paso por valor

```
main():                         duplicar():
a = 5                           x = 5 (copia)
  |                                |
  +-----> duplicar(a) ------> x = x*2 (x = 10)
                                   |
                                   | (x se destruye al salir)
a sigue siendo 5 <-------------+
```

Llamada por Referencia (Simulada con Punteros)

C no tiene paso por referencia como otros lenguajes (C++, Java). Sin embargo, podemos simularlo pasando **punteros**. Así, la función recibe una copia de la **dirección** de la variable, y usando el operador de desreferencia * podemos modificar el valor original.

```c
#include <stdio.h>

void duplicar_referencia(int *ptr) {
    *ptr = *ptr * 2; // Modifica el valor en la direccion apuntada
    printf("Dentro de duplicar_referencia: *ptr = %d\n", *ptr);
}

int main() {
    int a = 5;
    printf("Antes de llamar: a = %d\n", a);
    duplicar_referencia(&a); // Pasamos la direccion de a
```

```
    printf("Despues de llamar: a = %d\n", a); // a ahora ↩
        es 10!
    return 0;
}
```

Listing 50: Simulación de paso por referencia con punteros

Visualización del paso por referencia (con punteros)

```
main():                          duplicar_referencia():
a = 5 en direccion 0x100         ptr = 0x100 (copia de la direccion)
  |                                  |
  +-----> duplicar_referencia(&a)    |
          (pasa 0x100) -------------+
                                     |
                                     V
                                 *ptr = *ptr * 2
                                 (accede a la direccion 0x100
                                  y modifica a a 10)

main(): a ahora es 10 <-------------+
```

Comparación: Por Valor vs Por Referencia

Paso por Valor	**Paso por Referencia (con punteros)**
Se pasa una copia del dato.	Se pasa la dirección del dato.
La función no puede modificar la variable original.	La función puede modificar la variable original.
Más seguro (no hay efectos secundarios no deseados).	Más flexible (permite modificar múltiples variables).
Útil cuando solo necesitas el valor para calcular.	Necesario para modificar el argumento o para evitar copiar grandes estructuras.

Prototipos de Funciones

Un **prototipo** (o declaración) de función le dice al compilador: .Existe una función con este nombre, que recibe estos parámetros y devuelve este tipo". Esto permite llamar a la función antes de que el compilador haya visto su definición completa.

```
#include <stdio.h>

// Prototipos (declaraciones)
int sumar(int a, int b);
float dividir(float a, float b);
```

```c
void saludar(char *nombre);

int main() {
    // Podemos usar las funciones aunque su definicion ↩
       este despues
    printf("Suma: %d\n", sumar(5, 3));
    printf("Division: %.2f\n", dividir(10.0, 3.0));
    saludar("Ana");
    return 0;
}

// Definiciones de las funciones
int sumar(int a, int b) {
    return a + b;
}

float dividir(float a, float b) {
    if (b == 0) return 0;
    return a / b;
}

void saludar(char *nombre) {
    printf("Hola, %s!\n", nombre);
}
```

Listing 51: Prototipos de funciones

Beneficios de los prototipos:

- Permiten organizar el código con `main()` al principio y las funciones después.
- El compilador puede verificar que las llamadas a funciones coincidan con los tipos esperados.
- Son esenciales cuando trabajamos con múltiples archivos (los prototipos van en archivos `.h`).

Retorno de Punteros

Las funciones pueden devolver punteros. Esto es muy útil para devolver strings, estructuras asignadas dinámicamente, o para indicar la posición de un elemento en un array.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// 1. Retornar un puntero a un string literal
char *obtener_mensaje() {
    return "Este es un mensaje";
}
```

```c

// 2. Retornar un puntero a un elemento de un array
int *buscar_elemento(int *arr, int size, int valor) {
    for (int i = 0; i < size; i++) {
        if (arr[i] == valor) {
            return &arr[i]; // Retorna la direccion del elemento encontrado
        }
    }
    return NULL; // No encontrado
}

// 3. Retornar un puntero a memoria dinamica
char *duplicar_string(const char *original) {
    if (original == NULL) return NULL;
    int len = strlen(original);
    char *copia = (char*) malloc(len + 1); // +1 para el '\0'
    if (copia != NULL) {
        strcpy(copia, original);
    }
    return copia; // El llamante debe liberar esta memoria con free()
}

int main() {
    // Ejemplo 1
    printf("%s\n", obtener_mensaje());

    // Ejemplo 2
    int numeros[] = {10, 20, 30, 40};
    int *ptr = buscar_elemento(numeros, 4, 30);
    if (ptr != NULL) {
        printf("Elemento encontrado: %d en direccion %p\n", *ptr, ptr);
    }

    // Ejemplo 3
    char *copia = duplicar_string("Hola, mundo!");
    if (copia != NULL) {
        printf("Copia: %s\n", copia);
        free(copia); // LIBERAR MEMORIA! Muy importante
    }
    return 0;
}
```

Listing 52: Funciones que retornan punteros

¡PRECAUCIÓN! Nunca retornes un puntero a una variable local automática, porque esa variable se destruye al salir de la función y el puntero quedará colgando (apuntando

a memoria inválida).

```c
int *funcion_mal() {
    int x = 10;
    return &x; // GRAVE ERROR! x es local y deja de existir al salir
} // Al salir de la funcion, la direccion de x es invalida
```

Listing 53: ERROR: Retornar puntero a variable local

Modelos de Memoria en C

Para entender mejor cómo funcionan las variables y funciones, necesitamos conocer los diferentes segmentos de memoria donde se almacenan los datos.

Organización de la Memoria en C

```
+-------------------------+  Direcciones altas (ej. 0xFFFF)
|        STACK (Pila)     |
| (Variables locales,     |
|  parametros, direcciones |
|  de retorno)            |
|         |               |
|         | (crece hacia  |
|         V  abajo)       |
+-------------------------+
|            |            |
|            V            |
|                         |
|        HEAP (Montón)    |
| (Memoria dinamica:      |
|  malloc, calloc, realloc)|
|         ^               |
|         | (crece hacia  |
|         |  arriba)      |
+-------------------------+
|       DATOS (Data)      |
| (Variables globales     |
|  y static inicializadas) |
+-------------------------+
|        BSS (Block       |
|    Started by Symbol)   |
| (Variables globales     |
|  y static no iniciadas, |
|  se inicializan a 0)    |
+-------------------------+
```

```
|       TEXT (Código)      |
| (El programa en si,      |
|  las instrucciones)      |
+--------------------------+  Direcciones bajas (ej. 0x0000)
```

Segmentos de Memoria

- **TEXT (Código):** Contiene las instrucciones del programa. Es de solo lectura.
- **DATOS (Data):** Variables globales y `static` que tienen un valor inicial distinto de cero.
- **BSS:** Variables globales y `static` que no se inicializan explícitamente (el sistema las pone a 0).
- **HEAP (Montón):** Memoria asignada dinámicamente con `malloc()`, `calloc()`, `realloc()`. Crece hacia arriba.
- **STACK (Pila):** Variables locales automáticas, parámetros de funciones, direcciones de retorno. Se crea un "marco de pila"(stack frame) por cada llamada a función. Crece hacia abajo.

Ejemplo de ubicación de variables

```c
#include <stdio.h>
#include <stdlib.h>

int global_inicializada = 100;           // DATOS
int global_no_inicializada;               // BSS (se inicializa a 0)

static int estatica_inicializada = 200; // DATOS
static int estatica_no_inicializada;    // BSS

int main() {
    int local_auto = 10;                  // STACK
    static int local_estatica = 300;      // DATOS (aunque sea local, por 'static')
    int *ptr_dinamico;

    ptr_dinamico = (int*) malloc(sizeof(int)); // HEAP
    *ptr_dinamico = 50;

    printf("global_inicializada (DATOS): %d\n", global_inicializada);
    printf("local_auto (STACK): %d\n", local_auto);
    printf("local_estatica (DATOS): %d\n", local_estatica);
    printf("ptr_dinamico (STACK) apunta a HEAP: %d\n", *ptr_dinamico);
```

```
    free(ptr_dinamico);
    return 0;
}
```

Listing 54: ¿Dónde se almacena cada variable?

Recursividad

Una función **recursiva** es aquella que se llama a sí misma. Es una técnica poderosa para resolver problemas que pueden descomponerse en subproblemas más pequeños de la misma naturaleza.

Componentes de una Función Recursiva

1. **Caso base:** Una condición que detiene la recursión. Sin caso base, la función se llamaría infinitamente (hasta desbordar la pila).
2. **Llamada recursiva:** La función se llama a sí misma, pero con un argumento modificado que se acerca al caso base.

Ejemplo Clásico: Factorial

El factorial de un número n (n!) se define como:

- $0! = 1$ (caso base)
- $n! = n \times (n-1)!$ para $n > 0$ (paso recursivo)

```
#include <stdio.h>

int factorial(int n) {
    // Caso base
    if (n == 0) {
        return 1;
    }
    // Llamada recursiva
    return n * factorial(n - 1);
}

int main() {
    int num = 5;
    printf("Factorial de %d = %d\n", num, ↩
       factorial(num)); // 120
    return 0;
}
```

Listing 55: Factorial recursivo

Visualización de la pila en la recursión de factorial(5)

```
Llamadas sucesivas (ida):
main() llama a factorial(5)
  factorial(5) llama a factorial(4)
    factorial(4) llama a factorial(3)
      factorial(3) llama a factorial(2)
        factorial(2) llama a factorial(1)
          factorial(1) llama a factorial(0)

Resolviendo (vuelta):
          factorial(0) devuelve 1
        factorial(1) devuelve 1 * 1 = 1
      factorial(2) devuelve 2 * 1 = 2
    factorial(3) devuelve 3 * 2 = 6
  factorial(4) devuelve 4 * 6 = 24
factorial(5) devuelve 5 * 24 = 120
main() recibe 120
```

Ejemplo: Fibonacci

La secuencia de Fibonacci: $F(0) = 0, F(1) = 1, F(n) = F(n-1) + F(n-2)$.

```c
#include <stdio.h>

int fibonacci(int n) {
    if (n <= 1) {
        return n; // Casos base: F(0)=0, F(1)=1
    }
    return fibonacci(n - 1) + fibonacci(n - 2);
}

int main() {
    for (int i = 0; i <= 10; i++) {
        printf("F(%d) = %d\n", i, fibonacci(i));
    }
    return 0;
}
```

Listing 56: Fibonacci recursivo

Nota: Esta implementación es muy ineficiente porque recalcula los mismos valores muchas veces. Es un buen ejemplo de recursión simple, pero en la práctica se usaría iteración o memoización.

Recursividad vs Iteración

Recursividad	**Iteración**
Código más elegante y cercano a la definición matemática.	Código generalmente más eficiente (sin overhead de llamadas a función).
Útil para estructuras de datos recursivas (árboles, listas).	Usa bucles (`while`, `for`).
Puede causar desbordamiento de pila (stack overflow) si la profundidad es muy grande.	No tiene riesgo de desbordamiento de pila por repetición.
Cada llamada consume memoria en la pila.	No consume memoria extra por iteración.

Resumen de la Unidad V

Hemos profundizado en el concepto más importante de la programación estructurada: las funciones.

- **Funciones:** Bloques reutilizables de código con un nombre, parámetros y valor de retorno.
- **return:** Termina la función y opcionalmente devuelve un valor.
- **Ámbito:** Las funciones pueden ser globales o estáticas (privadas al archivo).
- **Paso de argumentos:**
  - **Por valor:** Se pasa una copia (el original no se modifica).
  - **Por referencia (simulado):** Se pasa un puntero (se puede modificar el original).
- **Prototipos:** Declaraciones que permiten usar funciones antes de su definición.
- **Retorno de punteros:** Útil para strings y memoria dinámica, pero cuidado con retornar punteros a locales.
- **Modelos de memoria:** TEXT (código), DATOS/BSS (globales/static), HEAP (dinámica), STACK (locales y llamadas a función).
- **Recursividad:** Función que se llama a sí misma. Requiere un caso base y una llamada que se acerque a él. Elegante pero puede ser ineficiente.

Las funciones son la herramienta fundamental para construir programas modulares, legibles y mantenibles. En la siguiente unidad, exploraremos cómo organizar múltiples funciones en proyectos más grandes usando compilación separada y el preprocesador.

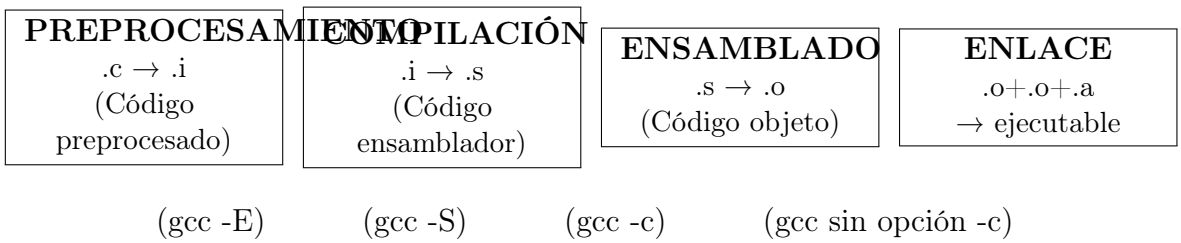
Unidad VI: Compilación, Enlace y el Preprocesador - Construyendo Proyectos Profesionales

Hasta ahora hemos trabajado con programas de un solo archivo, ejecutando comandos simples como `gcc programa.c -o programa`. Sin embargo, los proyectos de software del mundo real constan de decenas, cientos o miles de archivos, y su construcción es un proceso complejo que debe ser automatizado. Esta unidad te llevará al siguiente nivel: entenderás cómo se transforma un conjunto de archivos fuente en un programa ejecutable, cómo organizar tu código en módulos, y cómo usar las herramientas que C proporciona para gestionar todo el proceso.

El Proceso Completo: De Código Fuente a Ejecutable

Cuando escribimos un programa en C, el código fuente no es directamente ejecutable por la máquina. Debe pasar por un proceso de transformación que consta de cuatro fases principales. Es crucial entender cada una para diagnosticar errores y optimizar la compilación.

Las 4 Fases de la Transformación del Código



Fase 1: Preprocesamiento

El preprocesador (comúnmente llamado `cpp`) es la primera herramienta que actúa sobre nuestro código. No compila nada; simplemente realiza manipulaciones textuales basadas en directivas que comienzan con el símbolo `#`.

```
#include <stdio.h>
#define PI 3.14159
#define CUADRADO(x) ((x)*(x))
```

```c

int main() {
    double area = PI * CUADRADO(5);
    printf("Area: %f\n", area);
    return 0;
}
```

Listing 57: Código antes del preprocesamiento (entrada.c)

```c
// ... contenido literal de stdio.h (cientos de lineas) ...

int main() {
    double area = 3.14159 * ((5)*(5));
    printf("Area: %f\n", area);
    return 0;
}
```

Listing 58: Código después del preprocesamiento (salida.i) - simplificado

Acciones del preprocesador:

- Elimina todos los comentarios.
- Expande las macros (`#define`).
- Inserta el contenido de los archivos incluidos (`#include`).
- Procesa las directivas de compilación condicional (`#ifdef`, `#ifndef`, etc.).

Para ver el resultado del preprocesamiento, podemos usar:

```
gcc -E programa.c -o programa.i
```

Fase 2: Compilación

El compilador toma el archivo preprocesado (`.i`) y lo traduce a código **ensamblador** (`.s`). El código ensamblador es un lenguaje de bajo nivel específico de la arquitectura del procesador, pero aún legible por humanos.

```
gcc -S programa.i -o programa.s
```

Un fragmento de código ensamblador (para x86-64) podría verse así:

```
main:
    push    rbp
    mov     rbp, rsp
    sub     rsp, 16
    movsd   xmm0, QWORD PTR .LC0[rip]
    movsd   QWORD PTR [rbp-8], xmm0
    mov     eax, 0
```

```
    leave
    ret
.LC0:
    .long   1374389535
    .long   1074339512  # Esto es 3.14159 en representacion de punto flotante
```

Fase 3: Ensamblado

El ensamblador convierte el código ensamblador (`.s`) en **código máquina** (binario) y lo guarda en un **archivo objeto** (`.o` en Linux/Mac, `.obj` en Windows). Este archivo contiene instrucciones ejecutables, pero aún no es un programa completo porque las direcciones de funciones externas (como `printf`) no están resueltas.

```
gcc -c programa.s -o programa.o
# O directamente desde el fuente C:
gcc -c programa.c -o programa.o
```

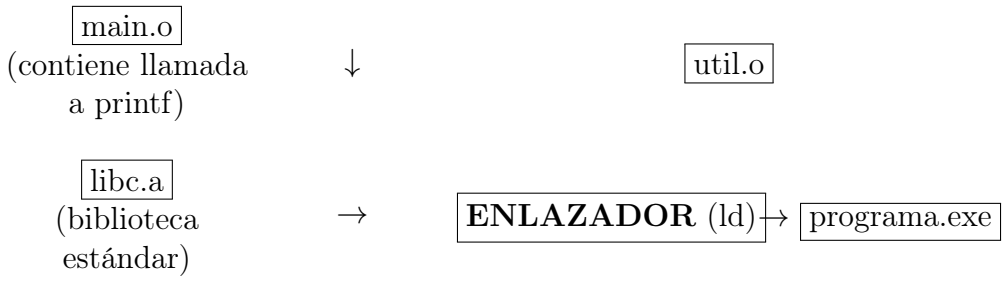
Podemos inspeccionar un archivo objeto con herramientas como `nm` (Linux/Mac) para ver sus símbolos:

```
$ nm programa.o
0000000000000000 T main
                 U printf  # La 'U' significa "Undefined" (indefinido)
```

Fase 4: Enlace (Link)

El **enlazador** (linker) es la pieza final del rompecabezas. Su trabajo es combinar uno o más archivos objeto y las bibliotecas necesarias para producir un único archivo **ejecutable**. Resuelve las referencias externas: cuando encuentra `U printf` en nuestro `programa.o`, busca la definición de `printf` en las bibliotecas (por defecto, la biblioteca estándar de C, `libc.so` o `libc.a`) y çonecta"la llamada con el código de la función.

El Proceso de Enlace




```
gcc programa.o util.o -o programa
# O directamente desde los fuentes (gcc llama automaticamente al enlazador):
gcc programa.c util.c -o programa
```

El Preprocesador de C en Profundidad

El preprocesador es una herramienta poderosa que actúa antes de la compilación. Todas sus directivas comienzan con # y terminan con un salto de línea (no llevan punto y coma).

Las Directivas del Preprocesador

`#include` - Inclusión de Archivos

Inserta el contenido de otro archivo en el archivo actual. Hay dos formas:

```
#include <stdio.h>      // Busca en los directorios del sistema (ej. /usr/include)
#include "mi_libreria.h" // Busca primero en el directorio actual, luego en los del sistema
```

Listing 59: Formas de `#include`

`#define` - Definición de Macros

Crea macros, que son como .ªbreviaturas"que el preprocesador expande textualmente.

```
#define PI 3.14159
#define EDAD_MAXIMA 150
#define NOMBRE_PROGRAMA "Calculadora v1.0"
#define ERROR -1

int main() {
    double radio = 10.0;
    double area = PI * radio * radio; // Se expande a: 3.14159 * radio * radio
    if (area > 1000) {
        return ERROR;                  // Se expande a: return -1;
    }
    return 0;
}
```

Listing 60: Macros simples

Macros con Parámetros (Macros tipo función)

Podemos crear macros que reciben parámetros. El preprocesador las expande haciendo una sustitución textual.

```
#define CUADRADO(x) ((x) * (x))
#define MAX(a, b) ((a) > (b) ? (a) : (b))
#define MIN(a, b) ((a) < (b) ? (a) : (b))
```

```
#define ES_PAR(n) ((n) % 2 == 0)

int main() {
    int a = 5, b = 10;
    int c = CUADRADO(a);          // c = ((5) * (5)) = 25
    int max = MAX(a, b);           // max = ((5) > (10) ? ↩
        (5) : (10)) = 10
    int d = CUADRADO(a + b);       // d = ((5+10) * (5+10)) ↩
        = 225
    int e = CUADRADO(++a);         //   PELIGRO ! Se expande ↩
        a ((++a) * (++a))
                                   // Comportamiento ↩
                                      indefinido (a se ↩
                                      incrementa dos veces)
    return 0;
}
```

Listing 61: Macros con parámetros

¡PRECAUCIÓN CON LAS MACROS! Como la expansión es textual, pueden ocurrir efectos secundarios inesperados. Por eso, es buena práctica:

- Encerrar cada parámetro entre paréntesis en la definición: `((x)*(x))`.
- Encerrar toda la macro entre paréntesis.
- Evitar pasar expresiones con efectos secundarios (como `++a`) a macros.

`#undef` - Eliminar una Macro

Permite .ºlvidarüna definición previa.

```
#define TEMP 100
// ... codigo que usa TEMP ...
#undef TEMP
// A partir de aqui, TEMP ya no esta definida
#define TEMP 200 // Se puede volver a definir
```

Operadores Especiales del Preprocesador

- `#` (operador de stringización): Convierte el parámetro de una macro en un string literal.
- `##` (operador de concatenación): Concatena dos tokens.

```
#define STRINGIFY(x) #x
#define CONCAT(a, b) a##b

int main() {
```

```
    printf(STRINGIFY(Hola mundo)); // Se expande a: printf("Hola mundo");

    int mi_variable_123 = 10;
    int CONCAT(mi_variable_, 123) = 20; // Se expande a: int mi_variable_123 = 20;
    //  Cuidado ! Esto puede ser confuso y dificil de depurar.
    return 0;
}
```

Listing 62: Operadores # y ##

Compilación Condicional

Esta es una de las características más útiles del preprocesador. Permite incluir o excluir partes del código basándose en condiciones que se evalúan en tiempo de preprocesamiento.

Directivas Principales

```
#include <stdio.h>

#define VERSION 2
#define DEBUG   // Definida, pero sin valor

int main() {
    #ifdef DEBUG  // Si DEBUG esta definido (con o sin valor)
        printf("Modo DEBUG activado.\n");
    #endif

    #if VERSION == 1
        printf("Version 1 del programa\n");
    #elif VERSION == 2
        printf("Version 2 del programa\n");
    #else
        printf("Version desconocida\n");
    #endif

    #ifndef RELEASE  // Si RELEASE NO esta definido
        printf("Esta es una compilacion de desarrollo.\n");
    #endif

    return 0;
}
```

Listing 63: Compilación condicional básica

Usos Comunes de la Compilación Condicional

1. **Código específico de plataforma:**

```
#ifdef _WIN32
    #include <windows.h>
    #define LIMPIAR_PANTALLA system("cls")
#elif __linux__
    #include <unistd.h>
    #define LIMPIAR_PANTALLA system("clear")
#elif __APPLE__
    #define LIMPIAR_PANTALLA system("clear")
#endif
```

2. **Depuración (debug):**

```
#define DEBUG 1

if (error) {
    #if DEBUG
        printf("Error en linea %d del archivo %s\n", ↩
            __LINE__, __FILE__);
    #endif
    // Manejo del error...
}
```

3. **Guardas de inclusión (*include guards*) - ¡ESENCIALES!**

   Cuando varios archivos incluyen el mismo archivo de cabecera, podemos terminar con definiciones duplicadas. Las guardas de inclusión lo evitan.

```
#ifndef MI_LIBRERIA_H  // Si NO esta definido ↩
    MI_LIBRERIA_H
#define MI_LIBRERIA_H  // Lo definimos

// Todo el contenido del archivo va aqui
int sumar(int a, int b);
int restar(int a, int b);

#endif // Fin de la guarda
```

Listing 64: Archivo mi¡*ibreria.hconguardadeinclusin*

   La primera vez que se incluye este archivo, `MI_LIBRERIA_H` no está definido, por lo que el preprocesador incluye el contenido y define la macro. Las siguientes veces, la macro ya está definida, por lo que el contenido se omite.

4. **Macros predefinidas útiles:** El estándar define varias macros que siempre están disponibles:

```c
#include <stdio.h>
int main() {
    printf("Archivo: %s\n", __FILE__);      // Nombre del archivo actual
    printf("Linea: %d\n", __LINE__);        // Linea actual
    printf("Fecha: %s\n", __DATE__);        // Fecha de compilacion
    printf("Hora: %s\n", __TIME__);         // Hora de compilacion
    printf("ANSI C: %d\n", __STDC__);       // 1 si el compilador es ANSI C
    return 0;
}
```

Archivos Cabecera (`.h`) - La Interfaz Pública

Los archivos de cabecera son la clave para la modularización en C. Actúan como el **contrato** entre el implementador de un módulo y sus usuarios. Contienen las **declaraciones** (qué hay disponible), mientras que los archivos fuente (`.c`) contienen las **definiciones** (cómo funciona).

Qué debe ir en un archivo `.h`

- **Prototipos de funciones públicas:** Las funciones que otros archivos pueden llamar.
- **Declaraciones de tipos:** `typedef`, `struct`, `enum`, `union` que deben ser conocidos por los usuarios.
- **Macros públicas:** Constantes y macros tipo función que forman parte de la interfaz.
- **Declaraciones de variables globales:** Usando `extern` (la definición real va en un `.c`).
- **Guardas de inclusión:** Siempre, para evitar múltiples inclusiones.

Qué NO debe ir en un archivo `.h`

- **Definiciones de funciones:** A menos que sean `static inline` (un caso avanzado). Si pones la definición completa de una función en un `.h` y ese `.h` se incluye en dos archivos `.c` diferentes, tendrás un error de "definición múltiple.^en la fase de enlace.
- **Definiciones de variables:** Sin `extern` o `static`, causarán el mismo problema de definición múltiple.
- **Código ejecutable:** El `.h` es para declaraciones, no para lógica.

Ejemplo Completo de un Módulo Bien Diseñado

```c
#ifndef CALCULOS_H
#define CALCULOS_H

// Constantes
#define PI 3.1415926535
#define GRAVEDAD 9.81

// Tipos
typedef struct {
    float x;
    float y;
} Punto;

// Prototipos de funciones publicas
float calcular_area_circulo(float radio);
float calcular_distancia(Punto p1, Punto p2);
int es_par(int numero);

#endif
```

Listing 65: calculos.h - El archivo de cabecera

```c
#include "calculos.h"
#include <math.h>  // Para sqrt

// Definiciones de las funciones declaradas en el .h
float calcular_area_circulo(float radio) {
    return PI * radio * radio;
}

float calcular_distancia(Punto p1, Punto p2) {
    float dx = p2.x - p1.x;
    float dy = p2.y - p1.y;
    return sqrt(dx*dx + dy*dy);
}

int es_par(int numero) {
    return numero % 2 == 0;
}

// Funcion privada (no declarada en el .h, solo visible ↩
    en este archivo)
static void funcion_privada() {
    // Solo puede ser llamada desde dentro de calculos.c
}
```

Listing 66: calculos.c - La implementación

```c
#include <stdio.h>
#include "calculos.h"  // Incluimos nuestra interfaz

int main() {
    float area = calcular_area_circulo(5.0);
    printf("Area del circulo: %.2f\n", area);

    Punto a = {0.0, 0.0};
    Punto b = {3.0, 4.0};
    float dist = calcular_distancia(a, b);
    printf("Distancia: %.2f\n", dist);

    if (es_par(10)) {
        printf("10 es par.\n");
    }

    // funcion_privada(); // ERROR! No esta declarada en calculos.h
    return 0;
}
```

Listing 67: main.c - Usando el módulo

Para compilar este proyecto:

```
gcc -c calculos.c -o calculos.o
gcc -c main.c -o main.o
gcc main.o calculos.o -o programa -lm
```

Concepto de Proyecto

Un **proyecto** en programación es un conjunto organizado de archivos que, al ser compilados y enlazados juntos, producen uno o más programas o bibliotecas. Un proyecto típico en C incluye:

```
mi_proyecto/
 src/                # Código fuente (.c)
    main.c
    calculos.c
    io.c
 include/            # Archivos de cabecera (.h) publicos
    calculos.h
    io.h
 lib/                # Bibliotecas externas (opcional)
 obj/                # Archivos objeto (.o) (generados)
 bin/                # Ejecutables (generados)
 tests/              # Pruebas unitarias
```

```
docs/              # Documentacion
Makefile           # Automatizacion de la compilacion
```

Concepto de Makefile

Cuando un proyecto tiene muchos archivos, compilar manualmente cada uno es tedioso y propenso a errores. Además, si solo cambias un archivo, quieres recompilar solo ese archivo y enlazar de nuevo, no todo el proyecto.

Make es una herramienta de automatización de compilación que existe desde los años 70. Lee un archivo llamado **Makefile** que contiene reglas sobre cómo construir el proyecto.

Estructura básica de una regla en Makefile

```
objetivo: dependencias
 comandos
```

- **objetivo:** El archivo que queremos generar (ej. `programa`, `main.o`).
- **dependencias:** Archivos de los que depende el objetivo. Si alguna dependencia es más nueva que el objetivo, Make ejecuta los comandos.
- **comandos:** Instrucciones (precedidas por un tabulador, ¡no espacios!) para generar el objetivo.

Ejemplo de Makefile para nuestro proyecto

```
# Variables (para facilitar cambios)
CC = gcc
CFLAGS = -Wall -Wextra -std=c99 -I./include
LDFLAGS = -lm
TARGET = bin/programa
SRCDIR = src
INCDIR = include
OBJDIR = obj
SOURCES = $(wildcard $(SRCDIR)/*.c)
OBJECTS = $(patsubst $(SRCDIR)/%.c, $(OBJDIR)/%.o, ↩
   $(SOURCES))

# Regla por defecto (la primera)
$(TARGET): $(OBJECTS) | $(BINDIR)
    $(CC) $^ -o $@ $(LDFLAGS)

# Como generar archivos .o a partir de .c
$(OBJDIR)/%.o: $(SRCDIR)/%.c | $(OBJDIR)
    $(CC) $(CFLAGS) -c $< -o $@

# Crear directorios si no existen
```

```
$(OBJDIR):
    mkdir -p $(OBJDIR)

$(BINDIR):
    mkdir -p $(BINDIR)

# Limpiar archivos generados
clean:
    rm -rf $(OBJDIR) $(TARGET)

# Regla para ejecutar el programa
run: $(TARGET)
    ./$(TARGET)

# Reglas phony (no representan archivos reales)
.PHONY: clean run
```

Listing 68: Makefile completo

Cómo usarlo:

```
$ make        # Compila y enlaza todo
$ make clean  # Limpia archivos objeto y ejecutable
$ make run    # Compila (si es necesario) y ejecuta el programa
```

¿Por qué es tan útil? Si modificas solo `src/calculos.c`, Make ve que `calculos.o` es más antiguo que `calculos.c`, así que recompila solo `calculos.o`. Luego ve que `programa` depende de `calculos.o` (y de `main.o`), y como `calculos.o` se actualizó, re-enlaza `programa`. ¡Ningún otro archivo se recompila innecesariamente!

Concepto de Biblioteca (Librería)

Una **biblioteca** es una colección de código precompilado que puede ser reutilizado por múltiples programas. En C, hay dos tipos principales:

Bibliotecas Estáticas (`.a` en Linux/Mac, `.lib` en Windows)

- El código de la biblioteca se copia directamente dentro del ejecutable en tiempo de enlace.
- El ejecutable final es más grande, pero es independiente (no necesita archivos adicionales para ejecutarse).
- Si la biblioteca se actualiza, todos los programas que la usan deben ser recompilados.

Crear y usar una biblioteca estática

```
# 1. Compilar los archivos fuente a objetos
gcc -c calculos.c -o calculos.o
```

```
gcc -c io.c -o io.o

# 2. Crear la biblioteca estatica (ar = archiver)
ar rcs libmimath.a calculos.o io.o

# 3. Usar la biblioteca al compilar un programa
gcc main.c -L. -lmimath -o programa
```

Bibliotecas Dinámicas (Compartidas) (`.so` en Linux, `.dylib` en Mac, `.dll` en Windows)

- El código de la biblioteca NO se copia al ejecutable. En su lugar, el ejecutable contiene referencias a la biblioteca.
- La biblioteca se carga en memoria en tiempo de ejecución (cuando el programa se inicia, o incluso después).
- Los ejecutables son más pequeños.
- Múltiples programas pueden compartir la misma biblioteca en memoria.
- Si la biblioteca se actualiza, los programas se benefician sin necesidad de recompilar (siempre que la interfaz no cambie).

Crear y usar una biblioteca dinámica

```
# 1. Compilar con -fPIC (Position Independent Code)
gcc -c -fPIC calculos.c -o calculos.o
gcc -c -fPIC io.c -o io.o

# 2. Crear la biblioteca compartida
gcc -shared -o libmimath.so calculos.o io.o

# 3. Usar la biblioteca
gcc main.c -L. -lmimath -o programa

# 4. Para ejecutar, la biblioteca debe estar en un lugar donde el sistema la encuentre
export LD_LIBRARY_PATH=.:$LD_LIBRARY_PATH  # Linux
./programa
```

Bibliotecas Estándar de C

El estándar de C define un conjunto de bibliotecas que deben estar disponibles en cualquier implementación del lenguaje.

Biblioteca Estándar de Entrada-Salida (<stdio.h>)

Es la biblioteca que más hemos usado. Proporciona funciones para interactuar con el usuario (consola) y con archivos.

Funciones para consola

```c
#include <stdio.h>

int main() {
    // Salida
    printf("Hola, %s! Tienes %d anos.\n", "Carlos", 25);
    puts("Una linea simple");  // A ade \n automaticamente
    putchar('A');              // Imprime un caracter

    // Entrada
    char nombre[50];
    int edad;
    printf("Ingresa tu nombre: ");
    scanf("%s", nombre);        //  Cuidado ! No verifica ↩
        longitud
    printf("Ingresa tu edad: ");
    scanf("%d", &edad);

    // Entrada de linea completa (mas segura)
    getchar(); // Para consumir el \n que quedo del scanf ↩
        anterior
    printf("Ingresa una frase: ");
    fgets(nombre, sizeof(nombre), stdin); // Lee ↩
        incluyendo espacios

    return 0;
}
```

Listing 69: Funciones de consola en stdio.h

Funciones para archivos

```c
#include <stdio.h>

int main() {
    FILE *archivo;
    char linea[100];

    // --- Escribir en un archivo ---
    archivo = fopen("datos.txt", "w"); // "w" = write ↩
        (sobrescribe)
    if (archivo == NULL) {
```

```
        printf("Error al abrir el archivo para escritura\n");
        return 1;
    }

    fprintf(archivo, "Linea 1: Hola mundo\n");
    fputs("Linea 2: Otra linea\n", archivo);
    fputc('X', archivo);

    fclose(archivo); //  Siempre  cerrar el archivo!

    // --- Leer de un archivo ---
    archivo = fopen("datos.txt", "r"); // "r" = read
    if (archivo == NULL) {
        printf("Error al abrir el archivo para lectura\n");
        return 1;
    }

    printf("Contenido del archivo:\n");
    while (fgets(linea, sizeof(linea), archivo) != NULL) {
        printf("%s", linea);
    }

    fclose(archivo);

    // --- Otras formas de apertura ---
    // "a" = append (agregar al final)
    // "r+" = lectura y escritura (el archivo debe existir)
    // "w+" = lectura y escritura (crea o sobrescribe)
    // "a+" = lectura y escritura (agrega al final)

    // Modo binario (a adir 'b'): "rb", "wb", "ab"

    return 0;
}
```

Listing 70: Manejo de archivos en C

Biblioteca para Manejo de Strings (<string.h>)

Proporciona funciones para manipular cadenas de caracteres (arrays de char terminados en \0).

```
#include <stdio.h>
#include <string.h>

int main() {
    char destino[50];
    char origen[] = "Hola, mundo!";
```

```
    char otra[] = "Hola, C!";

    // 1. Copiar strings
    strcpy(destino, origen);  // Copia origen a destino
    printf("strcpy: %s\n", destino);

    // Copia segura con limite
    strncpy(destino, otra, 5); // Copia solo 5 caracteres
    destino[5] = '\0'; //   IMPORTANTE  ! strncpy NO agrega ↩
       \0 si se llena
    printf("strncpy: %s\n", destino);

    // 2. Longitud
    printf("Longitud de '%s': %zu\n", origen, ↩
       strlen(origen));

    // 3. Concatenar
    char saludo[100] = "Hola, ";
    strcat(saludo, "Carlos");
    printf("strcat: %s\n", saludo);

    // Concatenacion segura
    strncat(saludo, " Perez", 5); // Agrega " Perez" pero ↩
       solo 5 chars
    printf("strncat: %s\n", saludo);

    // 4. Comparar
    if (strcmp(origen, otra) == 0) {
        printf("Las cadenas son iguales\n");
    } else {
        printf("Las cadenas son diferentes\n");
    }

    // 5. Buscar
    char *pos = strchr(saludo, 'C'); // Busca 'C' en saludo
    if (pos != NULL) {
        printf("Encontrada 'C' en posicion: %ld\n", pos - ↩
           saludo);
    }

    pos = strstr(saludo, "arlos"); // Busca subcadena
    if (pos != NULL) {
        printf("Subcadena encontrada: %s\n", pos);
    }

    // 6. Tokenizacion (dividir string)
    char str[] = "uno,dos,tres,cuatro";
    char *token = strtok(str, ",");
    while (token != NULL) {
```

```
        printf("Token: %s\n", token);
        token = strtok(NULL, ",");
    }

    return 0;
}
```

Listing 71: Funciones clave de string.h

¡PRECAUCIÓN! Muchas funciones de `string.h` (como `strcpy`, `strcat`) no verifican los límites de los buffers y pueden causar desbordamientos de búfer (una grave vulnerabilidad de seguridad). Siempre que sea posible, usa las versiones con `n`: `strncpy`, `strncat`, o funciones aún más seguras como `strlcpy` (no estándar, pero disponible en algunos sistemas).

Otras Bibliotecas Estándar Importantes

- **<stdlib.h>** (Standard Library):

```
#include <stdlib.h>
int a = atoi("123");          // String a entero
double d = atof("3.14");      // String a double
long l = strtol("0xFF", NULL, 16); // String a long ↩
    en base 16

int *p = malloc(10 * sizeof(int)); // Asignar memoria
if (p != NULL) {
    // usar p
    free(p);                  // Liberar memoria
}

int r = rand();               // Numero aleatorio (0 a ↩
    RAND_MAX)
srand(time(NULL));            // Semilla para rand

exit(0);                      // Terminar programa ↩
    inmediatamente

// Ordenar y buscar (avanzado)
qsort(array, size, sizeof(int), comparar); // ↩
    Ordenamiento rapido
```

- **<math.h>** (Matemáticas):

```
#include <math.h>
double x = sqrt(25.0);        // Raiz cuadrada ↩
    (resultado 5.0)
double y = pow(2.0, 3.0);     // Potencia (2^3 = 8.0)
double z = sin(3.14159 / 2); // Seno
```

```
double a = fabs(-5.5);          // Valor absoluto (5.5)
double b = ceil(3.2);           // Redondear hacia ↩
    arriba (4.0)
double c = floor(3.8);          // Redondear hacia ↩
    abajo (3.0)
double d = round(3.5);          // Redondear al entero ↩
    mas cercano (4.0)
```

Nota: En Linux, al compilar con `<math.h>`, hay que añadir `-lm` al final del comando de compilación: `gcc programa.c -o programa -lm`.

- **<ctype.h>** (Clasificación de caracteres):

```
#include <ctype.h>
if (isalpha('A')) { }         // Es letra?
if (isdigit('5')) { }         // Es digito?
if (isspace(' ')) { }         // Es espacio en blanco?
if (isupper('A')) { }         // Es mayuscula?
char may = toupper('a');       // Convertir a ↩
    mayuscula ('A')
char min = tolower('Z');       // Convertir a ↩
    minuscula ('z')
```

- **<time.h>** (Tiempo y fecha):

```
#include <stdio.h>
#include <time.h>

int main() {
    time_t t = time(NULL);     // Tiempo actual ↩
        (segundos desde 1970)
    printf("Segundos desde 1970: %ld\n", t);

    // Convertir a formato local
    struct tm *info = localtime(&t);
    printf("Fecha: %d/%d/%d\n", info->tm_mday, ↩
        info->tm_mon + 1, info->tm_year + 1900);

    // Medir tiempo de ejecucion
    clock_t inicio = clock();
    // ... codigo a medir ...
    for (long i = 0; i < 100000000; i++);
    clock_t fin = clock();

    double tiempo = (double)(fin - inicio) / ↩
        CLOCKS_PER_SEC;
    printf("Tiempo: %.2f segundos\n", tiempo);

    return 0;
```

```
}
```

Presentación Comparativa de Varios Compiladores

Existen múltiples compiladores de C, cada uno con sus fortalezas y plataformas objetivo.

Compilador	Plataformas Principales	Características Destacadas	Caso de Uso Típico
GCC	Linux, Windows (MinGW, Cygwin), macOS, BSD, embebidos.	Estándar en Unix/-Linux. Muy maduro, múltiples optimizaciones, soporta muchos dialectos de C. Amplísima comunidad.	Desarrollo general en Linux, sistemas embebidos, código abierto.
Clang	Linux, Windows, macOS (compilador por defecto en Xcode), BSD.	Mensajes de error excepcionalmente claros. Compilación rápida. Buen soporte para análisis estático.	Desarrollo en macOS/iOS, proyectos donde se valoren buenos diagnósticos.
MSVC	Windows.	Integración perfecta con Visual Studio. Excelente soporte para API de Windows. Depurador muy potente.	Desarrollo de aplicaciones nativas para Windows, videojuegos.
TCC	Windows, Linux.	Extremadamente rápido y pequeño. Compila casi al instante. Perfecto para aprender o para scripts.	Aprendizaje, ejecución de código C como script, entornos con recursos limitados.
Intel C++	Linux, Windows.	Altamente optimizado para procesadores Intel. Código más rápido en hardware Intel.	Computación de alto rendimiento (HPC), aplicaciones críticas en hardware Intel.

Comparativa de Mensajes de Error (Un ejemplo vale más que mil palabras)

```c
int main() {
    int x = "hola";  // Error: asignar string a int
    printf("Hola\n") // Error: falta punto y coma
    return 0;
}
```

Listing 72: Código con errores

Mensaje de GCC:

```
error: initialization of 'int' from 'char *' makes integer from pointer without a cast
error: expected ';' before 'return'
```

Mensaje de Clang:

```
error: incompatible pointer to integer conversion initializing 'int' with an expressio
error: expected ';' after expression
```

La diferencia es sutil, pero Clang tiende a ser más descriptivo y a sugerir correcciones.

Compilando con Diferentes Compiladores

```
# GCC
gcc -Wall -Wextra -std=c99 programa.c -o programa

# Clang (los parametros son compatibles)
clang -Wall -Wextra -std=c99 programa.c -o programa

# MSVC (en una terminal de desarrollador de Visual Studio)
cl /W4 programa.c

# TCC
tcc programa.c -o programa.exe
```

Resumen de la Unidad VI

Hemos dado un salto cualitativo, pasando de programas sencillos a proyectos complejos y profesionales.

- **Proceso de compilación:** Comprende 4 fases (preprocesamiento, compilación, ensamblado, enlace). Cada fase transforma el código hasta obtener un ejecutable.
- **Preprocesador:** Procesa directivas `#` (include, define, condicionales). Es una herramienta textual que actúa antes de la compilación real.
- **Compilación condicional:** Permite incluir/excluir código basado en condiciones de preprocesamiento (`#ifdef`, `#ifndef`, `#if`, `#elif`, `#else`). Esencial para código multiplataforma y depuración.
- **Archivos cabecera (`.h`):** Son las interfaces de los módulos. Contienen declaraciones, prototipos y macros. Usan guardas de inclusión (`#ifndef ... #define ... #endif`) para evitar inclusiones múltiples.
- **Proyecto:** Estructura organizada de archivos fuente, cabeceras, bibliotecas y herramientas de construcción.
- **Makefile:** Automatiza la compilación, recompilando solo lo necesario y evitando trabajo manual repetitivo.

- **Bibliotecas:** Código precompilado reutilizable. Estáticas (`.a`, `.lib`) se copian al ejecutable; dinámicas (`.so`, `.dll`) se cargan en tiempo de ejecución.
- **Bibliotecas estándar:** Conjunto esencial de funcionalidades: entrada/salida (`stdio.h`), strings (`string.h`), utilidades generales (`stdlib.h`), matemáticas (`math.h`), clasificación de caracteres (`ctype.h`), tiempo (`time.h`), etc.
- **Compiladores:** Diferentes opciones (GCC, Clang, MSVC, TCC) con ventajas según la plataforma y el caso de uso. Conocer sus diferencias ayuda a elegir la herramienta adecuada.

Con esta unidad, ya estás preparado para trabajar en proyectos de cualquier tamaño, organizar tu código de manera profesional y entender todo el proceso que hay detrás de un simple comando de compilación.

Unidad VIII: Entrada y Salida - Comunicación con el Mundo Exterior

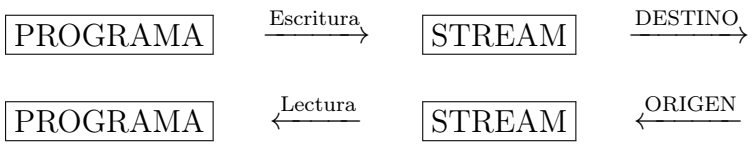
Hasta ahora, nuestros programas han sido entidades relativamente aisladas. Han recibido datos a través de argumentos de línea de comandos o de la función `scanf()`, y han mostrado resultados por la pantalla con `printf()`. Sin embargo, en el mundo real, los programas necesitan comunicarse con el exterior de formas mucho más diversas: leer archivos de configuración, procesar grandes volúmenes de datos almacenados en disco, generar reportes, comunicarse con otros programas, etc.

Esta unidad está dedicada por completo a las operaciones de **entrada y salida (I/O)**, el mecanismo fundamental que permite a nuestros programas interactuar con el mundo exterior.

El Concepto de Stream (Flujo)

Antes de hablar de archivos o de consola, debemos entender el concepto unificador de **stream** (flujo) en C. Un stream es una abstracción que representa un flujo de datos, independientemente de su origen o destino. Piensa en él como una tubería por la que circulan bytes.

Abstracción de Stream



La gran ventaja de esta abstracción es que las funciones de I/O en C trabajan con streams, no directamente con dispositivos. El mismo código que escribe en un archivo puede escribir en la consola o en una conexión de red simplemente cambiando el stream al que se dirige.

Entrada y Salida Estándar

Cuando un programa en C comienza, el sistema operativo le proporciona tres streams predefinidos, accesibles a través de las constantes definidas en `<stdio.h>`:

Stream	Constante	Pr
Entrada estándar	`stdin`	Normalmente conectada al teclad
Salida estándar	`stdout`	Normalmente conectada a la pant
Salida de error estándar	`stderr`	Normalmente conectada a la pantalla, pero separada de `stdo`

```c
#include <stdio.h>

int main() {
    // Escribir en salida estandar
    fprintf(stdout, "Este es un mensaje normal.\n");

    // Escribir en salida de error (inmediato, sin buffer)
    fprintf(stderr, "Este es un mensaje de ERROR.\n");

    // Leer de entrada estandar
    char nombre[50];
    fprintf(stdout, "Como te llamas? ");
    fscanf(stdin, "%s", nombre);  // Equivalente a ↩
       scanf("%s", nombre);

    fprintf(stdout, "Hola, %s!\n", nombre);

    return 0;
}
```

Listing 73: Uso de streams estándar

Redirección: Una característica poderosa de los sistemas operativos tipo Unix (y también en Windows) es que podemos redirigir estos streams. Por ejemplo:

```
$ ./programa < entrada.txt > salida.txt 2> error.txt
```

Esto hará que `stdin` lea de `entrada.txt`, `stdout` escriba en `salida.txt` y `stderr` escriba en `error.txt`, sin cambiar ni una línea de código.

Concepto de Archivo

Un **archivo** es una colección de datos almacenada en un dispositivo persistente (disco duro, SSD, memoria USB, etc.). En C, un archivo no es más que un stream que está conectado a un archivo en el sistema de archivos.

Tipos de Archivos (desde la perspectiva de C)

- **Archivos de texto:** Contienen datos legibles por humanos, organizados en líneas. Cuando se leen o escriben, pueden ocurrir traducciones de caracteres (por ejemplo, `\n` en C puede convertirse en `\r\n` en Windows al escribir).

- **Archivos binarios:** Contienen datos en el mismo formato en que se almacenan en memoria. No hay traducción de caracteres. Son más eficientes para almacenar datos numéricos o estructuras complejas.

Operaciones Básicas con Archivos

Trabajar con archivos en C sigue un patrón de tres pasos: **abrir**, **operar** (leer/escribir), **cerrar**.

Apertura y Cierre: `fopen()` y `fclose()`

```c
#include <stdio.h>

int main() {
    FILE *archivo;  // Puntero a FILE (la estructura que representa un stream)

    // Abrir archivo para escritura
    archivo = fopen("datos.txt", "w");  // "w" = write (crea o sobrescribe)

    if (archivo == NULL) {
        fprintf(stderr, "Error: No se pudo abrir el archivo para escritura.\n");
        return 1;
    }

    // ... operaciones de escritura ...

    // Cerrar el archivo (  IMPORTANTE  !)
    fclose(archivo);

    return 0;
}
```

Listing 74: Abrir y cerrar un archivo

Modos de apertura (el segundo argumento de `fopen()`)

Modo	Significado	Observaciones
`r"`	Lectura (read)	El archivo debe existir.
`"w"`	Escritura (write)	Crea el archivo si no existe; si existe, lo sobrescribe.
`.a"`	Añadir (append)	Crea el archivo si no existe; escribe al final.
`r+"`	Lectura y escritura	El archivo debe existir.
`"w+"`	Lectura y escritura	Crea o sobrescribe.
`.a+"`	Lectura y añadir	Crea si no existe; lectura desde cualquier posición, escritura siempre al
`rb"`, `"wb"`, etc.	Modo binario	Añadir `'b'` para modo binario (ej. `rb"` para leer binario).

Escritura en Archivos

Las funciones de escritura son análogas a las de consola, pero reciben el stream como primer argumento.

```c
#include <stdio.h>

int main() {
    FILE *archivo = fopen("salida.txt", "w");

    if (archivo == NULL) {
        perror("Error al abrir archivo");  // perror muestra el error del sistema
        return 1;
    }

    // fprintf: escritura formateada (como printf)
    fprintf(archivo, "Entero: %d, Float: %.2f, String: %s\n", 42, 3.14159, "hola");

    // fputs: escribe una cadena (sin formato, no a ade \n automaticamente)
    fputs("Esta es una linea de texto.\n", archivo);
    fputs("Esta es otra linea.\n", archivo);

    // fputc: escribe un caracter
    fputc('A', archivo);
    fputc('\n', archivo);

    // fwrite: escritura binaria (para datos en bruto)
    int numeros[] = {10, 20, 30, 40, 50};
    size_t elementos_escritos = fwrite(numeros, sizeof(int), 5, archivo);
    if (elementos_escritos != 5) {
        fprintf(stderr, "Error al escribir datos binarios\n");
    }

    fclose(archivo);
    return 0;
}
```

Listing 75: Escritura en archivos

Lectura de Archivos

Análogamente, tenemos funciones para leer.

```c
#include <stdio.h>
```

```
int main() {
    FILE *archivo = fopen("salida.txt", "r");

    if (archivo == NULL) {
        perror("Error al abrir archivo");
        return 1;
    }

    // fscanf: lectura formateada (como scanf)
    int entero;
    float flotante;
    char cadena[50];
    fscanf(archivo, "Entero: %d, Float: %f, String: %s\n", &entero, &flotante, cadena);
    printf("Leido: %d, %f, %s\n", entero, flotante, cadena);

    // fgets: lee una linea (MAS SEGURA que fscanf para strings)
    char linea[100];
    fgets(linea, sizeof(linea), archivo);  // Lee hasta 99 chars o hasta \n
    printf("Linea: %s", linea);  // fgets incluye el \n si hay espacio

    fgets(linea, sizeof(linea), archivo);
    printf("Linea: %s", linea);

    // fgetc: lee un caracter
    int c = fgetc(archivo);  // Retorna int, no char, para poder retornar EOF
    if (c != EOF) {
        printf("Caracter: %c\n", c);
    }

    // fread: lectura binaria
    int numeros_leidos[5];
    size_t elementos_leidos = fread(numeros_leidos, sizeof(int), 5, archivo);
    if (elementos_leidos == 5) {
        printf("Numeros leidos: ");
        for (int i = 0; i < 5; i++) {
            printf("%d ", numeros_leidos[i]);
        }
        printf("\n");
    }

    fclose(archivo);
    return 0;
```

```
}
```

Listing 76: Lectura de archivos

La Importancia de `fclose()`

Siempre debemos cerrar los archivos que abrimos por varias razones:

1. Los datos escritos con funciones como `fprintf()` o `fwrite()` pueden estar en un **búfer** en memoria, no en el disco físicamente. `fclose()` vacía (flushea) el búfer y garantiza que todos los datos se escriban.
2. Libera recursos del sistema (como descriptores de archivo) que son limitados.
3. Si el programa termina abruptamente sin cerrar, los datos pueden corromperse o perderse.

Funciones de Streams Avanzadas

Además de las funciones básicas, hay otras muy útiles para manipular streams.

Verificación de Fin de Archivo (EOF)

```c
#include <stdio.h>

int main() {
    FILE *archivo = fopen("datos.txt", "r");
    if (archivo == NULL) {
        perror("Error");
        return 1;
    }

    char linea[100];
    while (fgets(linea, sizeof(linea), archivo) != NULL) {
        printf("Leido: %s", linea);
    }

    if (feof(archivo)) {
        printf("Fin de archivo alcanzado.\n");
    } else if (ferror(archivo)) {
        printf("Error de lectura.\n");
    }

    fclose(archivo);
    return 0;
}
```

Listing 77: Detectar fin de archivo

Posicionamiento en el Archivo: `ftell()`, `fseek()`, `rewind()`

Podemos movernos a cualquier posición dentro de un archivo (útil para archivos binarios o de acceso aleatorio).

```
#include <stdio.h>

int main() {
    FILE *archivo = fopen("datos.txt", "r");
    if (archivo == NULL) {
        perror("Error");
        return 1;
    }

    // Obtener la posicion actual (desde el inicio)
    long pos = ftell(archivo);
    printf("Posicion actual: %ld\n", pos);  // 0

    // Leer una linea
    char linea[100];
    fgets(linea, sizeof(linea), archivo);
    printf("Leido: %s", linea);

    pos = ftell(archivo);
    printf("Posicion ahora: %ld\n", pos);

    // Volver al principio
    rewind(archivo);  // Equivalente a fseek(archivo, 0, SEEK_SET);
    printf("Despues de rewind, posicion: %ld\n", ftell(archivo));

    // fseek: moverse a una posicion
    fseek(archivo, 10, SEEK_SET);  // Desde el principio (SEEK_SET), byte 10
    printf("Posicion despues de fseek(...,10,SEEK_SET): %ld\n", ftell(archivo));

    fseek(archivo, 5, SEEK_CUR);   // 5 bytes desde la posicion actual
    printf("Posicion despues de SEEK_CUR +5: %ld\n", ftell(archivo));

    fseek(archivo, -3, SEEK_END);  // 3 bytes antes del final
    printf("Posicion desde el final: %ld\n", ftell(archivo));

    fclose(archivo);
    return 0;
```

```
}
```

Listing 78: Posicionamiento en archivos

Gestión de Errores

```
#include <stdio.h>
#include <errno.h>
#include <string.h>

int main() {
    FILE *archivo = fopen("archivo_que_no_existe.txt", "r");

    if (archivo == NULL) {
        // Forma 1: perror() - muestra el mensaje de error del sistema
        perror("Error al abrir archivo");

        // Forma 2: errno y strerror()
        fprintf(stderr, "Error %d: %s\n", errno, strerror(errno));

        return 1;
    }

    // ... operaciones ...

    if (ferror(archivo)) {
        fprintf(stderr, "Error durante operaciones de archivo.\n");
        clearerr(archivo);  // Limpiar el indicador de error
    }

    fclose(archivo);
    return 0;
}
```

Listing 79: Manejo de errores en archivos

Funciones Tipo Unix de Manejo de I/O (Bajo Nivel)

Hasta ahora hemos usado las funciones de la biblioteca estándar de C (`stdio.h`), que son **portables** (funcionan en cualquier sistema que tenga un compilador de C). Sin embargo, en sistemas Unix/Linux (y también en Windows con ciertas bibliotecas), existen funciones de **bajo nivel** que son llamadas directas al sistema operativo. Estas funciones trabajan con **descriptores de archivo** (file descriptors), que son números enteros que identifican un archivo abierto, en lugar de con punteros `FILE*`.

Comparación: Streams vs. Descriptores

Nivel C (stdio.h)

FILE* (buffered)

↓

Funciones: fprintf, fscanf, fgets, fputs, etc.

↓

BÚFER en espacio de usuario

Nivel Sistema (Unix)

int fd (sin buffer)

↓

Funciones: read, write, open, close, lseek

↓

Llamada directa al SO

↓

SISTEMA OPERATIVO

↓

DISPOSITIVO (disco, etc.)

¿Por qué usar funciones de bajo nivel?

- **Control más fino**: Puedes hacer cosas que las funciones estándar no permiten (ej. operaciones de control de dispositivo específicas con `ioctl`).
- **Eficiencia**: Evitas el buffering doble (aunque el sistema también hace buffering).
- **Comunicación con el sistema**: Necesarias para trabajar con tuberías (pipes), sockets, y otros mecanismos de IPC (Comunicación entre Procesos).

Las funciones clave (en sistemas Unix/POSIX)

Estas funciones están declaradas en `<unistd.h>`, `<fcntl.h>`, etc.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>      // read, write, close
#include <fcntl.h>       // open, modos O_RDONLY, etc.
#include <string.h>
#include <errno.h>

int main() {
    int fd;  // File descriptor (un entero)
    ssize_t bytes_leidos, bytes_escritos;
    char buffer[100];

    // 1. ABRIR ARCHIVO (similar a fopen)
    // O_RDONLY: solo lectura
    // O_WRONLY: solo escritura
    // O_RDWR: lectura y escritura
    // O_CREAT: crear si no existe
    // O_TRUNC: truncar si existe
```

```
    // O_APPEND: a adir al final
    // Los modos (tercer argumento) son los permisos si se crea el archivo
    fd = open("archivo.txt", O_RDWR | O_CREAT | O_TRUNC, 0644);

    if (fd == -1) {
        perror("Error al abrir archivo");
        return 1;
    }

    // 2. ESCRIBIR (write)
    char *mensaje = "Hola, mundo desde bajo nivel!\n";
    bytes_escritos = write(fd, mensaje, strlen(mensaje));

    if (bytes_escritos == -1) {
        perror("Error al escribir");
        close(fd);
        return 1;
    }
    printf("Escritos %ld bytes\n", bytes_escritos);

    // 3. POSICIONAMIENTO (lseek)
    // SEEK_SET: desde el inicio
    // SEEK_CUR: desde la posicion actual
    // SEEK_END: desde el final
    off_t pos = lseek(fd, 0, SEEK_SET);  // Volver al principio
    if (pos == -1) {
        perror("Error en lseek");
        close(fd);
        return 1;
    }

    // 4. LEER (read)
    bytes_leidos = read(fd, buffer, sizeof(buffer) - 1);
    if (bytes_leidos == -1) {
        perror("Error al leer");
        close(fd);
        return 1;
    }

    buffer[bytes_leidos] = '\0';  // Asegurar terminacion nula
    printf("Leidos %ld bytes: %s", bytes_leidos, buffer);

    // 5. CERRAR (close)
    close(fd);

```

```
    // --- Ejemplo de lectura de entrada estandar (stdin ↩
       tiene fd = 0) ---
    printf("\nEscribe algo (max 50 chars): ");
    fflush(stdout);  // Asegurar que el mensaje se muestra

    bytes_leidos = read(0, buffer, 50);  // 0 es ↩
       STDIN_FILENO
    if (bytes_leidos > 0) {
        buffer[bytes_leidos] = '\0';
        printf("Leido del teclado: %s", buffer);
    }

    return 0;
}
```

Listing 80: Funciones de I/O de bajo nivel

Tabla Comparativa: Funciones Estándar vs. Unix

Operación	Estándar (stdio.h)	Unix (unistd.h, fcntl.h)
Abrir archivo	`fopen()`	`open()`
Cerrar archivo	`fclose()`	`close()`
Leer	`fread()`, `fscanf()`, `fgets()`	`read()`
Escribir	`fwrite()`, `fprintf()`, `fputs()`	`write()`
Posicionarse	`fseek()`, `ftell()`, `rewind()`	`lseek()`
Manejo de errores	`ferror()`, `perror()`	`errno`, `perror()`
Stream estándar	`stdin`, `stdout`, `stderr`	0 (STDIN_FILENO), 1 (STDOUT_FILENO), 2 (ST
Buffering	Sí (controlable con `setbuf()`)	No (depende del sistema operativo
Portabilidad	Alta (C estándar)	Baja (solo sistemas Unix/POSIX)

Ejemplo: Copiar un archivo con funciones de bajo nivel

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>

#define BUFFER_SIZE 4096

int main(int argc, char *argv[]) {
    if (argc != 3) {
        fprintf(stderr, "Uso: %s origen destino\n", ↩
           argv[0]);
        return 1;
    }

    int fd_origen = open(argv[1], O_RDONLY);
```

```
    if (fd_origen == -1) {
        perror("Error abriendo origen");
        return 1;
    }

    int fd_destino = open(argv[2], O_WRONLY | O_CREAT | ↩
       O_TRUNC, 0644);
    if (fd_destino == -1) {
        perror("Error creando destino");
        close(fd_origen);
        return 1;
    }

    char buffer[BUFFER_SIZE];
    ssize_t bytes_leidos, bytes_escritos;

    while ((bytes_leidos = read(fd_origen, buffer, ↩
       BUFFER_SIZE)) > 0) {
        bytes_escritos = write(fd_destino, buffer, ↩
           bytes_leidos);
        if (bytes_escritos != bytes_leidos) {
            perror("Error escribiendo");
            close(fd_origen);
            close(fd_destino);
            return 1;
        }
    }

    if (bytes_leidos == -1) {
        perror("Error leyendo");
    }

    close(fd_origen);
    close(fd_destino);

    printf("Archivo copiado exitosamente.\n");
    return 0;
}
```

Listing 81: Programa cp simplificado

Resumen de la Unidad VIII

Hemos explorado en profundidad cómo los programas en C se comunican con el exterior.

- **Streams:** Abstracción unificadora para cualquier flujo de datos. La biblioteca estándar trabaja con `FILE*`.

- **Entrada/Salida estándar:** `stdin`, `stdout`, `stderr` son streams preabiertos para consola.
- **Archivos:** Conjunto de datos persistentes. Se acceden siguiendo el patrón abrir-operar-cerrar.
- **Modos de apertura:** `r"`, `"w"`, `.ª"`, y sus variantes con `'+'` y `'b'` para binario.
- **Funciones de stream:** `fopen`, `fclose`, `fprintf`, `fscanf`, `fgets`, `fputs`, `fread`, `fwrite`, `fseek`, `ftell`, `rewind`, `feof`, `ferror`.
- **Funciones de bajo nivel (Unix):** `open`, `close`, `read`, `write`, `lseek`. Trabajan con descriptores de archivo (enteros). Ofrecen más control pero menos portabilidad.
- **Gestión de errores:** `perror` y `strerror(errno)` para obtener mensajes descriptivos de los errores del sistema.

Con estas herramientas, nuestros programas ya pueden interactuar plenamente con el sistema de archivos, procesar datos persistentes y comunicarse con otros programas a través de tuberías y redirecciones. En la siguiente unidad, aplicaremos estos conocimientos para explorar algoritmos clásicos de ordenamiento y búsqueda.

Unidad IX: Algoritmos de Ordenamiento y Búsqueda - Organizando y Encontrando Datos

En las unidades anteriores, hemos aprendido a almacenar datos en arrays y a manipularlos. Sin embargo, en el mundo real, los datos rara vez vienen en el orden que necesitamos. Imagina una guía telefónica sin ordenar, o una lista de estudiantes de la que queremos encontrar a alguien por su nombre. Sería ineficiente hasta el punto de ser inútil.

Esta unidad está dedicada a dos familias de algoritmos fundamentales en computación:

- **Ordenamiento:** Organizar los elementos de una lista según un criterio (numérico, alfabético, etc.).
- **Búsqueda:** Encontrar un elemento específico dentro de una colección de datos.

Entender estos algoritmos no solo es crucial para resolver problemas prácticos, sino que también nos enseña a pensar en términos de **eficiencia** y **complejidad computacional**.

Conceptos Preliminares

Antes de sumergirnos en los algoritmos, debemos establecer un marco común.

¿Por qué ordenar?

- **Búsqueda eficiente:** En una lista ordenada, podemos encontrar elementos mucho más rápido (búsqueda binaria vs. búsqueda secuencial).
- **Presentación de datos:** Los humanos entendemos mejor la información ordenada.
- **Eliminación de duplicados:** Es más fácil detectar duplicados en una lista ordenada.
- **Optimización de otros algoritmos:** Muchos algoritmos (como ciertos de grafos) asumen datos ordenados.

Estabilidad en ordenamiento

Un algoritmo de ordenamiento es **estable** si mantiene el orden relativo de elementos con claves iguales. Por ejemplo, si ordenamos una lista de personas por edad, y dos personas tienen la misma edad, un algoritmo estable mantendrá el orden en que aparecían originalmente.

Notación de Complejidad (Breve introducción)

Para comparar algoritmos, usamos la **notación Big O** que describe cómo crece el tiempo de ejecución (o el uso de memoria) a medida que crece el tamaño de la entrada (n).

- **O(1):** Tiempo constante (no depende de n).
- **O(log n):** Tiempo logarítmico (muy eficiente).
- **O(n):** Tiempo lineal (crece proporcionalmente a n).
- **O(n log n):** Tiempo linealítmico (eficiente para ordenar).
- **O(n^2):** Tiempo cuadrático (ineficiente para grandes n).

Búsqueda Secuencial (Lineal)

Es el algoritmo de búsqueda más simple. Recorremos el elemento por elemento desde el principio hasta que encontramos el que buscamos o llegamos al final.

Visualización de Búsqueda Secuencial

```
Array: [5, 2, 8, 1, 9, 3]
Buscar: 8

Paso 1: [5] -> ¿Es 8? No.
Paso 2: [5, 2] -> ¿Es 8? No.
Paso 3: [5, 2, 8] -> ¿Es 8? ¡SÍ! Encontrado en índice 2.
```

Implementación en C

```c
#include <stdio.h>

// Busca 'valor' en 'arr' de tama o 'n'
// Retorna el indice si lo encuentra, -1 si no
int busqueda_secuencial(int arr[], int n, int valor) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == valor) {
            return i;  // Encontrado
        }
```

```c
    }
    return -1;  // No encontrado
}

int main() {
    int numeros[] = {5, 2, 8, 1, 9, 3};
    int n = sizeof(numeros) / sizeof(numeros[0]);
    int buscar = 8;

    int indice = busqueda_secuencial(numeros, n, buscar);

    if (indice != -1) {
        printf("Valor %d encontrado en indice %d\n", ↩
           buscar, indice);
    } else {
        printf("Valor %d no encontrado\n", buscar);
    }

    return 0;
}
```

Listing 1: Búsqueda secuencial

Análisis

- **Complejidad temporal:**
  - Mejor caso: O(1) (el elemento está al principio).
  - Peor caso: O(n) (el elemento está al final o no está).
  - Caso promedio: O(n).
- **Ventajas:** Simple, funciona en arrays no ordenados.
- **Desventajas:** Ineficiente para arrays grandes.

Búsqueda Binaria

La búsqueda binaria es mucho más rápida, pero requiere que el array esté **ordenado**. Funciona dividiendo repetidamente el intervalo de búsqueda a la mitad.

Visualización de Búsqueda Binaria

```
Array ordenado: [1, 3, 5, 7, 9, 11, 13, 15, 17]
Buscar: 11

Paso 1: [1, 3, 5, 7, 9, 11, 13, 15, 17]
         ^ bajo        ^ medio        ^ alto
```

```
        (0)           (4)           (8)
        arr[4] = 9. ¿9 < 11? Sí. El elemento debe estar a la derecha.
        nuevo bajo = medio + 1 = 5.

Paso 2: [1, 3, 5, 7, 9, 11, 13, 15, 17]
                        ^ bajo ^ medio ^ alto
                        (5)    (6)     (8)
        arr[6] = 13. ¿13 < 11? No. ¿13 > 11? Sí. El elemento debe estar a la izquierc
        nuevo alto = medio - 1 = 5.

Paso 3: [1, 3, 5, 7, 9, 11, 13, 15, 17]
                        ^ bajo
                        ^ medio
                        ^ alto (todos en 5)
        arr[5] = 11. ¿11 == 11? ¡SÍ! Encontrado en índice 5.
```

Implementación en C (Iterativa y Recursiva)

```c
#include <stdio.h>

int busqueda_binaria_iterativa(int arr[], int n, int valor) {
    int bajo = 0;
    int alto = n - 1;

    while (bajo <= alto) {
        int medio = bajo + (alto - bajo) / 2;  // Evita posible desbordamiento

        if (arr[medio] == valor) {
            return medio;  // Encontrado
        }
        else if (arr[medio] < valor) {
            bajo = medio + 1;  // Buscar en la mitad derecha
        }
        else {
            alto = medio - 1;  // Buscar en la mitad izquierda
        }
    }

    return -1;  // No encontrado
}

int busqueda_binaria_recursiva(int arr[], int bajo, int alto, int valor) {
    if (bajo > alto) {
```

```
        return -1;  // Caso base: no encontrado
    }

    int medio = bajo + (alto - bajo) / 2;

    if (arr[medio] == valor) {
        return medio;
    }
    else if (arr[medio] < valor) {
        return busqueda_binaria_recursiva(arr, medio + 1, alto, valor);
    }
    else {
        return busqueda_binaria_recursiva(arr, bajo, medio - 1, valor);
    }
}

int main() {
    int numeros[] = {1, 3, 5, 7, 9, 11, 13, 15, 17};
    int n = sizeof(numeros) / sizeof(numeros[0]);
    int buscar = 11;

    int indice1 = busqueda_binaria_iterativa(numeros, n, buscar);
    int indice2 = busqueda_binaria_recursiva(numeros, 0, n-1, buscar);

    printf("Iterativa: Valor %d en indice %d\n", buscar, indice1);
    printf("Recursiva: Valor %d en indice %d\n", buscar, indice2);

    return 0;
}
```

Listing 2: Búsqueda binaria iterativa

Análisis

- **Complejidad temporal:** O(log n) en todos los casos (mejor, peor y promedio).
- **Ventajas:** Extremadamente rápida incluso para arrays muy grandes.
- **Desventajas:** Requiere que el array esté ordenado.

Ordenamiento por Inserción (Insertion Sort)

Es uno de los algoritmos más intuitivos. Funciona como ordenamos naipes en la mano: tomamos un elemento y lo insertamos en la posición correcta entre los ya ordenados.

Visualización de Inserción

```
Array inicial: [5, 2, 4, 6, 1, 3]

Paso 1 (i=1, valor=2): Comparar con 5. 2 < 5, desplazar 5 a la derecha.
    [5, 5, 4, 6, 1, 3] -> Insertar 2 en posicion 0.
    [2, 5, 4, 6, 1, 3]

Paso 2 (i=2, valor=4): Comparar con 5. 4 < 5, desplazar 5.
    [2, 5, 5, 6, 1, 3] -> Comparar con 2. 4 > 2, insertar en posicion 1.
    [2, 4, 5, 6, 1, 3]

Paso 3 (i=3, valor=6): Comparar con 5. 6 > 5, no hay desplazamiento.
    [2, 4, 5, 6, 1, 3]

Paso 4 (i=4, valor=1): Comparar con 6,5,4,2. Desplazar todos y insertar al inicio.
    [1, 2, 4, 5, 6, 3]

Paso 5 (i=5, valor=3): Comparar con 6,5,4. Insertar entre 2 y 4.
    [1, 2, 3, 4, 5, 6]  -> Ordenado!
```

Implementación en C

```c
#include <stdio.h>

void insertion_sort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int valor_actual = arr[i];
        int j = i - 1;

        // Desplazar elementos mayores que valor_actual a la derecha
        while (j >= 0 && arr[j] > valor_actual) {
            arr[j + 1] = arr[j];
            j--;
        }

        // Insertar valor_actual en su lugar
        arr[j + 1] = valor_actual;

        // Mostrar paso a paso (opcional)
        printf("Paso %d: ", i);
```

```c
        for (int k = 0; k < n; k++) {
            printf("%d ", arr[k]);
        }
        printf("\n");
    }
}

int main() {
    int numeros[] = {5, 2, 4, 6, 1, 3};
    int n = sizeof(numeros) / sizeof(numeros[0]);

    printf("Array original: ");
    for (int i = 0; i < n; i++) printf("%d ", numeros[i]);
    printf("\n\n");

    insertion_sort(numeros, n);

    printf("\nArray ordenado: ");
    for (int i = 0; i < n; i++) printf("%d ", numeros[i]);
    printf("\n");

    return 0;
}
```

Listing 3: Ordenamiento por inserción

Análisis

- **Complejidad temporal:**
  - Mejor caso (array ya ordenado): O(n) (solo compara, no desplaza).
  - Peor caso (array en orden inverso): $O(n^2)$.
  - Caso promedio: $O(n^2)$.
- **Complejidad espacial:** O(1) (in-place, no usa memoria adicional).
- **Estabilidad:** Sí, es estable.
- **Ventajas:** Simple, eficiente para arrays pequeños o casi ordenados, ordena en línea (puede ordenar mientras recibe datos).
- **Desventajas:** Ineficiente para arrays grandes.

Ordenamiento por Selección (Selection Sort)

Este algoritmo funciona seleccionando repetidamente el elemento más pequeño (o más grande) del array desordenado y colocándolo al principio.

Visualización de Selección

```
Array inicial: [64, 25, 12, 22, 11]

Paso 1: Encontrar el mínimo entre [0..4] (11). Intercambiar con posicion 0.
    [11, 25, 12, 22, 64]

Paso 2: Encontrar el mínimo entre [1..4] (12). Intercambiar con posicion 1.
    [11, 12, 25, 22, 64]

Paso 3: Encontrar el mínimo entre [2..4] (22). Intercambiar con posicion 2.
    [11, 12, 22, 25, 64]

Paso 4: Encontrar el mínimo entre [3..4] (25). Ya esta en su lugar.
    [11, 12, 22, 25, 64]

Paso 5: Solo queda un elemento, fin.
```

Implementación en C

```c
#include <stdio.h>

void selection_sort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        // Encontrar el indice del minimo elemento en arr[i..n-1]
        int indice_minimo = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[indice_minimo]) {
                indice_minimo = j;
            }
        }

        // Intercambiar el minimo encontrado con el elemento en posicion i
        if (indice_minimo != i) {
            int temp = arr[i];
            arr[i] = arr[indice_minimo];
            arr[indice_minimo] = temp;
        }

        // Mostrar paso a paso
        printf("Paso %d: ", i+1);
        for (int k = 0; k < n; k++) {
            printf("%d ", arr[k]);
        }
        printf("\n");
    }
```

```c
}

int main() {
    int numeros[] = {64, 25, 12, 22, 11};
    int n = sizeof(numeros) / sizeof(numeros[0]);

    printf("Array original: ");
    for (int i = 0; i < n; i++) printf("%d ", numeros[i]);
    printf("\n\n");

    selection_sort(numeros, n);

    printf("\nArray ordenado: ");
    for (int i = 0; i < n; i++) printf("%d ", numeros[i]);
    printf("\n");

    return 0;
}
```

Listing 4: Ordenamiento por selección

Análisis

- **Complejidad temporal:** O(n^2) en todos los casos (siempre hace el mismo número de comparaciones).
- **Complejidad espacial:** O(1) (in-place).
- **Estabilidad:** No es estable (el intercambio puede alterar el orden de elementos iguales).
- **Ventajas:** Simple, hace el mínimo número de intercambios (n-1), útil cuando escribir en memoria es costoso.
- **Desventajas:** Siempre es O(n^2), incluso si el array está ordenado.

Ordenamiento Shell (Shell Sort)

El Shell Sort es una optimización del ordenamiento por inserción. Fue creado por Donald Shell en 1959. La idea es permitir el intercambio de elementos que están lejanos, no solo adyacentes.

Funciona dividiendo el array original en subarrays que consisten en elementos separados por un "gap"(hueco). Estos subarrays se ordenan con inserción. Luego se reduce el gap y se repite, hasta que el gap es 1, que es un ordenamiento por inserción normal, pero para entonces el array ya está çasi ordenado".

Visualización de Shell Sort (con gap decreciente)

```
Array inicial: [9, 8, 7, 6, 5, 4, 3, 2, 1]

Gap = 4 (floor(9/2)): Subarrays con indices separados por 4:
    Subarray1 (indices 0,4,8): [9, 5, 1] -> ordenado: [1, 5, 9]
    Subarray2 (indices 1,5):    [8, 4]    -> ordenado: [4, 8]
    Subarray3 (indices 2,6):    [7, 3]    -> ordenado: [3, 7]
    Subarray4 (indices 3,7):    [6, 2]    -> ordenado: [2, 6]
    Resultado tras gap=4: [1, 4, 3, 2, 5, 8, 7, 6, 9]

Gap = 2: Subarrays con indices separados por 2:
    Subarray1 (indices 0,2,4,6,8): [1, 3, 5, 7, 9] -> ya ordenado
    Subarray2 (indices 1,3,5,7):   [4, 2, 8, 6]    -> ordenado: [2, 4, 6, 8]
    Resultado: [1, 2, 3, 4, 5, 6, 7, 8, 9]

Gap = 1: Inserción normal sobre array casi ordenado (muy rapido)
```

Implementación en C

```c
#include <stdio.h>

void shell_sort(int arr[], int n) {
    // Comenzar con un gap grande, luego reducirlo
    for (int gap = n / 2; gap > 0; gap /= 2) {

        // Hacer una ordenacion por insercion para este gap
        for (int i = gap; i < n; i++) {
            int temp = arr[i];
            int j;

            // Desplazar elementos separados por gap
            for (j = i; j >= gap && arr[j - gap] > temp; j -= gap) {
                arr[j] = arr[j - gap];
            }

            arr[j] = temp;
        }

        // Mostrar progreso (opcional)
        printf("Gap = %d: ", gap);
        for (int k = 0; k < n; k++) {
            printf("%d ", arr[k]);
        }
        printf("\n");
    }
}

```

```c
int main() {
    int numeros[] = {9, 8, 7, 6, 5, 4, 3, 2, 1};
    int n = sizeof(numeros) / sizeof(numeros[0]);

    printf("Array original: ");
    for (int i = 0; i < n; i++) printf("%d ", numeros[i]);
    printf("\n\n");

    shell_sort(numeros, n);

    printf("\nArray ordenado: ");
    for (int i = 0; i < n; i++) printf("%d ", numeros[i]);
    printf("\n");

    return 0;
}
```

Listing 5: Ordenamiento Shell

Análisis

- **Complejidad temporal:** Depende de la secuencia de gaps. Para la secuencia original (dividir por 2), es aproximadamente $O(n^2)$ en el peor caso, pero en la práctica es mucho mejor. Con secuencias optimizadas, puede llegar a $O(n \log^2 n)$ o incluso $O(n \log n)$ en algunos casos.
- **Complejidad espacial:** $O(1)$ (in-place).
- **Estabilidad:** No es estable.
- **Ventajas:** Mucho más rápido que inserción y selección para arrays medianos/grandes. No usa memoria adicional.
- **Desventajas:** Su análisis teórico es complejo. El rendimiento depende críticamente de la secuencia de gaps elegida.

Dividir y Conquistar (Divide and Conquer)

La estrategia de "Dividir y Conquistar.es uno de los paradigmas más poderosos en el diseño de algoritmos. Consiste en tres pasos:

1. **Dividir:** El problema se divide en subproblemas más pequeños, generalmente de la misma naturaleza.
2. **Conquistar:** Los subproblemas se resuelven recursivamente.
3. **Combinar:** Las soluciones de los subproblemas se combinan para obtener la solución del problema original.

Visualización de Dividir y Conquistar

```
Problema original: [8, 3, 5, 1, 9, 2, 7, 4]
                      /                    \
        [8, 3, 5, 1]                       [9, 2, 7, 4]
         /         \                       /         \
    [8, 3]       [5, 1]              [9, 2]       [7, 4]
     /    \       /    \              /    \       /    \
   [8]    [3]    [5]    [1]          [9]    [2]    [7]    [4]
     \    /       \    /              \    /       \    /
    [3, 8]       [1, 5]              [2, 9]       [4, 7]
         \         /                       \         /
        [1, 3, 5, 8]                       [2, 4, 7, 9]
                      \                    /
                    [1, 2, 3, 4, 5, 7, 8, 9]
```

Algoritmos Basados en Dividir y Conquistar

Si bien el programa de estudios no especifica MergeSort o QuickSort en esta unidad, es imposible hablar de "Dividir y Conquistar"sin mencionarlos. Estos algoritmos son la aplicación más clara de esta estrategia.

Merge Sort (Ordenamiento por Mezcla)

Merge Sort es un algoritmo de ordenamiento estable que sigue el paradigma de dividir y conquistar.

1. **Dividir:** El array se divide en dos mitades (aproximadamente iguales).
2. **Conquistar:** Cada mitad se ordena recursivamente aplicando Merge Sort.
3. **Combinar:** Las dos mitades ordenadas se fusionan (merge) en un único array ordenado.

```c
#include <stdio.h>
#include <stdlib.h>

// Funcion para fusionar dos subarrays ordenados
void merge(int arr[], int izquierda, int medio, int derecha) {
    int i, j, k;
    int n1 = medio - izquierda + 1;
    int n2 = derecha - medio;

    // Crear arrays temporales
    int *L = (int*)malloc(n1 * sizeof(int));
    int *R = (int*)malloc(n2 * sizeof(int));
```

```
    // Copiar datos a los arrays temporales
    for (i = 0; i < n1; i++)
        L[i] = arr[izquierda + i];
    for (j = 0; j < n2; j++)
        R[j] = arr[medio + 1 + j];

    // Fusionar los arrays temporales de vuelta a ↩
       arr[izquierda..derecha]
    i = 0;  // Indice inicial del primer subarray
    j = 0;  // Indice inicial del segundo subarray
    k = izquierda;  // Indice inicial del array fusionado

    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    // Copiar los elementos restantes de L[], si los hay
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    // Copiar los elementos restantes de R[], si los hay
    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }

    free(L);
    free(R);
}

// Funcion principal de Merge Sort
void merge_sort(int arr[], int izquierda, int derecha) {
    if (izquierda < derecha) {
        // Calcula el punto medio
        int medio = izquierda + (derecha - izquierda) / 2;

        // Ordenar las dos mitades recursivamente
```

```c
        merge_sort(arr, izquierda, medio);
        merge_sort(arr, medio + 1, derecha);

        // Fusionar las mitades ordenadas
        merge(arr, izquierda, medio, derecha);
    }
}

void print_array(int arr[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main() {
    int arr[] = {38, 27, 43, 3, 9, 82, 10};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Array original: ");
    print_array(arr, n);

    merge_sort(arr, 0, n - 1);

    printf("Array ordenado: ");
    print_array(arr, n);

    return 0;
}
```

Listing 6: Merge Sort en C

Análisis de Merge Sort

- **Complejidad temporal:** O(n log n) en todos los casos (mejor, peor y promedio).
- **Complejidad espacial:** O(n) (requiere memoria adicional para los arrays temporales).
- **Estabilidad:** Sí, es estable.
- **Ventajas:** Rendimiento garantizado O(n log n), estable.
- **Desventajas:** Requiere memoria extra O(n).

Quick Sort (Ordenamiento Rápido)

Quick Sort es otro algoritmo de dividir y conquistar, pero con un enfoque diferente. Es el algoritmo de ordenamiento más utilizado en la práctica.

1. **Dividir:** Se selecciona un elemento como pivote. El array se particiona de modo que todos los elementos menores que el pivote estén a su izquierda, y todos los mayores a su derecha.

2. **Conquistar:** Las subarrays izquierdo y derecho se ordenan recursivamente aplicando Quick Sort.

3. **Combinar:** No hay paso de combinación, ya que después de la partición, los subarrays están en el lugar correcto relativo al pivote.

```c
#include <stdio.h>

// Funcion para intercambiar dos elementos
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Funcion de particion: coloca el pivote en su posicion correcta
// y retorna su indice
int partition(int arr[], int bajo, int alto) {
    // Elegir el ultimo elemento como pivote
    int pivote = arr[alto];

    // Indice del elemento mas pequeno (indica la posicion correcta del pivote)
    int i = (bajo - 1);

    for (int j = bajo; j <= alto - 1; j++) {
        // Si el elemento actual es menor o igual que el pivote
        if (arr[j] <= pivote) {
            i++;  // Incrementar el indice del elemento mas pequeno
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[alto]);
    return (i + 1);
}

// Funcion principal de Quick Sort
void quick_sort(int arr[], int bajo, int alto) {
    if (bajo < alto) {
        // pi es el indice de particion, arr[pi] ya esta en su lugar
        int pi = partition(arr, bajo, alto);
```

```c

        // Ordenar recursivamente los elementos antes y despues de la particion
        quick_sort(arr, bajo, pi - 1);
        quick_sort(arr, pi + 1, alto);
    }
}

void print_array(int arr[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main() {
    int arr[] = {10, 7, 8, 9, 1, 5};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Array original: ");
    print_array(arr, n);

    quick_sort(arr, 0, n - 1);

    printf("Array ordenado: ");
    print_array(arr, n);

    return 0;
}
```

Listing 7: Quick Sort en C

Análisis de Quick Sort

- **Complejidad temporal:**
  - Mejor caso: O(n log n) (cuando el pivote siempre divide el array en mitades iguales).
  - Peor caso: $O(n^2)$ (cuando el pivote es siempre el mínimo o máximo, ej. array ya ordenado y se elige el último como pivote).
  - Caso promedio: O(n log n).
- **Complejidad espacial:** O(log n) (por la pila de recursión, es in-place).
- **Estabilidad:** No es estable (el intercambio puede alterar el orden de elementos iguales).
- **Ventajas:** Muy rápido en la práctica, in-place (no requiere memoria extra significativa).
- **Desventajas:** Peor caso $O(n^2)$, no es estable.

Comparativa de Algoritmos de Ordenamiento

Algoritmo	Mejor Caso	Peor Caso	Promedio	Espacio
Inserción	O(n)	$O(n^2)$	$O(n^2)$	O(1)
Selección	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)
Shell	O(n log n)	$O(n^2)$	Depende	O(1)
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)
Quick Sort	O(n log n)	$O(n^2)$	O(n log n)	O(log n)

Aplicación Práctica: Búsqueda Binaria en Array Ordenado

Ahora que sabemos ordenar, podemos aplicar búsqueda binaria de manera eficiente.

```c
#include <stdio.h>
#include <stdlib.h>

// Funciones de ordenamiento (seleccion, por simplicidad)
void selection_sort(int arr[], int n) {
    for (int i = 0; i < n-1; i++) {
        int min_idx = i;
        for (int j = i+1; j < n; j++)
            if (arr[j] < arr[min_idx])
                min_idx = j;
        if (min_idx != i) {
            int temp = arr[i];
            arr[i] = arr[min_idx];
            arr[min_idx] = temp;
        }
    }
}

// B squeda binaria
int busqueda_binaria(int arr[], int n, int valor) {
    int bajo = 0, alto = n-1;
    while (bajo <= alto) {
        int medio = bajo + (alto - bajo) / 2;
        if (arr[medio] == valor)
            return medio;
        else if (arr[medio] < valor)
            bajo = medio + 1;
        else
            alto = medio - 1;
    }
    return -1;
}

int main() {
```

```c
    int datos[] = {42, 17, 8, 99, 23, 56, 11, 34, 77, 5};
    int n = sizeof(datos) / sizeof(datos[0]);
    int buscar = 23;

    printf("Datos originales: ");
    for (int i = 0; i < n; i++) printf("%d ", datos[i]);
    printf("\n");

    // Ordenar
    selection_sort(datos, n);
    printf("Datos ordenados:  ");
    for (int i = 0; i < n; i++) printf("%d ", datos[i]);
    printf("\n");

    // Buscar
    int indice = busqueda_binaria(datos, n, buscar);
    if (indice != -1) {
        printf("Valor %d encontrado en indice %d\n", ↩
           buscar, indice);
    } else {
        printf("Valor %d no encontrado\n", buscar);
    }

    return 0;
}
```

Listing 8: Sistema completo: ordenar y buscar

Resumen de la Unidad IX

Hemos explorado los algoritmos fundamentales para organizar y encontrar datos.

- **Búsqueda:**
  - **Secuencial:** Simple, O(n), funciona en arrays no ordenados.
  - **Binaria:** Muy eficiente O(log n), pero requiere array ordenado.
- **Ordenamiento:**
  - **Inserción:** $O(n^2)$, bueno para arrays pequeños o casi ordenados, estable.
  - **Selección:** $O(n^2)$ siempre, minimiza intercambios, inestable.
  - **Shell:** Mejora de inserción, mucho más rápido, O(n log n) en la práctica.
  - **Merge Sort:** O(n log n) garantizado, estable, requiere memoria extra.
  - **Quick Sort:** O(n log n) promedio, in-place, más rápido en la práctica, pero $O(n^2)$ en el peor caso.

- **Dividir y conquistar:** Paradigma poderoso para diseñar algoritmos eficientes (Merge Sort, Quick Sort, y muchos otros).

Estos algoritmos no solo son herramientas prácticas, sino que también nos enseñan a pensar en términos de eficiencia y a elegir la estrategia adecuada según el contexto. En la siguiente unidad, exploraremos cómo estos conceptos evolucionan en el paradigma de la programación orientada a objetos.

Unidad X: Introducción a la Programación Orientada a Objetos - Un Nuevo Paradigma

Hasta ahora, hemos trabajado exclusivamente con C, un lenguaje **procedural** (o estructurado). En este paradigma, nos enfocamos en las **funciones** que realizan tareas y en los **datos** que esas funciones procesan. Los datos y las funciones están separados.

Sin embargo, a medida que los programas crecen en complejidad, este enfoque puede volverse difícil de manejar. Surge entonces un nuevo paradigma: la **Programación Orientada a Objetos (POO)**. En POO, combinamos datos y funciones en una sola entidad llamada **objeto**. Los objetos son como çápsulas"que contienen tanto información (atributos) como comportamientos (métodos).

En esta unidad, exploraremos los conceptos fundamentales de la POO. Aunque C no es un lenguaje orientado a objetos, sentaremos las bases conceptuales y luego haremos una introducción a **C++**, que es una extensión de C que sí soporta POO.

Paradigma Procedural vs. Orientado a Objetos

```
PROCEDURAL:
+-------------------+
|  Datos (struct)   |  <---- Separados ----  Funciones que operan
|  - nombre         |                        sobre los datos
|  - edad           |                        calcular_imc(Persona p)
|  - altura         |                        imprimir_persona(Persona p)
+-------------------+                        cambiar_nombre(Persona p, char *nuevo)

ORIENTADO A OBJETOS:
+-----------------------------------+
|            OBJETO                 |
|  +----------------------------+  |
|  |  ATRIBUTOS (datos)          |  |
|  |  - nombre                    |  |
|  |  - edad                      |  |
|  |  - altura                    |  |
|  +----------------------------+  |
|  |  MÉTODOS (funciones)         |  |
|  |  + calcular_imc()            |  |
```

```
|  |  + imprimir()                 |  |
|  |  + cambiar_nombre()           |  |
|  +-----------------------------+  |
+-----------------------------------+
```

Conceptos Fundamentales de la POO

La POO se basa en cuatro pilares fundamentales:

Abstracción de Datos

La **abstracción** consiste en simplificar la realidad modelando solo las características esenciales de un objeto, ignorando los detalles irrelevantes. Es como cuando usamos un celular: no necesitamos saber cómo funciona internamente el procesador para hacer una llamada; solo necesitamos conocer la interfaz (la pantalla, los botones).

En programación, la abstracción nos permite:

- Ocultar la complejidad interna.
- Exponer solo las operaciones relevantes (la interfaz pública).
- Cambiar la implementación interna sin afectar a quienes usan el objeto.

Ya vimos un ejemplo de abstracción en C con los Tipos de Datos Abstractos (TDA) en la Unidad IV, donde ocultamos la estructura interna de un `Punto` y solo expusimos funciones para manipularlo. La POO lleva este concepto al siguiente nivel, integrando datos y operaciones en una sola entidad.

Clases y Objetos

- **Clase:** Es una "plantilla.º "molde"que define las características (atributos) y comportamientos (métodos) que tendrán los objetos de ese tipo. Es un tipo de dato definido por el programador. No ocupa memoria; es solo una definición.
- **Objeto:** Es una **instancia** concreta de una clase. Tiene valores específicos para sus atributos y ocupa memoria. Podemos tener muchos objetos de la misma clase.

Clase vs. Objetos

```
CLASE: "Perro"
+-----------------------------------+
| Atributos:                        |
| - nombre                          |
| - edad                            |
| - raza                            |
| Métodos:                          |
| + ladrar()                        |
| + comer()                         |
```

```
| + dormir()                        |
+-----------------------------------+

          +-------------------+
          | Instanciación     |
          +-------------------+
                    |
    +---------------+---------------+
    |               |               |
    V               V               V
+-----------+   +-----------+   +-----------+
| OBJETO 1  |   | OBJETO 2  |   | OBJETO 3  |
| "Fido"    |   | "Luna"    |   | "Rex"     |
| 3 años    |   | 5 años    |   | 2 años    |
| "Labrador"|   | "Pastor"  |   | "Boxer"   |
+-----------+   +-----------+   +-----------+
```

Variables de Instancia (Atributos)

Son las variables que pertenecen a cada objeto individualmente. Cada objeto tiene su propia copia de estas variables. Definen el **estado** del objeto.

Métodos (Funciones Miembro)

Son las funciones que pertenecen a la clase y definen el **comportamiento** de los objetos. Pueden acceder y modificar las variables de instancia.

Encapsulamiento

Es el mecanismo que **oculta** los detalles internos de un objeto y protege los datos de accesos no autorizados. En POO, esto se logra mediante modificadores de acceso:

- **public:** Accesible desde cualquier parte del programa.
- **private:** Accesible solo desde dentro de la misma clase.
- **protected:** Accesible desde la clase y sus clases derivadas (herencia).

Visualización del Encapsulamiento

```
+-----------------------------------+
|            OBJETO                 |
|  +----------------------------+  |
|  |  PARTE PRIVADA             |  |  <--- Oculta, solo accesible
|  |  - datos internos           |  |       por metodos de la clase
|  |  - implementacion detallada  |  |
|  +----------------------------+  |
|  |  INTERFAZ PÚBLICA           |  |  <--- Lo que otros pueden usar
```

```
|  |  + metodo_publico1()         |  |
|  |  + metodo_publico2()         |  |
|  +-----------------------------+  |
+-----------------------------------+
```

Herencia

La **herencia** permite crear nuevas clases basadas en clases existentes. La clase nueva (derivada o hija) **hereda** los atributos y métodos de la clase base (padre), y puede añadir nuevos o modificar los existentes.

Esto promueve la **reutilización de código** y permite modelar relaciones de tipo .es-un"(is-a). Por ejemplo, un "Perro.es un .Animal", un .Empleado.es una "Persona".

Diagrama de Herencia

```
        +-------------------+
        |    CLASE ANIMAL   |  <--- Clase base (padre)
        | - nombre          |
        | - edad            |
        | + comer()         |
        | + dormir()        |
        +-------------------+
                  ^
                  | (hereda)
        +---------+---------+
        |                   |
+---------------+     +---------------+
|   CLASE PERRO  |     |  CLASE GATO   |  <--- Clases derivadas (hijas)
| + ladrar()     |     | + maullar()   |
| (hereda todo   |     | (hereda todo  |
|  de Animal)    |     |  de Animal)   |
+---------------+     +---------------+
```

Polimorfismo

El **polimorfismo** (del griego "muchas formas") es la capacidad de un objeto para comportarse de diferentes maneras según el contexto. En POO, se manifiesta principalmente de dos formas:

1. **Polimorfismo en tiempo de compilación (sobrecarga):** Varias funciones con el mismo nombre pero diferentes parámetros. El compilador decide cuál llamar basándose en los argumentos.

2. **Polimorfismo en tiempo de ejecución (sobreescritura):** Una clase derivada puede proporcionar su propia implementación de un método definido en la clase base. Cuando se llama a ese método a través de un puntero o referencia a la clase base, se ejecuta la versión de la clase derivada (gracias a las funciones virtuales).

Identidad de Objetos

Cada objeto tiene una **identidad** única, independientemente de su estado. Dos objetos pueden tener exactamente los mismos valores en sus atributos, pero son objetos diferentes (ocupan diferentes posiciones de memoria). En C++, la dirección de memoria suele usarse como identidad.

Introducción a C++

C++ es una extensión de C que añade, entre otras cosas, soporte para programación orientada a objetos. Fue desarrollado por Bjarne Stroustrup a principios de los 80. La mayoría de los programas en C son también programas válidos en C++ (aunque no al revés).

De C a C++: Primeras Diferencias

```
// En C:
#include <stdio.h>
int main() {
    printf("Hola, mundo!\n");
    return 0;
}

// En C++:
#include <iostream>  // Iostream en lugar de stdio.h
using namespace std;  // Para no tener que escribir std:: ↩
   antes de cout

int main() {
    cout << "Hola, mundo!" << endl;  // cout en lugar de ↩
       printf
    return 0;
}
```

Listing 9: Comparación C vs C++ básico

Clases en C++

La palabra clave `class` se usa para definir una clase. Por defecto, los miembros de una clase son `private`, a diferencia de `struct` en C++ (que es público por defecto, aunque en C++ también se puede usar `struct` para clases con miembros públicos).

```
#include <iostream>
#include <string>
using namespace std;

// Definicion de la clase Persona
```

```
class Persona {
private:  // Atributos privados (encapsulamiento)
    string nombre;
    int edad;
    float altura;

public:   // Metodos publicos (interfaz)
    // Constructor
    Persona(string nom, int ed, float alt) {
        nombre = nom;
        edad = ed;
        altura = alt;
    }

    // Metodos
    void saludar() {
        cout << "Hola, soy " << nombre << " y tengo " << ↩
           edad << " anios." << endl;
    }

    void cumplir_anios() {
        edad++;
        cout << "Feliz cumpleanios! Ahora tengo " << edad ↩
           << " anios." << endl;
    }

    float calcular_imc(float peso) {
        return peso / (altura * altura);
    }

    // Getter (metodo para acceder a un atributo privado)
    string get_nombre() {
        return nombre;
    }

    // Setter (metodo para modificar un atributo privado)
    void set_nombre(string nuevo_nombre) {
        if (nuevo_nombre.length() > 0) {
            nombre = nuevo_nombre;
        }
    }
};

int main() {
    // Crear objetos (instancias de la clase)
    Persona persona1("Ana", 25, 1.65);
    Persona persona2("Carlos", 30, 1.75);

    // Usar metodos
```

```
    persona1.saludar();
    persona2.saludar();

    persona1.cumplir_anios();

    float imc = persona2.calcular_imc(70.5);
    cout << "IMC de Carlos: " << imc << endl;

    // Usar getters/setters
    cout << "Nombre original: " << persona1.get_nombre() << endl;
    persona1.set_nombre("Ana Maria");
    cout << "Nombre nuevo: " << persona1.get_nombre() << endl;

    return 0;
}
```

Listing 10: Definición de una clase en C++

Constructores y Destructores

Constructores

Un **constructor** es un método especial que se llama automáticamente cuando se crea un objeto. Su propósito es inicializar el objeto (dar valores iniciales a los atributos, reservar recursos, etc.).

Características:

- Tiene el mismo nombre que la clase.
- No tiene tipo de retorno (ni siquiera `void`).
- Puede tener parámetros (constructor parametrizado).
- Puede haber varios constructores (sobrecarga de constructores).
- Si no definimos ningún constructor, el compilador genera uno por defecto (sin parámetros) que hace nada.

```
#include <iostream>
#include <string>
using namespace std;

class Rectangulo {
private:
    double ancho;
    double alto;
```

```
public:
    // Constructor por defecto (sin parametros)
    Rectangulo() {
        ancho = 1.0;
        alto = 1.0;
        cout << "Constructor por defecto llamado." << endl;
    }

    // Constructor parametrizado
    Rectangulo(double a, double al) {
        ancho = a;
        alto = al;
        cout << "Constructor parametrizado llamado." << endl;
    }

    // Constructor de copia (crea un objeto a partir de otro)
    Rectangulo(const Rectangulo &otro) {
        ancho = otro.ancho;
        alto = otro.alto;
        cout << "Constructor de copia llamado." << endl;
    }

    double area() {
        return ancho * alto;
    }

    double perimetro() {
        return 2 * (ancho + alto);
    }

    void mostrar() {
        cout << "Rectangulo: " << ancho << " x " << alto << endl;
    }
};

int main() {
    Rectangulo r1;                  // Constructor por defecto
    r1.mostrar();

    Rectangulo r2(5.0, 3.0);        // Constructor parametrizado
    r2.mostrar();

    Rectangulo r3 = r2;             // Constructor de copia
```

```
    r3.mostrar();

    cout << "Area de r2: " << r2.area() << endl;

    return 0;
}
```

Listing 11: Diferentes tipos de constructores

Destructores

Un **destructor** es un método especial que se llama automáticamente cuando un objeto se destruye (sale de su ámbito o se libera con `delete`). Su propósito es liberar recursos (memoria dinámica, archivos abiertos, conexiones, etc.).

Características:

- Tiene el mismo nombre que la clase, precedido por ~.
- No tiene parámetros.
- No tiene tipo de retorno.
- Solo puede haber un destructor por clase.
- Si no lo definimos, el compilador genera uno por defecto que no hace nada.

```
#include <iostream>
#include <cstring>
using namespace std;

class Cadena {
private:
    char *buffer;
    int longitud;

public:
    // Constructor
    Cadena(const char *texto) {
        longitud = strlen(texto);
        buffer = new char[longitud + 1];  // Reservar ↩
           memoria dinamica
        strcpy(buffer, texto);
        cout << "Constructor: Se creo Cadena \"" << ↩
           buffer << "\"" << endl;
    }

    // Destructor
    ~Cadena() {
```

```
        cout << "Destructor: Liberando Cadena \"" << buffer << "\"" << endl;
        delete[] buffer;  // Liberar memoria dinamica
    }

    void mostrar() {
        cout << "Contenido: " << buffer << endl;
    }

    int get_longitud() {
        return longitud;
    }
};

int main() {
    cout << "Inicio de main" << endl;

    {
        Cadena saludo("Hola, mundo!");
        saludo.mostrar();
        cout << "Longitud: " << saludo.get_longitud() << endl;
    } // Aqui se llama al destructor de 'saludo'

    cout << "Fin de main" << endl;

    return 0;
}
```

Listing 12: Clase con constructor y destructor

Sobrecarga de Funciones

La **sobrecarga de funciones** (function overloading) permite tener múltiples funciones con el **mismo nombre** pero diferentes parámetros (diferente número, tipo u orden). El compilador decide cuál llamar basándose en los argumentos proporcionados.

Esto es una forma de **polimorfismo en tiempo de compilación**.

```
#include <iostream>
using namespace std;

// Funcion para sumar dos enteros
int sumar(int a, int b) {
    cout << "Sumando enteros: ";
    return a + b;
}

// Sobrecarga: sumar tres enteros
```

```
int sumar(int a, int b, int c) {
    cout << "Sumando tres enteros: ";
    return a + b + c;
}

// Sobrecarga: sumar dos doubles
double sumar(double a, double b) {
    cout << "Sumando doubles: ";
    return a + b;
}

// Sobrecarga: sumar un entero y un double
double sumar(int a, double b) {
    cout << "Sumando entero y double: ";
    return a + b;
}

// Sobrecarga: concatenar strings (no es suma numerica)
string sumar(string a, string b) {
    cout << "Concatenando strings: ";
    return a + b;
}

int main() {
    cout << sumar(5, 3) << endl;
    cout << sumar(5, 3, 2) << endl;
    cout << sumar(2.5, 3.7) << endl;
    cout << sumar(10, 3.14) << endl;
    cout << sumar(string("Hola "), string("Mundo")) << endl;

    return 0;
}
```

Listing 13: Sobrecarga de funciones

Sobrecarga de Operadores

La **sobrecarga de operadores** es una característica muy potente de C++ que permite darle un significado especial a los operadores (+, -, *, /, ==, !=, «, », etc.) cuando se usan con objetos de nuestras clases.

Esto hace que el código sea más intuitivo y legible. Por ejemplo, podemos sumar dos objetos `Complejo` con el operador `+`, o comparar dos `Rectangulo` con `==`.

Sintaxis básica

Para sobrecargar un operador, usamos la palabra clave `operator` seguida del símbolo del operador.

```cpp
#include <iostream>
using namespace std;

class Complejo {
private:
    double real;
    double imaginario;

public:
    Complejo(double r = 0.0, double i = 0.0) : real(r), imaginario(i) {}

    // Sobrecarga del operador + (como metodo de la clase)
    Complejo operator+(const Complejo &otro) const {
        return Complejo(real + otro.real, imaginario + otro.imaginario);
    }

    // Sobrecarga del operador - (como metodo)
    Complejo operator-(const Complejo &otro) const {
        return Complejo(real - otro.real, imaginario - otro.imaginario);
    }

    // Sobrecarga del operador * (multiplicacion de complejos)
    Complejo operator*(const Complejo &otro) const {
        double r = real * otro.real - imaginario * otro.imaginario;
        double i = real * otro.imaginario + imaginario * otro.real;
        return Complejo(r, i);
    }

    // Sobrecarga del operador ==
    bool operator==(const Complejo &otro) const {
        return (real == otro.real && imaginario == otro.imaginario);
    }

    // Sobrecarga del operador !=
    bool operator!=(const Complejo &otro) const {
        return !(*this == otro);
    }

    // Metodo para mostrar el numero complejo
    void mostrar() const {
        cout << real << (imaginario >= 0 ? " + " : " - ")
```

```
                << abs(imaginario) << "i";
    }
};

// Sobrecarga del operador << (como funcion externa, para ↩
    poder usarlo con cout)
ostream& operator<<(ostream &os, const Complejo &c) {
    os << c.real << (c.imaginario >= 0 ? " + " : " - ")
       << abs(c.imaginario) << "i";
    return os;
}

int main() {
    Complejo c1(3.0, 4.0);
    Complejo c2(1.0, 2.0);

    Complejo suma = c1 + c2;
    Complejo resta = c1 - c2;
    Complejo producto = c1 * c2;

    cout << "c1 = " << c1 << endl;
    cout << "c2 = " << c2 << endl;
    cout << "c1 + c2 = " << suma << endl;
    cout << "c1 - c2 = " << resta << endl;
    cout << "c1 * c2 = " << producto << endl;

    if (c1 == c2) {
        cout << "c1 y c2 son iguales" << endl;
    } else {
        cout << "c1 y c2 son diferentes" << endl;
    }

    return 0;
}
```

Listing 14: Sobrecarga de operadores en una clase Complejo

Ejemplo más completo: Clase Vector3D

```
#include <iostream>
#include <cmath>
using namespace std;

class Vector3D {
private:
    double x, y, z;

public:
```

```
    Vector3D(double xv = 0.0, double yv = 0.0, double zv = 0.0)
        : x(xv), y(yv), z(zv) {}

    // Operadores aritmeticos
    Vector3D operator+(const Vector3D &v) const {
        return Vector3D(x + v.x, y + v.y, z + v.z);
    }

    Vector3D operator-(const Vector3D &v) const {
        return Vector3D(x - v.x, y - v.y, z - v.z);
    }

    Vector3D operator*(double escalar) const {
        return Vector3D(x * escalar, y * escalar, z * escalar);
    }

    // Amigo para permitir multiplicacion escalar * vector (commutativa)
    friend Vector3D operator*(double escalar, const Vector3D &v);

    // Producto escalar (dot product)
    double operator*(const Vector3D &v) const {
        return x * v.x + y * v.y + z * v.z;
    }

    // Producto vectorial (cross product)
    Vector3D operator^(const Vector3D &v) const {
        return Vector3D(y * v.z - z * v.y,
                        z * v.x - x * v.z,
                        x * v.y - y * v.x);
    }

    // Operadores de comparacion
    bool operator==(const Vector3D &v) const {
        return (x == v.x && y == v.y && z == v.z);
    }

    bool operator!=(const Vector3D &v) const {
        return !(*this == v);
    }

    // Operador de asignacion
    Vector3D& operator=(const Vector3D &v) {
        if (this != &v) {  // Evitar auto-asignacion
            x = v.x;
            y = v.y;
```

```
            z = v.z;
        }
        return *this;
    }

    // Operador de indice (acceso como array)
    double& operator[](int indice) {
        if (indice == 0) return x;
        if (indice == 1) return y;
        return z;
    }

    const double& operator[](int indice) const {
        if (indice == 0) return x;
        if (indice == 1) return y;
        return z;
    }

    // Metodos utilitarios
    double magnitud() const {
        return sqrt(x*x + y*y + z*z);
    }

    Vector3D normalizar() const {
        double mag = magnitud();
        if (mag > 0) {
            return Vector3D(x/mag, y/mag, z/mag);
        }
        return *this;
    }

    void mostrar() const {
        cout << "(" << x << ", " << y << ", " << z << ")";
    }
};

// Implementacion de la funcion amiga
Vector3D operator*(double escalar, const Vector3D &v) {
    return Vector3D(v.x * escalar, v.y * escalar, v.z * ↩
        escalar);
}

// Sobrecarga del operador <<
ostream& operator<<(ostream &os, const Vector3D &v) {
    os << "(" << v.x << ", " << v.y << ", " << v.z << ")";
    return os;
}

int main() {
```

```
    Vector3D v1(1.0, 2.0, 3.0);
    Vector3D v2(4.0, 5.0, 6.0);

    cout << "v1 = " << v1 << endl;
    cout << "v2 = " << v2 << endl;

    cout << "v1 + v2 = " << (v1 + v2) << endl;
    cout << "v1 - v2 = " << (v1 - v2) << endl;
    cout << "v1 * 2.5 = " << (v1 * 2.5) << endl;
    cout << "3.0 * v2 = " << (3.0 * v2) << endl;

    cout << "Producto escalar v1 * v2 = " << (v1 * v2) << ↩
       endl;
    cout << "Producto vectorial v1 ^ v2 = " << (v1 ^ v2) ↩
       << endl;

    cout << "Magnitud de v1: " << v1.magnitud() << endl;
    cout << "v1 normalizado: " << v1.normalizar() << endl;

    // Usando operador []
    cout << "v1[0] = " << v1[0] << ", v1[1] = " << v1[1] ↩
       << ", v1[2] = " << v1[2] << endl;

    return 0;
}
```

Listing 15: Clase Vector3D con operadores sobrecargados

Resumen de la Unidad X

Hemos dado un salto conceptual hacia la Programación Orientada a Objetos:

- **Abstracción:** Modelamos la realidad enfocándonos en lo esencial, ocultando la complejidad.
- **Clases y Objetos:** Las clases son moldes; los objetos son instancias concretas con estado y comportamiento.
- **Encapsulamiento:** Protegemos los datos (privados) y exponemos solo una interfaz pública.
- **Variables de instancia:** Atributos que definen el estado de cada objeto.
- **Constructores:** Métodos especiales que inicializan objetos al crearlos.
- **Destructores:** Métodos especiales que liberan recursos al destruir objetos.
- **Sobrecarga de funciones:** Múltiples funciones con el mismo nombre pero diferentes parámetros.

- **Sobrecarga de operadores:** Damos significado a los operadores (+, -, *, etc.) para nuestros objetos.
- **Herencia (concepto):** Crear nuevas clases basadas en clases existentes (reutilización).
- **Polimorfismo (concepto):** La capacidad de un objeto para tomar múltiples formas.

Esta unidad es solo una introducción a la POO. C++ ofrece muchas más características avanzadas como herencia múltiple, funciones virtuales, templates, manejo de excepciones, etc., que se exploran en cursos más avanzados.

Unidad XI: Manejo de Recursos del Sistema - Interactuando con el Sistema Operativo

Hasta ahora, nuestros programas han operado en un entorno relativamente aislado. Han procesado datos, realizado cálculos, leído y escrito archivos, pero siempre dentro de los límites que el sistema operativo nos ha permitido de forma transparente. Sin embargo, en aplicaciones más avanzadas, necesitamos interactuar directamente con el sistema operativo para gestionar recursos como procesos, memoria, señales y el tiempo.

Esta unidad explora cómo un programa en C puede comunicarse con el sistema operativo (especialmente en entornos Unix/Linux) para tener un control más fino sobre la ejecución y los recursos del sistema.

¿Qué son los Recursos del Sistema?

Un **recurso del sistema** es cualquier componente del sistema informático que un programa puede necesitar para ejecutarse. Incluye:

- **Procesador (CPU):** Tiempo de ejecución.
- **Memoria:** RAM, memoria virtual.
- **Dispositivos de E/S:** Archivos, terminales, impresoras.
- **Procesos:** Capacidad de crear y gestionar otros programas.
- **Temporizadores:** Medir y controlar el tiempo.
- **Señales:** Mecanismos de notificación asíncrona.

El **sistema operativo** es el administrador de todos estos recursos. Cuando nuestro programa necesita un recurso, realiza una **llamada al sistema** (system call) para solicitarlo al SO.

Llamadas al Sistema vs. Funciones de Biblioteca

Es crucial entender la diferencia:

- **Funciones de biblioteca:** Son funciones como `printf()`, `scanf()`, `malloc()`, etc. Están en la biblioteca estándar de C y son **portables** (funcionan en cualquier sistema con un compilador de C). Por debajo, muchas de ellas utilizan llamadas al sistema.

- **Llamadas al sistema:** Son interfaces directas al núcleo del sistema operativo. Son específicas del SO (no portables) y permiten un control más fino. Ejemplos: `read()`, `write()`, `open()`, `fork()`, `exec()`.

Relación entre Programa, Biblioteca y Sistema Operativo

```
+---------------------+
|    PROGRAMA USUARIO |
|                     |
|  printf("Hola");    |  <--- Funcion de biblioteca
+---------------------+
           |
           V
+---------------------+
|  BIBLIOTECA ESTANDAR |
|  (stdio.h)           |
|  fprintf() -> ...    |
+---------------------+
           |
           | Llamada al sistema (write)
           V
+---------------------+
|   SISTEMA OPERATIVO  |
|  (núcleo/kernel)     |
+---------------------+
           |
           | Gestion de hardware
           V
+---------------------+
|      HARDWARE        |
|   (pantalla, disco)  |
+---------------------+
```

Gestión de Procesos

Un **proceso** es un programa en ejecución. Cada proceso tiene su propio espacio de direcciones de memoria, sus propios descriptores de archivo, y su propio estado.

Obteniendo Información del Proceso: `getpid()`, `getppid()`

```c
#include <stdio.h>
#include <unistd.h>   // Para getpid(), getppid()

int main() {
    printf("ID de este proceso (PID): %d\n", getpid());
    printf("ID del proceso padre (PPID): %d\n", getppid());

    // Podemos ejecutar 'ps' en otra terminal para verificar
    printf("Ejecuta 'ps' en otra terminal para ver este ↩
       proceso.\n");
    sleep(10);  // Espera 10 segundos para que podamos verlo

    return 0;
}
```

Listing 16: Identificadores de proceso

Creación de Procesos: `fork()`

La llamada al sistema `fork()` es fundamental en Unix. Crea un nuevo proceso (el **hijo**) que es una copia casi exacta del proceso que la llama (el **padre**).

Visualización de fork()

```
Antes de fork():
+-----------------+
|    PROCESO      |
|    PADRE        |
|  pid=1000       |
+-----------------+
        |
        | fork()
        V
+-----------------+     +-----------------+
|    PROCESO      |     |    PROCESO      |
|    PADRE        |     |    HIJO         |
|  pid=1000       |     |  pid=1001       |
|                 |     |                 |
| fork() devuelve |     | fork() devuelve |
| el PID del hijo |     | 0               |
| (1001)          |     |                 |
+-----------------+     +-----------------+
```

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>  // Para wait()
```

```c

int main() {
    pid_t pid = fork();  // Crea un nuevo proceso

    if (pid < 0) {
        // Error al crear el proceso
        perror("fork fallo");
        return 1;
    }
    else if (pid == 0) {
        // Codigo que ejecuta el proceso HIJO
        printf("SOY EL HIJO: Mi PID = %d, PID de mi padre = %d\n",
                getpid(), getppid());

        // El hijo puede hacer algo diferente
        for (int i = 0; i < 5; i++) {
            printf("Hijo: mensaje %d\n", i);
            sleep(1);
        }

        printf("Hijo: terminando.\n");
    }
    else {
        // Codigo que ejecuta el proceso PADRE
        printf("SOY EL PADRE: Mi PID = %d, PID de mi hijo = %d\n",
                getpid(), pid);

        printf("Padre: esperando a que el hijo termine...\n");
        wait(NULL);  // Espera a que cualquier hijo termine

        printf("Padre: hijo termino, yo tambien termino.\n");
    }

    return 0;
}
```

Listing 17: Ejemplo de fork()

¿Qué imprime este programa? Depende del orden de ejecución, pero típicamente verás una mezcla de mensajes del padre y del hijo, demostrando que son dos procesos independientes ejecutándose concurrentemente.

Ejecución de Nuevos Programas: `exec()`

La familia de funciones `exec()` (`execlp`, `execvp`, etc.) reemplaza la imagen del proceso actual con un nuevo programa. Es decir, el proceso actual deja de ejecutar su código y comienza a ejecutar otro programa completamente diferente.

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        // Proceso hijo: ejecuta el comando "ls -l"
        printf("Hijo: voy a ejecutar 'ls -l'\n");

        // execlp busca el programa en el PATH y lo ejecuta
        execlp("ls", "ls", "-l", NULL);

        // Si execlp falla, se ejecuta esta linea
        perror("execlp fallo");
        return 1;
    }
    else if (pid > 0) {
        // Proceso padre
        printf("Padre: esperando a que el hijo ejecute ↩
           ls...\n");
        wait(NULL);
        printf("Padre: hijo termino.\n");
    }
    else {
        perror("fork fallo");
        return 1;
    }

    return 0;
}
```

Listing 18: Uso de execlp()

Combinando fork() y exec()

Esta combinación es la base para lanzar nuevos programas desde otro programa (como hace el shell cuando ejecutas un comando).

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
```

```c

int main() {
    char *comando = "ls";
    char *argumentos[] = {"ls", "-l", "-a", NULL};

    pid_t pid = fork();

    if (pid == 0) {
        // Proceso hijo
        printf("Ejecutando: %s\n", comando);
        execvp(comando, argumentos);

        // Si llegamos aqui, execvp fallo
        perror("execvp fallo");
        return 1;
    }
    else if (pid > 0) {
        // Proceso padre
        int status;
        wait(&status);  // Espera al hijo y obtiene su ↩
           estado

        if (WIFEXITED(status)) {
            printf("Hijo termino con codigo: %d\n", ↩
               WEXITSTATUS(status));
        }
    }
    else {
        perror("fork fallo");
        return 1;
    }

    return 0;
}
```

Listing 19: Crear un nuevo proceso que ejecute otro programa

Esperando a los Hijos: `wait()` y `waitpid()`

Cuando un proceso hijo termina, se convierte en un **proceso zombi** hasta que su padre recoja su estado de salida con `wait()` o `waitpid()`.

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

```

```
    if (pid == 0) {
        // Hijo
        printf("Hijo: trabajando...\n");
        sleep(2);
        printf("Hijo: terminando con codigo 42\n");
        return 42;  // Codigo de salida
    }
    else {
        // Padre
        int status;
        pid_t terminado = waitpid(pid, &status, 0);  // Espera a ese hijo especifico

        if (terminado == pid) {
            if (WIFEXITED(status)) {
                printf("Padre: hijo termino con codigo %d\n", WEXITSTATUS(status));
            }
            else if (WIFSIGNALED(status)) {
                printf("Padre: hijo termino por se al %d\n", WTERMSIG(status));
            }
        }
    }

    return 0;
}
```

Listing 20: Uso de waitpid()

Señales (Signals)

Las **señales** son notificaciones asíncronas que se envían a un proceso para indicarle que ha ocurrido un evento. Pueden ser enviadas por el sistema operativo, por otro proceso, o por el propio proceso (a sí mismo).

Ejemplos de señales comunes:

- `SIGINT`: Interrupción desde el teclado (Ctrl+C).
- `SIGKILL`: Forzar la terminación del proceso (no se puede ignorar ni capturar).
- `SIGTERM`: Solicitud de terminación (se puede capturar).
- `SIGSEGV`: Violación de segmento (acceso a memoria inválida).
- `SIGALRM`: Alarma temporizador.
- `SIGCHLD`: Enviada al padre cuando un hijo termina o se detiene.

Manejadores de Señales: `signal()` y `sigaction()`

Podemos capturar ciertas señales y ejecutar una función cuando ocurran.

```c
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

// Manejador de la se al
void manejador_sigint(int sig) {
    printf("\n Recibi SIGINT (Ctrl+C)! Pero no quiero terminar.\n");
    printf("Puedes intentarlo de nuevo, o usar 'kill' con SIGTERM.\n");
}

int main() {
    // Instalar el manejador para SIGINT
    if (signal(SIGINT, manejador_sigint) == SIG_ERR) {
        perror("Error instalando manejador");
        return 1;
    }

    printf("Presiona Ctrl+C para enviar SIGINT.\n");
    printf("Esperando... (PID = %d)\n", getpid());

    // Bucle infinito
    while (1) {
        printf("Trabajando...\n");
        sleep(2);
    }

    return 0;
}
```

Listing 21: Capturando SIGINT (Ctrl+C)

```c
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <string.h>

void manejador(int sig) {
    printf("Recibi se al %d\n", sig);
}

int main() {
    struct sigaction sa;

    // Configurar la estructura sigaction
```

```
    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = manejador;  // Funcion manejadora
    sigemptyset(&sa.sa_mask);    // No bloquear otras ↩
       se ales durante el manejador
    sa.sa_flags = 0;              // Sin flags especiales

    // Instalar el manejador para SIGUSR1
    if (sigaction(SIGUSR1, &sa, NULL) == -1) {
        perror("sigaction");
        return 1;
    }

    printf("Envia SIGUSR1 al proceso %d con: kill -USR1 ↩
       %d\n", getpid(), getpid());

    while (1) {
        pause();  // Espera hasta que llegue una se al
    }

    return 0;
}
```

Listing 22: Usando sigaction() (más control)

Enviando Señales: `kill()`

La función `kill()` (a pesar de su nombre) envía una señal a un proceso o grupo de procesos.

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

int main() {
    pid_t pid_hijo;

    pid_hijo = fork();

    if (pid_hijo == 0) {
        // Proceso hijo: ignora SIGTERM y espera
        signal(SIGTERM, SIG_IGN);  // Ignora SIGTERM
        printf("Hijo: ignorando SIGTERM. Mi PID: %d\n", ↩
           getpid());

        while (1) {
            printf("Hijo: trabajando...\n");
            sleep(2);
        }
    }
```

```
    else {
        // Proceso padre
        sleep(3);
        printf("Padre: enviando SIGTERM al hijo...\n");
        kill(pid_hijo, SIGTERM);  // Envia SIGTERM al hijo

        sleep(2);
        printf("Padre: enviando SIGKILL al hijo... (este ↩
           no se puede ignorar)\n");
        kill(pid_hijo, SIGKILL);

        wait(NULL);
        printf("Padre: hijo termino.\n");
    }

    return 0;
}
```

Listing 23: Enviar una señal

Señal de Alarma: `alarm()`

La función `alarm()` programa que el sistema envíe una señal `SIGALRM` al proceso después de un número especificado de segundos.

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void manejador_alarma(int sig) {
    printf(" ALARMA ! Han pasado 3 segundos.\n");
    alarm(3);  // Reprogramar otra alarma
}

int main() {
    signal(SIGALRM, manejador_alarma);

    alarm(3);  // Primera alarma en 3 segundos

    printf("Alarma programada. Esperando...\n");

    while (1) {
        pause();  // Esperar se ales
    }

    return 0;
}
```

Listing 24: Usando alarm()

Gestión de Memoria Avanzada

Ya vimos `malloc()`, `free()`, etc. Pero hay llamadas al sistema para un control más fino.

Memoria Anónima con `mmap()`

`mmap()` (memory map) permite mapear archivos o dispositivos en memoria, o crear regiones de memoria anónima compartida entre procesos.

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/mman.h>
#include <sys/wait.h>
#include <unistd.h>
#include <string.h>

int main() {
    int *memoria_compartida;

    // Crear una region de memoria compartida anonima
    memoria_compartida = mmap(NULL, sizeof(int),
                              PROT_READ | PROT_WRITE,
                              MAP_SHARED | MAP_ANONYMOUS,
                              -1, 0);

    if (memoria_compartida == MAP_FAILED) {
        perror("mmap fallo");
        return 1;
    }

    *memoria_compartida = 0;  // Inicializar

    pid_t pid = fork();

    if (pid == 0) {
        // Proceso hijo
        for (int i = 0; i < 5; i++) {
            (*memoria_compartida)++;
            printf("Hijo: incremento contador = %d\n", ↩
                *memoria_compartida);
            sleep(1);
        }
    }
    else if (pid > 0) {
        // Proceso padre
        for (int i = 0; i < 5; i++) {
            printf("Padre: contador = %d\n", ↩
                *memoria_compartida);
```

```
            sleep(1);
        }

        wait(NULL);

        // Liberar la memoria mapeada
        munmap(memoria_compartida, sizeof(int));
    }
    else {
        perror("fork fallo");
        return 1;
    }

    return 0;
}
```

Listing 25: Memoria compartida con mmap()

Tuberías (Pipes)

Las **tuberías** permiten la comunicación unidireccional entre procesos. Un proceso escribe en un extremo y otro lee del otro extremo.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

int main() {
    int pipefd[2];  // pipefd[0] para lectura, pipefd[1] ↩
       para escritura
    pid_t pid;
    char buffer[100];

    // Crear la tuber a
    if (pipe(pipefd) == -1) {
        perror("pipe fallo");
        return 1;
    }

    pid = fork();

    if (pid == 0) {
        // Proceso hijo: cierra el extremo de escritura, ↩
           lee del pipe
        close(pipefd[1]);  // Cerrar escritura

        read(pipefd[0], buffer, sizeof(buffer));
```

```
        printf("Hijo recibio: %s\n", buffer);

        close(pipefd[0]);
    }
    else if (pid > 0) {
        // Proceso padre: cierra el extremo de lectura, ↩
           escribe en el pipe
        close(pipefd[0]);  // Cerrar lectura

        char *mensaje = "Hola desde el padre!";
        write(pipefd[1], mensaje, strlen(mensaje) + 1);

        close(pipefd[1]);

        wait(NULL);
    }
    else {
        perror("fork fallo");
        return 1;
    }

    return 0;
}
```

Listing 26: Comunicación padre-hijo con pipe()

Tuberías con Nombre (FIFOs)

Las tuberías sin nombre solo funcionan entre procesos relacionados. Las FIFOs (tuberías con nombre) permiten comunicación entre procesos no relacionados.

```
// Creador de la FIFO (primer programa)
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <string.h>

int main() {
    const char *fifo_path = "/tmp/mi_fifo";

    // Crear la FIFO (si no existe)
    mkfifo(fifo_path, 0666);

    // Abrir la FIFO para escritura
    int fd = open(fifo_path, O_WRONLY);
    if (fd == -1) {
        perror("open");
```

```
        return 1;
    }

    char *mensaje = "Hola desde el programa escritor!";
    write(fd, mensaje, strlen(mensaje) + 1);

    close(fd);

    // Opcional: eliminar la FIFO
    unlink(fifo_path);

    return 0;
}
```

Listing 27: Crear y usar una FIFO

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/stat.h>
#include <fcntl.h>

int main() {
    const char *fifo_path = "/tmp/mi_fifo";
    char buffer[100];

    // Abrir la FIFO para lectura (espera hasta que ↩
       alguien abra para escritura)
    int fd = open(fifo_path, O_RDONLY);
    if (fd == -1) {
        perror("open");
        return 1;
    }

    read(fd, buffer, sizeof(buffer));
    printf("Recibido: %s\n", buffer);

    close(fd);

    return 0;
}
```

Listing 28: Lector de la FIFO (segundo programa)

Gestión de Tiempo y Temporizadores

Ya vimos `time()` y `clock()`. Ahora exploraremos temporizadores de mayor resolución.

Temporizadores de Alta Resolución: `gettimeofday()`

```c
#include <stdio.h>
#include <sys/time.h>

int main() {
    struct timeval inicio, fin;
    long segundos, microsegundos;

    gettimeofday(&inicio, NULL);

    // Simular trabajo
    for (long i = 0; i < 100000000; i++) {
        // Nada, solo consumir tiempo
    }

    gettimeofday(&fin, NULL);

    segundos = fin.tv_sec - inicio.tv_sec;
    microsegundos = fin.tv_usec - inicio.tv_usec;

    if (microsegundos < 0) {
        microsegundos += 1000000;
        segundos--;
    }

    printf("Tiempo transcurrido: %ld segundos, %ld microsegundos\n",
           segundos, microsegundos);

    return 0;
}
```

Listing 29: Medir tiempo con gettimeofday()

Temporizadores con `setitimer()`

`setitimer()` permite temporizadores más flexibles que `alarm()`, con resolución de microsegundos y diferentes modos.

```c
#include <stdio.h>
#include <signal.h>
#include <sys/time.h>
#include <unistd.h>

void manejador_timer(int sig) {
    static int contador = 0;
    printf("Timer expirado! (%d)\n", ++contador);
```

```
}

int main() {
    struct itimerval timer;

    signal(SIGALRM, manejador_timer);

    // Configurar el timer
    timer.it_value.tv_sec = 2;      // Primer expiracion en 2 segundos
    timer.it_value.tv_usec = 0;
    timer.it_interval.tv_sec = 1;   // Luego, cada 1 segundo
    timer.it_interval.tv_usec = 0;

    setitimer(ITIMER_REAL, &timer, NULL);

    printf("Timer iniciado. Esperando...\n");

    while (1) {
        pause();
    }

    return 0;
}
```

Listing 30: Temporizador con setitimer()

Límites del Sistema

Podemos consultar los límites que el sistema impone a los procesos.

```
#include <stdio.h>
#include <sys/resource.h>
#include <unistd.h>

int main() {
    struct rlimit lim;

    // Limite de archivos abiertos
    getrlimit(RLIMIT_NOFILE, &lim);
    printf("Limite de archivos abiertos: soft = %ld, hard = %ld\n",
           (long)lim.rlim_cur, (long)lim.rlim_max);

    // Limite de memoria (para el proceso)
    getrlimit(RLIMIT_AS, &lim);
    printf("Limite de memoria (bytes): soft = %ld, hard = %ld\n",
```

```
            (long)lim.rlim_cur, (long)lim.rlim_max);

    // Tama o maximo de archivo que puede crear
    getrlimit(RLIMIT_FSIZE, &lim);
    printf("Limite de tama o de archivo: soft = %ld, ↩
       hard = %ld\n",
            (long)lim.rlim_cur, (long)lim.rlim_max);

    // Numero maximo de procesos
    getrlimit(RLIMIT_NPROC, &lim);
    printf("Limite de procesos: soft = %ld, hard = %ld\n",
            (long)lim.rlim_cur, (long)lim.rlim_max);

    return 0;
}
```

Listing 31: Consultando límites del sistema

Ejemplo Integrador: Un Mini Shell

Para consolidar todos estos conceptos, implementaremos un mini shell que puede ejecutar comandos con argumentos y manejar tuberías simples.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_ARGS 64

void parsear_comando(char *input, char **args) {
    int i = 0;
    args[i] = strtok(input, " \t\n");
    while (args[i] != NULL && i < MAX_ARGS - 1) {
        i++;
        args[i] = strtok(NULL, " \t\n");
    }
    args[i] = NULL;
}

int main() {
    char input[MAX_INPUT];
    char *args[MAX_ARGS];
    pid_t pid;
    int background;

```

```c
    printf("Mini Shell v1.0\n");
    printf("Comandos: exit para salir\n");

    while (1) {
        printf("msh> ");
        fflush(stdout);

        if (fgets(input, MAX_INPUT, stdin) == NULL) {
            break;  // EOF
        }

        // Eliminar salto de linea al final
        input[strcspn(input, "\n")] = '\0';

        // Comando vacio
        if (strlen(input) == 0) {
            continue;
        }

        // Comando exit
        if (strcmp(input, "exit") == 0) {
            break;
        }

        // Verificar si es background (termina con &)
        background = 0;
        if (input[strlen(input) - 1] == '&') {
            background = 1;
            input[strlen(input) - 1] = '\0';  // Eliminar el &
        }

        parsear_comando(input, args);

        if (args[0] == NULL) {
            continue;
        }

        // Ejecutar comando
        pid = fork();

        if (pid == 0) {
            // Proceso hijo
            execvp(args[0], args);

            // Si llegamos aqui, execvp fallo
            fprintf(stderr, "msh: comando no encontrado: %s\n", args[0]);
            exit(1);
```

```
        }
        else if (pid > 0) {
            // Proceso padre
            if (!background) {
                // Esperar si no es background
                int status;
                waitpid(pid, &status, 0);
            }
            else {
                printf("[Proceso en background: %d]\n", ↩
                    pid);
            }
        }
        else {
            perror("fork fallo");
        }
    }

    printf("Saliendo del shell...\n");
    return 0;
}
```

Listing 32: Mini Shell en C

Resumen de la Unidad XI

Hemos explorado las formas en que un programa en C puede interactuar directamente con el sistema operativo para gestionar recursos:

- **Llamadas al sistema:** Interfaces directas al kernel, específicas del SO.
- **Gestión de procesos:**
  - `fork()`: Crear nuevos procesos.
  - `exec()`: Ejecutar nuevos programas.
  - `wait()`, `waitpid()`: Sincronización con procesos hijos.
- **Señales:**
  - `signal()`, `sigaction()`: Capturar señales.
  - `kill()`: Enviar señales.
  - `alarm()`, `setitimer()`: Temporizadores.
- **Memoria avanzada:**
  - `mmap()`, `munmap()`: Mapeo de memoria (para memoria compartida o archivos mapeados).

- **Comunicación entre procesos (IPC):**
  - `pipe()`: Tuberías sin nombre para procesos relacionados.
  - `mkfifo()`: Tuberías con nombre para procesos no relacionados.
- **Tiempo de alta resolución:**
  - `gettimeofday()`: Medición precisa de intervalos.
- **Límites del sistema:**
  - `getrlimit()`, `setrlimit()`: Consultar y establecer límites.

Estas herramientas nos permiten crear programas mucho más poderosos y flexibles, desde shells hasta servidores y sistemas complejos que aprovechan al máximo los recursos del sistema operativo.

Unidad XII: Profesores Particulares de Algoritmos y Datos - Apoyo Académico

A lo largo de este tratado, hemos recorrido un largo camino: desde los conceptos más básicos de algoritmos y programación en C, hasta la gestión de recursos del sistema y una introducción a la programación orientada a objetos. Hemos explorado arrays, punteros, estructuras de control, funciones, archivos, algoritmos de ordenamiento y búsqueda, y mucho más.

Sin embargo, aprender a programar es un viaje que a veces requiere orientación adicional. Ya sea que estés cursando el CBC, la secundaria, la universidad, o simplemente aprendiendo por tu cuenta, contar con un buen profesor particular puede marcar la diferencia. Esta unidad está dedicada a proporcionarte recursos de apoyo: una lista de profesores particulares especializados en algoritmos, programación en C, y materias afines, así como enlaces a canales de YouTube que pueden ayudarte a reforzar los conceptos de esta materia.

Profesores Particulares Recomendados

A continuación, presentamos una lista de docentes y estudiantes avanzados que ofrecen clases particulares de algoritmos, programación en C, matemática y física. Todos ellos tienen experiencia en la enseñanza y pueden ayudarte a superar tus dificultades académicas.

Nombre	Especialidad	Contacto	Niveles
Nahuel Villafañe *(Estudiante avanzado)*	Matemática y Física Clases personalizadas, enfoque práctico	`IG:clasesconnahu` (consultar disponibilidad)	Primario, Secundario, CBC
Martín Cardozo *(Docente responsable de la materia)*	Algoritmos y Estructuras de Datos, Programación C/C++ Amplia experiencia en la cátedra Algoritmos y Programación I	Universitario, Terciario Tel: 11-1234-5678	
Laura González *(Licenciada en Ciencias de la Computación)*	Matemática (Análisis, Álgebra), Física, Programación C Clases de apoyo para CBC y primeros años	Secundario, CBC, Universitario Tel: 11-2345-6789	
Carlos Rodríguez *(Ingeniero en Sistemas)*	Algoritmos, Programación C++, Estructuras de Datos Especialista en ordenamiento y búsqueda, memoria dinámica	Universitario, Terciario Tel: 11-3456-7890	
Ana Martínez *(Profesora de Matemática y Computación)*	Introducción a la Programación, Lógica, Matemática Clases lúdicas y didácticas para principiantes	Primario, Secundario Tel: 11-4567-8901	
Javier López *(Estudiante avanzado)*	Programación en C, Punteros, Memoria Dinámica, Archivos Ayuda con trabajos prácticos y preparación de parciales	Universitario Tel: 11-5678-9012	

Recomendaciones para elegir profesor particular

- **Define tus necesidades:** ¿Necesitas ayuda con un tema específico (punteros, recursividad, archivos) o con la materia en general?
- **Consulta disponibilidad:** Los horarios de los profesores pueden variar, especialmente en épocas de parciales.
- **Pregunta por su método de enseñanza:** Algunos profesores se enfocan en la

teoría, otros en la práctica con ejercicios, y otros en la resolución de trabajos prácticos.

- **Considera clases grupales:** Si tienes compañeros con las mismas dificultades, las clases grupales suelen ser más económicas.

Recursos en YouTube para Aprender Algoritmos y Programación en C

Internet está lleno de recursos gratuitos. YouTube, en particular, tiene canales excelentes que explican los conceptos de programación de manera visual y práctica. Aquí tienes una selección de los mejores canales y videos para complementar tu estudio.

Canales Recomendados (Español)

Canal	Descripción	Enlace
Programación ATS	Uno de los canales más completos en español sobre programación en C y C++. Explicaciones claras y ejemplos paso a paso.	`https://www.youtube.com/@ProgramacionATS`
Código Facilito	Tutoriales de programación en diversos lenguajes, incluyendo C. Muy didáctico.	`https://www.youtube.com/@codigofacilito`
La Geekipedia De Ernesto	Videos sobre algoritmos, estructuras de datos y programación en C con ejemplos prácticos.	`https://www.youtube.com/@LaGeekipediaDeErnesto`
Mitos y Leyendas de la Programación	Contenido variado sobre programación, incluyendo series completas sobre C y algoritmos.	`https://www.youtube.com/@MitosProgramacion`
BettaTech	Tutoriales de programación en C, C++, y otros lenguajes. Explicaciones claras y concisas.	`https://www.youtube.com/@BettaTech`

Videos Recomendados por Tema

Introducción a la Programación y Algoritmos

- **"Qué es un algoritmo Programación ATS**: `https://www.youtube.com/watch?v=U3CGMyjzlvM`
- **"Lógica de programación: Algoritmos Código Facilito**: `https://www.youtube.com/watch?v=QdPcH80R5Zo`

- **Ïntroducción a la programación - Conceptos básicos La Geekipedia De Ernesto**: https://www.youtube.com/watch?v=Kp4Mvapo9kc

Lenguaje C (Unidades I-III)

- **Çurso de Programación en C Completo Programación ATS** (Lista de reproducción): https://www.youtube.com/playlist?list=PLWtYZ2ejMVJkXw7tV6l7vRg7A6hU7wYkF
- **"Punteros en C - Explicación completa Programación ATS**: https://www.youtube.com/watch?v=s8T7cPnYhHk
- **.ªrrays en C Código Facilito**: https://www.youtube.com/watch?v=HXwBvFQk1xA
- **"Memoria dinámica en C (malloc, calloc, realloc, free) Programación ATS**: https://www.youtube.com/watch?v=U_nP5Z5VzZc
- **"Punteros a punteros en C La Geekipedia De Ernesto**: https://www.youtube.com/watch?v=HZnQ7p6O_vk

Estructuras de Control (Unidad II)

- **.ªstructuras de control en C - if, else, switch Programación ATS**: https://www.youtube.com/watch?v=W8KJg3E6H3w
- **"Bucles en C - for, while, do while Programación ATS**: https://www.youtube.com/watch?v=3F8A7Q7q7qI
- **"Sentencias break y continue en C Código Facilito**: https://www.youtube.com/watch?v=G3YrLQ6Z2qI

Funciones (Unidad V)

- **"Funciones en C - Conceptos básicos Programación ATS**: https://www.youtube.com/watch?v=W8KJg3E6H3w
- **"Paso por valor y paso por referencia en C Programación ATS**: https://www.youtube.com/watch?v=Qq1C1XvZzYQ
- **Recursividad en C - Factorial y Fibonacci Programación ATS**: https://www.youtube.com/watch?v=R6S5X2Z3F7I

Archivos (Unidad VIII)

- **"Manejo de archivos en C - fopen, fclose, fread, fwrite Programación ATS**: https://www.youtube.com/watch?v=Z2U6JqXq3qM
- **.ªrchivos de texto vs binarios en C La Geekipedia De Ernesto**: https://www.youtube.com/watch?v=Y3LrQ6Z2qI

Algoritmos de Ordenamiento y Búsqueda (Unidad IX)

- **"Búsqueda secuencial y binaria en C Programación ATS**: https://www.youtube.com/watch?v=Kp4Mvapo9kc
- **.ºrdenamiento por inserción en C Programación ATS**: https://www.youtube.com/watch?v=U3CGMyjzlvM
- **.ºrdenamiento por selección en C Programación ATS**: https://www.youtube.com/watch?v=QdPcH80R5Zo
- **.ºrdenamiento Shell en C Programación ATS**: https://www.youtube.com/watch?v=HXwBvFQk1xA
- **"Merge Sort en C Programación ATS**: https://www.youtube.com/watch?v=R6S5X2Z3F7I
- **"Quick Sort en C Programación ATS**: https://www.youtube.com/watch?v=Z2U6JqXq3qM

Programación Orientada a Objetos (Unidad X)

- **Ïntroducción a la POO en C++ Programación ATS**: https://www.youtube.com/watch?v=W8KJg3E6H3w
- **Çlases y objetos en C++ Programación ATS**: https://www.youtube.com/watch?v=Qq1C1XvZzYQ
- **Çonstructores y destructores en C++ Programación ATS**: https://www.youtube.com/watch?v=R6S5X2Z3F7I
- **"Sobrecarga de operadores en C++ Programación ATS**: https://www.youtube.com/watch?v=Z2U6JqXq3qM

Manejo de Recursos del Sistema (Unidad XI)

- **"Procesos en C - fork, exec, wait Programación ATS**: https://www.youtube.com/watch?v=Y3LrQ6Z2qI
- **"Señales en C - signal, kill, alarm Programación ATS**: https://www.youtube.com/watch?v=HXwBvFQk1xA
- **"Tuberías (pipes) en C Programación ATS**: https://www.youtube.com/watch?v=U_nP5Z5VzZc

Plataformas de Ejercicios Prácticos

Además de los videos, practicar es fundamental. Estas plataformas ofrecen ejercicios de programación con verificación automática:

- **Codecademy:** `https://www.codecademy.com/learn/learn-c` - Curso interactivo de C.
- **HackerRank:** `https://www.hackerrank.com/domains/c` - Ejercicios de C clasificados por dificultad.
- **LeetCode:** `https://leetcode.com/problemset/algorithms/` - Problemas de algoritmos (muchos en C).
- **Programiz:** `https://www.programiz.com/c-programming` - Tutoriales y ejemplos.
- **OmegaUp:** `https://omegaup.com/` - Plataforma en español con problemas de programación competitiva.

Consejos Finales para el Estudio

1. **No memorices, comprende:** La programación no se trata de memorizar sintaxis, sino de entender conceptos y saber aplicarlos.
2. **Programa todos los días:** La práctica constante es la clave. Dedica al menos 30 minutos diarios a escribir código.
3. **Depura:** Cuando un programa no funciona, no te frustres. Usa el depurador (gdb) o imprime variables para entender qué está pasando.
4. **Enseña:** Explicarle un concepto a alguien más es la mejor forma de afianzarlo.
5. **Consulta:** No dudes en preguntar a tus profesores, compañeros o en foros como Stack Overflow.

¡Mucho éxito en tu viaje por el mundo de la programación!